

SYLION AEIS

Implementation Masterplan · LEGO Architecture, Production-First · v3.5

NADRZĘDNA TEZA MASTERPLANU (verbatim)

„Budujemy pełny SYLION od początku jako system modułowy oparty o kontrakty. Security istnieje od pierwszego dnia jako zestaw wymienialnych modułów, ale na początku działa w profilach uproszczonych. Docelowe utwardzenie następuje przez podmianę implementacji, a nie przez przebudowę architektury.”

PIĄTA META-ZASADA (NOWA)

Modularity by Contract

SYLION MUSI być budowany jako zbiór wymienialnych klocków (modułów) rozmawiających przez zamrożone kontrakty, nie przez współdzielone struktury. Klocki można wyjmować, dokładać, podmieniać i testować równolegle — bez znaczącego wpływu na resztę systemu. Zamrożone są: manifest schema, contract registry, event taxonomy, module lifecycle, dependency rules, decision boundaries i security profile abstraction.

Dokument	SYLION AEIS — Implementation Masterplan v3.5
Status	Plan wykonawczy · komplementarny wobec Dokumentacji v3.5.1
Rodzic kanoniczny	SYLION AEIS Dokumentacja v3.5.1 (Pipeline 8.5.0, MZ 3.5.5)
Architektura	LEGO · Kernel 8 + 6 klas modułów + ~65 klocków
Kontrakty	gRPC + Protobuf + buf (breaking detection)
Runtime	Docker + docker-compose → K8s (M5)
Core	Python 3.12 + FastAPI + Pydantic v2
Storage	PostgreSQL 16 + JSONB + pgvector

Events	NATS JetStream · idempotent · event taxonomy zamrożona
Security	4 profile (dev-light / test-light / staging-strict / prod-strict)
Milestones	M0–M5 · 20 sprintów · ~12 miesięcy
Autonomy	5 etapów: observe → propose → sandbox → limited-prod → full-governed
Checklists	20 planów wykonawczych P01–P20 (1 na klocek logiczny)
Meta-zasady	Rebuildability · Reversibility · Efficiency · Autonomy · Modularity by Contract
Data	kwiecień 2026
Autor	Perplexity Computer

Abstrakt Masterplanu

Niniejszy dokument to plan wykonawczy budowy pełnego systemu SYLION AEIS od zera w architekturze modułowej (LEGO) opartej o zamrożone kontrakty. Nie jest to MVP ani prototyp — celem jest produkcyjny system gotowy do wdrożenia pod koniec M5, z Kernelem 8 modułów jako niewymienialnym rdzeniem oraz ~65 klockami w 12 klasach (6 podstawowych + 6 rozszerzających) rozwijanymi przez 8 strumieni równoległych w 20 sprintach.

Dokumentacja kanoniczna v3.5.1 (zgodna z SYLION AEIS Pipeline 8.5.0, MZ 3.5.5) definiuje CO budujemy — zasady, drabinę D0–D5, cztery meta-zasady (Rebuildability / Reversibility / Efficiency / Autonomy under Canon), 20 planów kanonicznych i 18 aneksów. Masterplan definiuje JAK to zbudować: rdzeń 8 modułów, 6 klas modułów, 4 centralne rejestry, 7 zamrożonych artefaktów (Contract Freeze Milestone), 5 faz Integration Assembly, 5 etapów Autonomy Rollout, 4 profile security z jednym kontraktem, 20 execution checklists i 12 ryzyk z mitygacjami.

Masterplan proponuje piątą meta-zasadę „Modularity by Contract” jako nowy wpis do kanonu v3.5 (zaplanowana edycja dokumentacji w R0.7). Reguła: system MUSI być budowany jako zbiór wymienialnych klocków rozmawiających przez kontrakty, nie przez współdzielone struktury. Klocki można wyjmować, dokładać, podmieniać i testować równolegle. Security jest zestawem 8 takich klocków — od pierwszego dnia istnieje jako kontrakty, ale na początku działa w profilach uproszczonych (dev-light, test-light). Utwardzenie do profili strict następuje przez podmianę implementacji modułu, nie przez przebudowę architektury. Żaden moduł biznesowy nie zmienia się między dev a prod.

Dokument ma strukturę 10 rozdziałów R0–R9 (~130 stron). R0 to executive overview (5 meta-zasad, 10 kluczowych decyzji, mapa rozdziałów, 6 proponowanych zmian w v3.5). R1 to architektura Kernel + 6 klas + katalog ~65 modułów. R2 to Contract Freeze Milestone (7 rzeczy do zamrożenia przed startem M1). R3 to tabela główna 65 modułów × wszystkie atrybuty. R4 to critical path i 8 strumieni równoległych. R5 to milestones M0–M5 szczegółowo. R6 to 20 execution checklists P01–P20. R7 to autonomy rollout 5 etapów. R8 to security hardening przez swap, nie rebuild, + Integration Assembly Phases. R9 to top 12 ryzyk, Evidence Pack schema i post-M5.

Masterplan jest dokumentem operacyjnym dla wszystkich zespołów realizujących implementację. Każdy klocek ma właściciela (plan wykonawczy P01–P20), kontrakt (.proto zamrożony przed implementacją), testy (golden set dla kontraktu), politykę lifecycle (draft → build → validate → shadow → dual → cutover → stable → deprecated) oraz Evidence Pack dla każdej decyzji D3+. Cięcie M0 → M1 to Contract Freeze — po nim kontrakty zmieniają się wyłącznie przez plan migracyjny z buf breaking detection i Council aproval.

Spis rozdziałów

Masterplan jest złożony z dziesięciu rozdziałów (R0–R9). Rozdział R0 ma inny kolor baneru (MASTERCANON, ciemny fiolet) bo jest mostem do kanonu v3.5 — zawiera sześć propozycji zmian w dokumentacji kanonicznej. Pozostałe rozdziały używają baneru MASTERPLAN (turkus).

R0	Executive Overview	5 meta-zasad, 10 kluczowych decyzji, mapa rozdziałów, 12 klas modułów, 6 zmian w v3.5
R1	Kernel & Module Architecture	Rdzeń 8 modułów (niewymienialne), 6 klas modułów, katalog ~65, plug-in/plugin-out protocol
R2	Contract Freeze Milestone	7 zamrożonych artefaktów: manifest, contracts, events, lifecycle, deps, D-class, security
R3	Tabela główna 65 modułów	12 klas × 3 tabele per klasa: atrybuty / DoD + bramy / profil security M0–M5
R4	Critical Path + 8 strumieni równoległych	15 modułów krytycznych, diagram sprint 1–20, synchronization points, dependency graph
R5	Milestones M0–M5	M0 Foundation · M1 Kernel Frozen · M2 Core Online · M3 Governance · M4 Autonomy · M5 Prod
R6	Execution Checklists P01–P20	20 planów wykonawczych: moduły, kontrakty, tabele, endpointy, golden set, bramy
R7	Autonomy Rollout (5 etapów)	observe → propose → sandbox → limited-prod → full-governed, gating G-AUTONOMY-1..5
R8	Security Hardening Path	8 modułów security jako swap nie rebuild, 4 profile, 5 faz Integration Assembly, pen-testy
R9	Ryzyka + Evidence + Post-M5	Top 12 ryzyk, Evidence Pack schema, steady-state operations, 5 rozszerzeń post-M5

R0 to must-read (executive). R1–R2 to kontrakt i architektura — czytać sekwencyjnie. R3 to referencja (tabela 65 modułów — możesz do niej wracać w trakcie implementacji). R4–R5 to harmonogram (kierownictwo techniczne + PMO). R6 to operacyjne checklisty dla zespołów implementujących — każdy klocek dostaje właściciela planu P01–P20. R7 to roadmapa autonomii (CTO + Council). R8 to ścieżka security (security lead, post M3). R9 to ryzyka + Evidence + roadmapa post-M5 (Council + Autonomy Auditor).

ROZDZIAŁ R0 · EXECUTIVE OVERVIEW

Executive Overview

R0.1 — Nadrzędna teza

SYLION AEIS Masterplan v3.5 jest planem budowy pełnego systemu produkcyjnego — nie prototypu, nie demonstratora, nie „MVP do późniejszego przebudowania”. Każda decyzja architektoniczna, każdy moduł i każdy kontrakt zostały zaprojektowane z myślą o systemie, który od dnia pierwszego jest gotowy do uruchomienia na produkcji i który rośnie przez podmiannę implementacji, a nie przez przebudowę architektury. Poniższy cytat użytkownika — inicjatora Masterplanu — wyraża tę tezę precyzyjnie:

AUTONOMY·CANON · Teza nadrzędna Masterplanu

„Budujemy pełny SYLION od początku jako system modułowy oparty o kontrakty. Security istnieje od pierwszego dnia jako zestaw wymienialnych modułów, ale na początku działa w profilach uproszczonych. Docelowe utwardzenie następuje przez podmiannę implementacji, a nie przez przebudowę architektury.”

R0.2 — Pięć meta-zasad Masterplanu

SYLION Masterplan operuje na pięciu meta-zasadach. Cztery pierwsze zostały sformułowane w Dokumentacji v3.0–v3.5. Piąta — Modularity by Contract — jest nowym dopełnieniem warstwy wykonawczej Masterplanu i nie konkuruje z pozostałymi: ona je operacjonalizuje.

REBUILD · v3.0 · Rebuildability over Lineage

System SYLION musi być zdolny do pełnego przebudowania od zera pod kontrolą Księgi i człowieka przy decyzjach systemowych. Żadna implementacja nie jest święta — tylko intencja zapisana w Księdze i kontrakty międzymodulowe są trwałe. Zasada nadrzędna: lineage (historia kodu) nigdy nie blokuje prawidłowego rebuildu.

REVERSIBILITY · v3.1 · Reversibility & Fidelity

Każda zmiana w SYLION musi być odwracalna. Każde przejście lifecycle (shadow → dual → cutover) ma zdefiniowany rollback contract. Fidelity oznacza, że po kompaktacji lub przebudowie system zachowuje semantyczną wierność względem Księgi — mierzoną przez Compact Fidelity Tests.

EFFICIENCY · v3.3 · Efficiency by Default

Efektywność nie jest cechą dodawaną po fakcie. Cztery wymiary — Code Bloat, Runtime Performance, Memory Footprint, Cost Envelope — są mierzone od dnia 1 przez dedykowane moduły klasy G. Każdy moduł ma performance budget zanim wejdzie w lifecycle „validate”.

AUTONOMY·CANON · v3.5 · Autonomy under Canon

AEIS może modyfikować sam siebie tylko w granicach określonych przez Księgę i Drabinę Decyzji D0–D5. Autonomia nie oznacza dowolności: każda modyfikacja powyżej D2 wymaga Council 4/4 i człowieka. D5 (zmiany systemowe) wymaga dodatkowo External Review. Pełna autonomia „Full Governed” wchodzi dopiero po przejściu 5 etapów Autonomy Rollout.

ZASADA · v3.5 masterplan-dopełnienie · Modularity by Contract — NOWA

Każdy moduł jest wymieniałny, o ile zachowuje publiczne kontrakty, manifest i artefakty Evidence. Wymieniałny oznacza: ten sam plik .proto, ta sama nazwa eventu, ten sam module_id — nowa implementacja w nowym kontenerze.

Dlaczego piąta zasada nie konkuruje z czterema?

Meta-zasady v3.0–v3.5 opisują co system SYLION musi osiągać: możliwość przebudowy, odwracalność, efektywność i autonomię pod kontrolą Kanonu. Zasada piąta — Modularity by Contract — odpowiada na pytanie jak te cele są osiągnięte w warstwie inżynierskiej.

Architektura Lego (opisana szczegółowo w R1) realizuje Rebuildability przez podmiannę klocków, Reversibility przez rollback contracts na każdym porcie modułu, Efficiency przez izolację performance budgets per moduł, a Autonomy przez precyzyjne granice decision_class_entry. Bez kontraktów jako mechanizmu wyższego rzędu — bez gRPC .proto, Event Taxonomy, Module Manifest Schema i Contract Freeze Ceremony — cztery meta-zasady pozostałyby postulatami bez egzekwowania w CI.

R0.3 — 10 kluczowych decyzji Masterplanu

Poniższe dziesięć decyzji stanowi trzon Masterplanu. Każda z nich wynikała z analizy ograniczeń i wymagań systemu produkcyjnego budowanego równolegle przez kilka zespołów. Szczegółowe uzasadnienia są rozwijane w rozdziałach R1–R9.

#	Decyzja	Uzasadnienie	Implikacja wykonawcza
1	Architektura Lego z małym twardym rdzeniem	Umożliwia równoległą pracę zespołów, wymieniałość modułów, brak długu	8 klocków Kernel + 6 klas modułów + ~65 modułów łącznie

#	Decyzja	Uzasadnienie	Implikacja wykonawcza
2	Production-first, NIE MVP	SYLION ma być systemem produkcyjnym od dnia 1, nie demo	Każdy moduł dostarcza pełną funkcjonalność produkcyjną; feature-flags zastępują „MVP mode”
3	Kontrakty gRPC + Protobuf jako uniwersalny protokół	Schema-first, buf breaking enforcement, język-niezależne	Każdy moduł ma .proto w Contract Registry przed implementacją
4	Python 3.12 + FastAPI + Pydantic v2 jako core	Najlepszy ekosystem AI/LLM; egzekwowanie kontraktów w runtime	Pydantic modele generowane z .proto; FastAPI endpointy dla HTTP
5	PostgreSQL 16 + JSONB + pgvector jako single storage	Wspiera Rebuildability (jeden silnik = jeden backup/restore); JSON + wektory w jednym	Każdy moduł ma własny schemat sylon_<module_id>; cross-module queries przez widoki w Contract Registry
6	NATS JetStream jako event bus	Lekki, idempotentny, dobry pod self-* loops AEIS	Event Taxonomy zamrożona w Aneksie M-Events
7	Docker compose → K8s dopiero w M5	Compose wystarcza do M4; K8s wchodzi razem z security hardening	Środowisko orchestrator abstrahuje oba środowiska
8	Security jako zestaw modułów od dnia 1 w profilach uproszczonych	Security Profile Abstraction: ten sam kontrakt, różne implementacje	8 modułów security (auth/bootstrap/session/policy/guard/secret/audit/phantom) w dev-light → prod-strict
9	Contract Freeze Milestone przed równoległą pracą	Bez zamrożenia 7 rzeczy klocki rozsy pią się przy integracji	M0 kończy się Contract Freeze Ceremony — 7 zamrożonych artefaktów w Aneksach masterplanu
10	AEIS Self-* PEŁNE od M4; aktywna self-modyfikacja progresywna przez 5 etapów Autonomy Rollout	Nie budujemy „AEIS-junior” — architektura pełna, switch aktywności stopniowy	5 modułów AEIS self-* + 5 etapów rollout (observe → propose → sandbox → limited prod → full governed)

RO.4 — Trzy wymiary SYLION-a

SYLION nie jest klasycznym silnikiem LLM ani orkiestratorem workflow. Jest systemem o trzech wymiarach działania, które wzajemnie się warunkują:

Wymiar 1 — Poznawczy (Cognitive)

SYLION rozumie, planuje i proponuje. Warstwa Cognitive obejmuje siedem modułów: Planner, Evaluator, Reasoner, Code Agent, Context Builder, Model Router i LLM Adapter. Każdy z nich operuje na danych z Księgi i Pamięci, formułuje propozycje, a następnie przekazuje je do warstwy wykonawczej lub — jeśli wymagają zatwierdzenia — do Governance przez Drabinę Decyzji D0–D5. Wymienialność modeli LLM jest zapewniona przez LiteLLM jako wymienialny adapter (patrz Rebuildability over Lineage).

Wymiar 2 — Wykonawczy (Execution)

SYLION wykonuje, integruje i łączy. Warstwa Execution obejmuje sześć modułów: Tool Runner, Connector Framework, Workflow Engine, Job Runner, Adapter Bus i Retry Orchestrator. Każde działanie wykonawcze przechodzi przez bramę Execution Guard (moduł security) i jest logowane przez Evidence Spine — co zapewnia pełny audit trail nawet w profilu dev-light. Zewnętrzne systemy są dołączane wyłącznie przez Connector Framework — bezpośredniego importu cudzego kodu między modułami nie ma.

Wymiar 3 — Samouczący się (Self-Evolution)

SYLION mierzy siebie, uczy się i proponuje zmiany we własnej strukturze. Warstwa AEIS Self-* obejmuje pięć modułów: Self-Observation, Improvement Queue, Self-Explanation, Self-Limitation i Self-Preservation. Self-Observation rejestruje 100% runtime od momentu uruchomienia (M4); Improvement Queue klasyfikuje propozycje zmian według Drabiny D0-D5; Self-Explanation waliduje, czy każda propozycja jest spójna z Księgą; Self-Limitation pilnuje granic autonomii; Self-Preservation chroni misję systemu przed modyfikacjami sprzecznymi z Kanonem. Pełna aktywność Self-Evolution wchodzi po przejściu 5 etapów Autonomy Rollout (patrz R7 — Autonomy Rollout Strategy).

R0.5 — Mapa 10 rozdziałów Masterplanu

Masterplan jest podzielony na dziesięć rozdziałów. Rozdział R0 (ten dokument) pełni rolę Executive Overview i nie zakłada znajomości pozostałych. Każdy następny rozdział zawiera pełną specyfikację wykonawczą swojego obszaru.

Rozdział	Tytuł	Zakres	Stron (ca)
R0	Executive Overview	Nadrzędna teza, 5 meta-zasad, 10 decyzji Masterplanu, mapa rozdziałów	~8
R1	Kernel & Module Architecture	Rdzeń (8 klocków) + 6 klas modułów + katalog ~65 modułów + diagram warstw; inline .proto examples dla Module Registry, Contract Registry, Event Bus	~20
R2	Contract Freeze Milestone	7 zamrożonych artefaktów: Manifest Schema, Contract Registry, Event Taxonomy, Module Lifecycle, Dependency Rules, Decision Boundaries per Module, Security Profile Abstraction	~18
R3	Tabela główna 65 modułów	Każdy moduł: klasa, owner plan, strategia, contract_version, depends_on, decision_class_entry, security_profile M0/M5, DoD, bramy entry/exit, testy, estymacja sprintów	~20
R4	Critical Path + Strumienie równoległe	Dependency graph (topological sort), 8 strumieni budowy, max parallelism analysis, synchronization points przez kontrakty	~12

Rozdział	Tytuł	Zakres	Stron (ca)
R5	Milestones M0–M5 szczegółowo	Per milestone: moduły w stanie stable, bramy wejścia/wyjścia, metryki sukcesu, profil security aktywny, wybrane kontrakty do zamrożenia	~14
R6	Execution Checklists per Plan (P01–P20)	20 checklist — dla każdego planu: moduły, kontrakty do napisania, tabele Postgres, endpointy gRPC, golden set minimum	~22
R7	Autonomy Rollout Strategy (5 etapów)	Każdy etap: moduły aktywowane, kryteria gating entry/exit, metryki sukcesu, rollback plan, interakcja z Security Hardening	~10
R8	Security Hardening Path + Integration Assembly	Security jako wymiana implementacji — 7 faz hardening; Integration Assembly Phases: scaffolding → contract conformance → shadow → dual → cutover	~12
R9	Ryzyka, Evidence Integration, Post-M5	Top 12 ryzyk z mitigacjami, Evidence pack schema dla każdej decyzji, co po M5 (steady state, federation, multi-tenant)	~10

RO.6 — Mapa klas modułów + Kernel

System SYLION składa się z Kernela (twardy rdzeń, nieruchomy) oraz dwunastu klas modułów umieszczonych „nad” Kernelem. Każda klasa jest zbiorem wymienialnych modułów o tym samym kontekście działania. Szczegółowy katalog każdego modułu (wraz z owner plan, strategią implementacji, decision_class_entry i security_profile) znajduje się w R3.

- A. Core Kernel — 8 modułów (P01): Module Registry, Manifest Loader, Contract Registry, Event Bus (NATS JetStream), Decision/Gate Engine, Evidence Spine, Environment Orchestrator, Bundle Assembler. Nieruchoma kość pacierzowa systemu. decision_class_entry = D3 dla zmian kontraktów.
- B. Cognitive — 7 modułów (P08): Planner, Evaluator, Reasoner, Code Agent, Context Builder, Model Router, LLM Adapter. Warstwa rozumowania — stubowana na fake-LLM adapter do czasu gotowości Model Routera.
- C. Execution — 6 modułów (P07): Tool Runner, Connector Framework, Workflow Engine, Job Runner, Adapter Bus, Retry Orchestrator. Każde wywołanie zewnętrzne przechodzi przez Execution Guard.
- D. Memory — 7 modułów (P02, P16, P06, P18, P19): Księga Access Layer, Compact Layer, Evidence Store, Self-Model Store, KB Adapter, Indexer (pgvector), Retrieval. Jedynym back-endem storage jest PostgreSQL 16 + JSONB + pgvector.
- E. Governance — 7 modułów (P04, P05, P06, P12, P15, P19): Decision Ladder Engine, Council Workflow, Roles, Gates Registry, Policy Registry, Self-Explanation Validator, Evidence Workflow. Drabina D0–D5 jest egzekwowana przez Decision/Gate Engine z Kernela.
- F. Security — 8 modułów (P09): auth-provider, bootstrap-init, session-broker, policy-engine, execution-guard, secret-provider, audit-sink, phantom-wrapper. Wszystkie od dnia 1, profil dev-light; podmiana na prod-strict w M5.

- G. Efficiency — 4 moduły (P10): code-bloat, runtime-perf, memory-footprint, cost-envelope. Cztery wymiary efektywności mierzone od M0.
- H. AEIS Self-* — 5 modułów (P19): self-observation, improvement-queue, self-explanation, self-limitation, self-preservation. Architektura pełna od dnia 1; aktywność stopniowa przez 5 etapów Autonomy Rollout.
- I. Skills/Demand — 3 moduły (P18, P20): skills.registry, skills.executor, aeis.demand-signal. Skill lifecycle: DRAFT → PUBLISHED → DEPRECATED.
- J. Surface — 3 moduły (P17): console-api (gRPC-web/tRPC gateway), console-ui (Next.js 14 + shadcn/ui), ws-gateway. Budowane równolegle z Kernelem na stubowanych endpointach.
- K. Rebuildability — 4 moduły (P14): rebuild-orchestrator, lpw-manager (Legacy Preservation Window), cutover-controller (shadow/dual/cutover state machine), cft-runner (Compact Fidelity Test).
- L. Quality — 3 moduły (P11): golden-set-registry, test-runner (contract/integration/performance), regression-detector (auto-rollback trigger).

ZASADA · Zasada architektury

Rdzeń jest mały i nieruchomy; wszystko poza rdzeniem jest wymienialne przez kontrakt. Łącznie ~65 modułów (nie klocków-liter — każdy moduł to osobny deployable brick).

R0.7 — 6 propozycji zmian w dokumentacji v3.5

Masterplan uzupełnia Dokumentację v3.5, a nie ją zastępuje. Poniższe sześć zmian powinno zostać wprowadzone do kanonicznej dokumentacji przed lub równolegle z M0 Contract Freeze Ceremony:

1. Dopisać meta-zasadę Modularity by Contract — do sekcji meta-zasad obok czterech istniejących (Rebuildability, Reversibility, Efficiency, Autonomy). Definicja: „Każdy moduł jest wymienialny, o ile zachowuje publiczne kontrakty, manifest i artefakty Evidence.”
2. Rozszerzyć P01 ACP o cztery elementy: registry modułów, registry kontraktów, loader profili security, orchestrator bundle composition. Obecny P01 nie odzwierciedla roli Kernela jako mechanizmu wkładania/wyjmowania klocków.
3. Dopisać do P04/P06/P12 pojęcie security_profile (dev-light / test-light / staging-strict / prod-strict) jako pole manifestu modułu i jako parametr do gate evaluatora w Decision/Gate Engine.
4. Dodać bootstrap-init jako osobny moduł security (syllion.security.bootstrap_init), nie jako część application shell. Bootstrap-init odpowiada za inicjalizację pierwszego administratora i seedowanie kluczy w profilu dev-light; w prod-strict jest podmieniany na secure seed + hardware token.
5. Dodać Contract Freeze Milestone jako formalny etap M0, kończący się Contract Freeze Ceremony. Bez zamrożenia 7 artefaktów (patrz R2) równoległa praca sześciu zespołów kończy się kolizją kontraktów.

6. Dodać Integration Assembly Phases: scaffolding → contract conformance → shadow → dual → cutover. Fazy te równoległe są z Module Lifecycle i opisują sposób integracji modułu z resztą systemu, nie tylko jego wewnętrzną dojrzałość.

R0.8 — Zakończenie i wezwanie do zamrożenia Contract Freeze

Rozdział R0 sformułował tezę nadrzędną, pięć meta-zasad, dziesięć kluczowych decyzji architektonicznych, trzy wymiary systemu, mapę dziesięciu rozdziałów Masterplanu, katalog dwunastu klas modułów i sześć zmian postulowanych w Dokumentacji v3.5. Wszystkie szczegóły — od .proto per moduł, przez tabele Postgres, po checklists per plan — są rozwijane w rozdziałach R1–R9.

AUTONOMY·CANON · Wezwanie do Contract Freeze

Masterplan nie zastępuje Dokumentacji v3.5 — on ją wykonuje. Pierwszym zadaniem M0 jest zamrożenie siedmiu artefaktów Contract Freeze. Po tym sześć zespołów może ruszyć równoległe bez ryzyka rozsypania integracji.

ROZDZIAŁ R1 · AEIS IMPLEMENTATION MASTERPLAN

Kernel & Module Architecture

R1 — Kernel & Module Architecture

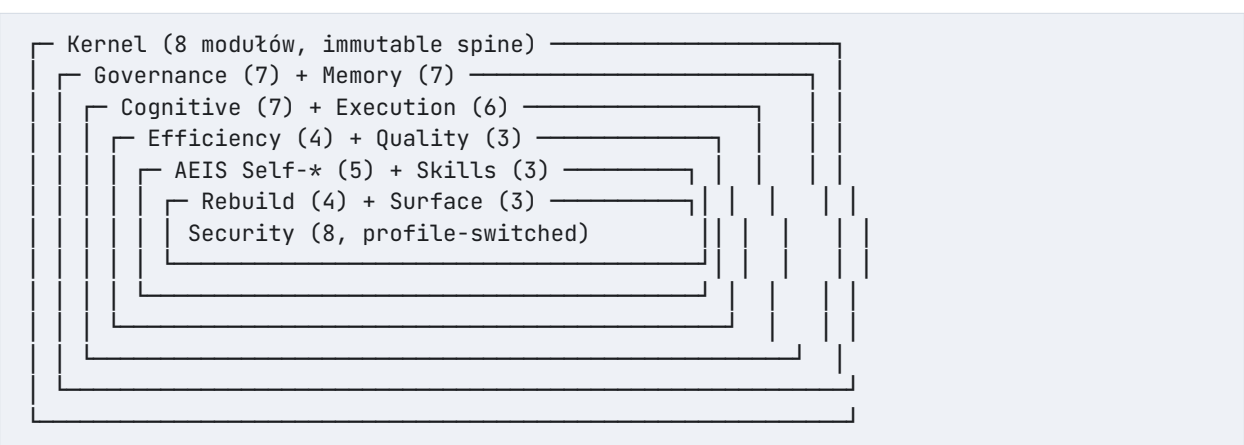
Niniejszy rozdział definiuje architekturę modułową SYLION AEIS: ośmiomodułowe jądro systemu (Kernel), sześć klas modułów nadbudowujących się nad jądrem, pełny katalog ~65 wymieniających klocków, protokół plug-in/plug-out oraz reguły komunikacji między modułami. Zrozumienie tej struktury jest warunkiem koniecznym wdrożenia zasady Modularity by Contract — piątej meta-zasady masterplanu v3.5.

R1.1 — Filozofia architektury

„Rdzeń nie jest całym SYLION-em — rdzeń to mechanizm wkładania i wyjmowania klocków”. To zdanie jest fundamentem całego masterplanu. Jądro systemu nie rozwiązuje problemów biznesowych — ono umożliwia innym modułom ich rozwiązywanie w sposób kontrolowany, wersjonowany i wymieniający.

Z filozofii tej wynikają dwie reguły strukturalne. Po pierwsze: im bardziej rdzenny moduł, tym sztywniejszy — modyfikacja wymaga wyższej klasy decyzyjnej (Kernel = D3+). Po drugie: im dalej od rdzenia, tym bardziej wymieniający — moduły peryferyjne (Skills, Surface, Rebuild) można podmieniać przy D2, nie naruszając fundamentów systemu. Ta asymetria nie jest przypadkowa: chroni integralność kontraktów przy maksymalnej elastyczności implementacji.

Poniższy diagram koncentryczny ilustruje warstwy systemu od jądra na zewnątrz. Każda warstwa jest zbiorem wymieniających modułów tej samej klasy — nie monolitem, ale rodziną klocków połączonych kontraktami:



AUTONOMY·CANON · Meta-zasada: Modularity by Contract

„Każdy moduł jest wymienialny, o ile zachowuje publiczne kontrakty, manifest i artefakty Evidence.” Podmiana implementacji (dev-light → prod-strict) nie jest zmianą architektury — to wymiana wkładu w to samo gniazdo. Ta zasada siada obok czterech istniejących meta-zasad masterplanu i jest warunkiem poprawnego odczytu wszystkich dalszych rozdziałów.

R1.2 — Rdzeń systemu: 8 modułów Kernel

Osiem modułów Kernel stanowi nieruchomą kość pacierzową SYLION. Żaden z nich nie rozwiązuje problemów domenowych — każdy realizuje jeden mechanizm infrastrukturalny, na którym opierają się wszystkie pozostałe klocki. Modyfikacja dowolnego modułu Kernel wymaga decyzji klasy D3 (Contract change = Council 4/4) lub D4/D5 dla zmian mechaniki lub fundamentów architektury.

1. *syllion.core.module_registry*

Jedyne źródło prawdy o żyjących modułach w systemie. Każdy moduł przed uruchomieniem musi się zarejestrować — bez wpisu w rejestrze moduł jest niewidoczny dla reszty systemu. Rejestr przechowuje `module_id`, `module_kind`, `contract_version`, `security_profile` oraz `lifecycle_stage`. Emituje zdarzenia `module.registered` i `module.deregistered` do Event Bus, dzięki czemu każda zmiana składu systemu jest utrwalona w Evidence Spine.

RPC	Kierunek	Opis
RegisterModule	write	Rejestruje nowy moduł lub aktualizuje istniejący wpis
ListModules	read	Zwraca pełną listę modułów z filtrowaniem po <code>module_kind</code> i <code>lifecycle_stage</code>
GetModule	read	Pobiera pojedynczy wpis po <code>module_id</code>
DeregisterModule	write	Usuwa wpis i emituje zdarzenie <code>module.deregistered</code>

PROTO

```
syntax = "proto3";
package syllion.core.module_registry.v1;

import "google/protobuf/timestamp.proto";

service ModuleRegistry {
  rpc RegisterModule(RegisterModuleRequest) returns (RegisterModuleResponse);
  rpc ListModules(ListModulesRequest) returns (ListModulesResponse);
  rpc GetModule(GetModuleRequest) returns (GetModuleResponse);
  rpc DeregisterModule(DeregisterModuleRequest) returns (DeregisterModuleResponse);
}

message ModuleEntry {
  string module_id = 1; // e.g. "syllion.governance.council"
  string module_kind = 2; // core-kernel|cognitive|execution|memory|...
  string contract_version = 3; // SemVer "1.0.0"
```

```
string security_profile = 4; // dev-light|test-light|staging-strict|prod-strict
string lifecycle_stage = 5; // draft|build|validate|shadow|dual|cutover|stable|depr
ecated
repeated string depends_on = 6;
google.protobuf.Timestamp registered_at = 7;
}

message RegisterModuleRequest { ModuleEntry entry = 1; }
message RegisterModuleResponse { string module_id = 1; bool accepted = 2; }
message ListModulesRequest { string kind_filter = 1; string stage_filter = 2; }
message ListModulesResponse { repeated ModuleEntry modules = 1; }
message GetModuleRequest { string module_id = 1; }
message GetModuleResponse { ModuleEntry entry = 1; }
message DeregisterModuleRequest { string module_id = 1; string reason = 2; }
message DeregisterModuleResponse { bool accepted = 1; }
```

2. *syllion.core.manifest_loader*

Parser i walidator pliku module.yaml — kanonicznego manifestu każdego klocka. Manifest Loader czyta YAML z katalogu modułu, waliduje go względem schematu M-Schema (Aneks M-Schema), rozwiązuje drzewo zależności i przekazuje wynik do Module Registry. Blokuje ładowanie modułu, jeśli manifest jest niekompletny lub zależności są niespełnione.

RPC	Kierunek	Opis
LoadManifest	write	Czyta module.yaml ze wskazanej ścieżki i inicjuje walidację
ValidateManifest	read	Waliduje przekazany YAML względem M-Schema bez ładowania
ResolveDependencies	read	Sprawdza, czy wszystkie depends_on są spełnione w rejestrze

```
PROTO

syntax = "proto3";
package syllion.core.manifest_loader.v1;

service ManifestLoader {
  rpc LoadManifest(LoadManifestRequest) returns (LoadManifestResponse);
  rpc ValidateManifest(ValidateManifestRequest) returns (ValidateManifestResponse);
  rpc ResolveDependencies(ResolveDepsRequest) returns (ResolveDepsResponse);
}

message LoadManifestRequest { string path = 1; }
message LoadManifestResponse { bool valid = 1; string module_id = 2;
  repeated string errors = 3; }
message ValidateManifestRequest { string yaml_content = 1; }
message ValidateManifestResponse { bool valid = 1; repeated string errors = 2; }
message ResolveDepsRequest { string module_id = 1; }
message ResolveDepsResponse { bool satisfied = 1;
  repeated string missing = 2; }
```

3. *syllion.core.contract_registry*

Wersjonowane repozytorium wszystkich kontraktów systemu: Command contracts (RPC ze zmianą stanu), Query contracts (RPC read-only), Event contracts (schema payloadu NATS), Telemetry contracts (metryki OTel) oraz Rollback contracts (dla modułów D3+). Zintegrowany z buf breaking — CI blokuje merge przy próbie niekompatybilnej zmiany .proto.

RPC	Kierunek	Opis
PublishContract	write	Publikuje nową wersję kontraktu (proto lub JSON Schema)
GetContract	read	Pobiera kontrakt po (module_id, contract_type, version)
CheckCompatibility	read	Sprawdza backward compatibility dwóch wersji kontraktu
ListBreakingChanges	read	Lista breaking changes między wskazanymi wersjami (buf diff)

4. *sylicon.core.event_bus*

Wrapper NATS JetStream realizujący trzy semantyki: publish (at-most-once dla zdarzeń fire-and-forget), subscribe (at-least-once z idempotentnym konsumentem) oraz replay (odtworzenie historii eventów od podanego checkpointu). Idempotency zapewniają event_id UUIDv7 — time-ordered, unikalny per zdarzenie. Format nazw: <domain>.<subject>.<verb> (np. council.vote.cast).

RPC	Kierunek	Opis
Publish	write	Publikuje zdarzenie do NATS z event_id UUIDv7 i podpisem
Subscribe	write	Rejestruje konsumenta dla wzorca subject z semantyką at-least-once
Replay	read	Odtwarza strumień eventów od wskazanego event_id lub timestampu

5. *sylicon.core.decision_gate_engine*

Silnik drabiny decyzyjnej D0–D5 oraz bram G-* (G-AEIS, G-EFF, G-PERF, G-MEM, G-COST). Każda propozycja zmiany przechodzi przez EvaluateDecision, które klasyfikuje ją do klasy D0–D5 na podstawie scope zmiany. Następnie EvaluateGate sprawdza, czy wymagane bramy są otwarte (quorum Council, Human Gate, Evidence wymagany). Emituje: decision.classified i gate.result.

RPC	Kierunek	Opis
EvaluateDecision	read	Klasyfikuje zmianę do klasy D0–D5 na podstawie deklarowanego scope
EvaluateGate	read	Sprawdza stan bramy G-* (otwarta/zamknięta/pending)

RPC	Kierunek	Opis
RegisterGate	write	Rejestruje nową bramę G-* z kryterium otwarcia

6. *sylion.core.evidence_spine*

Append-only log podpisany kluczem Ed25519 — niemutowalny łańcuch dowodów dla każdej decyzji, każdej zmiany lifecycle modułu i każdej operacji klasy D2+. Evidence Spine nie jest bazą danych do queryowania w locie — jest archiwum o sądowej mocy dowodowej. Replay do checkpointu służy do odtworzenia stanu systemu w dowolnym momencie przeszłości (patrz Plan 06 §6.3).

RPC	Kierunek	Opis
AppendEvidence	write	Dodaje podpisany wpis do łańcucha (Ed25519, immutable)
QueryEvidence	read	Wyszukuje wpisy po filtrach (module_id, decision_class, time range)
ReplayToCheckpoint	read	Odtwarza sekwencję zdarzeń od początku do podanego checkpoint_id

7. *sylion.core.environment_orchestrator*

Abstrakcja środowiska uruchomieniowego ukrywająca różnicę między Docker Compose (M0–M4) a Kubernetes (M5). API pozostaje identyczne — tylko implementacja się zmienia. SwapImplementation realizuje hot swap: uruchamia nową implementację w shadow, przesuwa ruch (0% → 10% → 50% → 90% → 100%) i deprecjonuje starą. Każda operacja jest atomowa z perspektywy Bundle Assembler.

RPC	Kierunek	Opis
DeployModule	write	Wdraża moduł w środowisku (compose service lub K8s Deployment)
UndeployModule	write	Usuwa moduł ze środowiska (po LPW 7 dni)
SwapImplementation	write	Wykonuje hot swap: shadow → dual → cutover przez lifecycle
ListDeployed	read	Lista aktywnych deploymentów z wersjami i wagami ruchu

8. *sylion.core.bundle_assembler*

Komponent składający zestaw modułów w bundle gotowy do uruchomienia. Bundle to deklaratywna lista modułów z wersjami kontraktów i profilami security, która przechodzi przez ValidateBundle (zgodność wersji, brak conflict depends_on) przed ShipBundle (przekazaniem do Environment Orchestrator). Bundle Assembler jest jedyną ścieżką do produkcji — nic nie trafia do runtime bez bundla.

RPC	Kierunek	Opis
AssembleBundle	write	Składa bundle z listy module_id z rozwiązanymi wersjami
ValidateBundle	read	Sprawdza spójność wersji, contract compatibility i security profiles
ShipBundle	write	Przekazuje zatwierdzony bundle do Environment Orchestrator

KRYTYCZNE · Kernel — zasada niezmienności

Żaden z 8 modułów Kernel nie może być modyfikowany bez decyzji D3 (zmiana kontraktu = Council 4/4) lub D4/D5 dla głębszych zmian. Kernel jest zamrożony po Contract Freeze Milestone (M0). Cała ewolucja systemu po M0 odbywa się przez dodawanie i podmianę modułów klasy B–L — nigdy przez ingerencję w Kernel.

R1.3 — Sześć klas modułów: kryteria członkostwa

Nad Kernelem (klasa A) istnieje sześć głównych klas modułów (B–F) oraz sześć klas rozszerzeń (G–L). Każda klasa ma zdefiniowane: zakres funkcjonalny, właściciela (Owner Plan), kryterium przynależności modułu do klasy oraz domyślny profil security na różnych etapach milestones.

Klasa	Nazwa	Modułów	Owner Plan(s)	Kryterium członkostwa	Profil domyślny
A	Core Kernel	8	P01 + P04 + P06 + P12	Stabilna kość pacierzowa; nie modyfikowalna bez D3+; infrastruktura registry/bus/evidence/orchestration	prod-strict od M1 (Kernel hardened first)
B	Cognitive	7	P08	Rozumowanie, planowanie, dekompozycja zadań, LLM routing; nie wykonuje — tylko przetwarza i produkuje plan	dev-light do M3; staging-strict M4
C	Execution	6	P07	Wykonanie, integracje zewnętrzne, adaptery, workflow runtime; działa na polecenie Cognitive — nie planuje samodzielnie	test-light do M3; staging-strict M4
D	Memory	7	P02 + P16 + P18 + P19	Persystencja, indeksowanie, retrieval — Księga, Evidence, Self-Model; jedyne moduły z trwałym stanem cross-session	dev-light M0; staging-strict M2
E	Governance	7	P04 + P05 + P06 + P11 + P12 + P13 + P15	Drabina decyzyjna D0–D5, Council Workflow, Gates, Policy Registry; każda decyzja systemowa przechodzi przez ten blok	staging-strict od M1 (Governance = early hardening)

Klasa	Nazwa	Modułów	Owner Plan(s)	Kryterium członkostwa	Profil domyślny
F	Security	8	P09	Auth, policy, guard, secrets, audit — od dnia 1 w profilu uproszczonym; podmiana implementacji (nie architektury) w M5	dev-light M0; prod-strict cutover M5

Ponadto masterplan definiuje sześć klas rozszerzeń, które budowane są w późniejszych milestones i mogą być wymieniane przy niższej klasie decyzji (D2):

- G — Efficiency (4 moduły, P10): Code Bloat, Runtime Perf, Memory Footprint, Cost Envelope — cztery wymiary efektywności mierzone i optymalizowane autonomicznie przez AEIS.
- H — AEIS Self-* (5 modułów, P19): Self-Observation, Improvement Queue, Self-Explanation, Self-Limitation, Self-Preservation — mechanizmy samokierowania systemu pod nadzorem drabiny D0–D5.
- I — Skills/Demand (3 moduły, P18+P20): Skills Registry, Skills Executor, Demand Signal — pipeline umiejętności operacyjnych SYLION.
- J — Surface (3 moduły, P17): Console API, Console UI, WebSocket Gateway — warstwa prezentacji dla operatora enterprise.
- K — Rebuildability (4 moduły, P14): Rebuild Orchestrator, LPW Manager, Cutover Controller, CFT Runner — infrastruktura bezpiecznej wymiany klocków.
- L — Quality (3 moduły, P11): Golden Set Registry, Test Runner, Regression Detector — automatyczne zabezpieczenie przed regresją przy podmianach modułów.

R1.4 — Pełny katalog modułów SYLION (~65 klocków)

Poniższe tabele zawierają kompletny inwentarz modułów systemu z przypisaniem do klas, właścicielami planów, strategią implementacji oraz profilami security na milestone M0 i M5. Każdy moduł to osobny deployable brick — osobny obraz Docker, osobny kontrakt .proto, osobny schemat Postgres. Patrz R3 — Tabela główna 65 modułów dla pełnych atrybutów per moduł (DoD, bramy entry/exit, estymacja sprintów).

Klasa A — Core Kernel (8 modułów)

Module ID	Klasa	Owner Plan	Strategia	Profil M0 → M5
syllion.core.module_registry	A Kernel	P01	Greenfield	dev-light → prod-strict M1
syllion.core.manifest_loader	A Kernel	P01	Greenfield	dev-light → prod-strict M1
syllion.core.contract_registry	A Kernel	P01	Greenfield	dev-light → prod-strict M1

Module ID	Klasa	Owner Plan	Strategia	Profil M0 → M5
syllion.core.event_bus	A Kernel	P01	Greenfield (NATS JetStream wrapper)	dev-light → prod-strict M1
syllion.core.decision_gate_engine	A Kernel	P04+P12	Greenfield	staging-strict M1 → prod-strict M2
syllion.core.evidence_spine	A Kernel	P06	Greenfield	staging-strict M1 → prod-strict M2
syllion.core.environment_orchestrator	A Kernel	P01	Greenfield	dev-light → prod-strict M5 (K8s)
syllion.core.bundle_assembler	A Kernel	P01	Greenfield	dev-light → prod-strict M1

Klasa B — Cognitive (7 modułów)

Module ID	Klasa	Owner Plan	Strategia	Profil M0 → M5
syllion.cognitive.planner	B Cognitive	P08	Greenfield	dev-light → staging-strict M4
syllion.cognitive.evaluator	B Cognitive	P08	Greenfield	dev-light → staging-strict M4
syllion.cognitive.reasoner	B Cognitive	P08	Greenfield	dev-light → staging-strict M4
syllion.cognitive.code_agent	B Cognitive	P08	Greenfield	dev-light → staging-strict M4
syllion.cognitive.context_builder	B Cognitive	P02+P16	Greenfield	dev-light → staging-strict M4
syllion.cognitive.model_router	B Cognitive	P08	Greenfield (LiteLLM wrapper)	dev-light → staging-strict M4
syllion.cognitive.llm_adapter	B Cognitive	P08	Greenfield	dev-light → staging-strict M4

Klasa C — Execution (6 modułów)

Module ID	Klasa	Owner Plan	Strategia	Profil M0 → M5
syllion.execution.tool_runner	C Execution	P07	Greenfield	test-light → staging-strict M4
syllion.execution.connector_framework	C Execution	P07	Greenfield	test-light → staging-strict M4

Module ID	Klasa	Owner Plan	Strategia	Profil M0 → M5
syllion.execution.workflow_engine	C Execution	P07	Greenfield	test-light → staging-strict M4
syllion.execution.job_runner	C Execution	P07	Greenfield	test-light → staging-strict M4
syllion.execution.adapter_bus	C Execution	P07	Greenfield	test-light → staging-strict M4
syllion.execution.retry_orchestrator	C Execution	P07	Greenfield	test-light → staging-strict M4

Klasa D — Memory (7 modułów)

Module ID	Klasa	Owner Plan	Strategia	Profil M0 → M5
syllion.memory.kanon_access	D Memory	P02	Greenfield	dev-light M0 → staging-strict M2
syllion.memory.compact_layer	D Memory	P16	Greenfield	dev-light M0 → staging-strict M2
syllion.memory.evidence_store	D Memory	P06	Greenfield	dev-light M0 → prod-strict M2
syllion.memory.self_model_store	D Memory	P19	Greenfield	dev-light M0 → staging-strict M4
syllion.memory.kb_adapter	D Memory	P18	Greenfield	dev-light M0 → staging-strict M3
syllion.memory.indexer	D Memory	P02+P16	Greenfield (pgvector)	dev-light M0 → staging-strict M2
syllion.memory.retrieval	D Memory	P02+P16	Greenfield	dev-light M0 → staging-strict M2

Klasa E — Governance (7 modułów)

Module ID	Klasa	Owner Plan	Strategia	Profil M0 → M5
syllion.governance.decision_ladder	E Governance	P04	Greenfield	staging-strict M1 → prod-strict M3
syllion.governance.council_workflow	E Governance	P05	Greenfield	staging-strict M1 → prod-strict M3
syllion.governance.roles	E Governance	P05+P15	Greenfield	staging-strict M1 → prod-strict M3

Module ID	Klasa	Owner Plan	Strategia	Profil M0 → M5
syllion.governance.gates_registry	E Governance	P12+P13	Greenfield	staging-strict M1 → prod-strict M3
syllion.governance.policy_registry	E Governance	P11+Q	Greenfield	staging-strict M1 → prod-strict M3
syllion.governance.self_explanation_validator	E Governance	P19+R	Greenfield	staging-strict M4 → prod-strict M5
syllion.governance.evidence_workflow	E Governance	P06	Greenfield	staging-strict M1 → prod-strict M2

Klasa F — Security (8 modułów, wszystkie od dnia 1)

Module ID	Klasa	Owner Plan	Profil M0 (dev-light)	Profil M5 (prod-strict)
syllion.security.auth_provider	F Security	P09	local token bootstrap	Keycloak OIDC
syllion.security.bootstrap_init	F Security	P09	plaintext admin init	secure seed + hardware token
syllion.security.session_broker	F Security	P09	in-memory sessions	Redis + JWT + refresh rotation
syllion.security.policy_engine	F Security	P09	allow-list basic	OPA + fine-grained policies
syllion.security.execution_guard	F Security	P09	subprocess isolation	PHANTOM + gVisor/Firecracker
syllion.security.secret_provider	F Security	P09	.env + local file	HashiCorp Vault
syllion.security.audit_sink	F Security	P06+P09	append log (basic)	Ed25519 signed chain (signed)
syllion.security.phantom_wrapper	F Security	P09	skeleton (light sandbox)	full PHANTOM strict

Klasy G–L — Rozszerzenia systemu

Module ID	Klasa	Owner Plan	Strategia	Profil M0 → M5
syllion.efficiency.code_bloat	G Efficiency	P10	Greenfield	dev-light → staging-strict M3
syllion.efficiency.runtime_perf	G Efficiency	P10	Greenfield	dev-light → staging-strict M3

<i>Module ID</i>	<i>Klasa</i>	<i>Owner Plan</i>	<i>Strategia</i>	<i>Profil M0 → M5</i>
syllion.efficiency.memory_footprint	G Efficiency	P10	Greenfield	dev-light → staging-strict M3
syllion.efficiency.cost_envelope	G Efficiency	P10	Greenfield	dev-light → staging-strict M3
syllion.aeis.self_observation	H AEIS	P19	Greenfield	dev-light M4 → prod-strict M5
syllion.aeis.improvement_queue	H AEIS	P19	Greenfield	dev-light M4 → prod-strict M5
syllion.aeis.self_explanation	H AEIS	P19	Greenfield	dev-light M4 → prod-strict M5
syllion.aeis.self_limitation	H AEIS	P19	Greenfield	dev-light M4 → prod-strict M5
syllion.aeis.self_preservation	H AEIS	P19	Greenfield	dev-light M4 → prod-strict M5
syllion.skills.registry	I Skills	P18+P20	Greenfield	dev-light M4 → staging-strict M5
syllion.skills.executor	I Skills	P18+P20	Greenfield	dev-light M4 → staging-strict M5
syllion.aeis.demand_signal	I Skills	P18+P20	Greenfield	dev-light M4 → staging-strict M5
syllion.surface.console_api	J Surface	P17	Greenfield	dev-light M3 → staging-strict M5
syllion.surface.console_ui	J Surface	P17	Greenfield	dev-light M3 → staging-strict M5
syllion.surface.ws_gateway	J Surface	P17	Greenfield	dev-light M3 → staging-strict M5
syllion.rebuild.orchestrator	K Rebuild	P14	Greenfield	dev-light M3 → staging-strict M4
syllion.rebuild.lpw_manager	K Rebuild	P14	Greenfield	dev-light M3 → staging-strict M4
syllion.rebuild.cutover_controller	K Rebuild	P14	Greenfield	dev-light M3 → staging-strict M4
syllion.rebuild.cft_runner	K Rebuild	P14+P16	Greenfield	dev-light M3 → staging-strict M4
syllion.quality.golden_set_registry	L Quality	P11	Greenfield	dev-light M2 → staging-strict M3

Module ID	Klasa	Owner Plan	Strategia	Profil M0 → M5
syllion.quality.test_runner	L Quality	P11	Greenfield	dev-light M2 → staging-strict M3
syllion.quality.regression_detector	L Quality	P11	Greenfield	dev-light M2 → staging-strict M3

R1.5 — Canonical Module Manifest: pełny przykład

Każdy moduł SYLION ma identyczną strukturę pliku module.yaml — kanonicznego manifestu definiowanego w Aneksie M-Schema. Manifest jest jedynym miejscem, w którym moduł deklaruje swoją tożsamość, zależności, kontrakty, profil security i strategię wdrożenia. Manifest Loader czyta go przy każdym starcie modułu i rejestruje wynik w Module Registry.

Poniżej pełny manifest modułu syllion.governance.council — jeden z bardziej rozbudowanych manifestów systemu, obejmujący wszystkie możliwe pola schematu:

YAML

```
module_id: syllion.governance.council
module_kind: governance          # core-kernel|cognitive|execution|memory|governance|security
owner_plan: P05
implementation_strategy: greenfield # reuse|refactor|rebuild|greenfield
contract_version: "1.0.0"         # SemVer

depends_on:
  - syllion.core.event_bus >= 1.0.0
  - syllion.core.decision_gate_engine >= 1.0.0
  - syllion.governance.roles >= 1.0.0

publishes_events:
  - council.vote.cast
  - council.quorum.reached
  - council.decision.finalized

consumes_events:
  - decision.proposed
  - role.assigned

entrypoints:
  grpc:
    - service: CouncilService
      port: 50061
      proto: contracts/governance/council/v1/council.proto
  http:
    - path: /health
    - path: /metrics

decision_class_entry: D2          # najniższa klasa decyzji, którą moduł przyjmuje od governance
security_profile: dev-light       # dev-light|test-light|staging-strict|prod-strict
auth_mode: bootstrap             # bootstrap|session|oidc|service
```

```

execution_guard: light          # off|light|strict
audit_mode: basic              # basic|extended|signed

bundle_compatibility:
  - bundle.core >= 1.0.0
  - bundle.governance >= 1.0.0

cutover_strategy:
  rollout: shadow -> dual -> cutover
  legacy_preservation_window_days: 7

rollback:
  auto_on_regression: true
  procedure_ref: "docs/rollback/council.md"

lifecycle_stage: draft         # draft|build|validate|shadow|dual|cutover|stable|depreca
ted

```

Opis kluczowych pól manifestu:

- `module_id` — unikalny identyfikator w przestrzeni nazw sylion.*. Niezmienny po pierwszej rejestracji w Module Registry.
- `module_kind` — przynależność do klasy A–L. Determinuje domyślny profil security i zakres decyzji wymaganych przy zmianach.
- `contract_version` — wersja SemVer kontraktu .proto. Zmiana minor = backward compatible; zmiana major = breaking change → decyzja D3.
- `depends_on` — lista zależności z constraint wersji. Manifest Loader weryfikuje spełnienie wszystkich `depends_on` przed rejestracją.
- `publishes_events` / `consumes_events` — deklaratywna lista eventów. Event Bus weryfikuje, że publisher jest zarejestrowanym właścicielem tematu.
- `decision_class_entry` — najniższa klasa decyzji, jaką moduł przyjmuje. Moduły Kernel mają D3; większość modułów B–L ma D2.
- `security_profile` / `auth_mode` / `execution_guard` / `audit_mode` — trzy ortogonalne wymiary profilu security (patrz R2.7 — Security Profile Abstraction).
- `cutover_strategy` — zadeklarowana ścieżka hot swap. Standardowo: shadow → dual → cutover z LPW 7 dni.
- `rollback` — referencja do procedury rollback i flaga auto-trigger przy regresji wykrytej przez Regression Detector.

R1.6 — Jak klocki rozmawiają: trzy dozwolone drogi

Architektura modularna traci sens, jeśli moduły komunikują się bezpośrednio — omijając kontrakty. SYLION definiuje dokładnie trzy dozwolone kanały komunikacji między modułami. Wszystkie inne są blokowane przez CI linter i runtime Module Loader.

1. Synchroniczne — gRPC przez lookup w Module Registry. Moduł A pobiera adres modułu B z Module Registry (GetModule), nawiązuje połączenie gRPC i wywołuje opublikowany RPC. Kontrakt .proto jest wersjonowany w Contract Registry — zmiana musi przejść przez CheckCompatibility przed wdrożeniem.

2. Asynchroniczne — NATS event publish/subscribe zgodnie z Event Taxonomy. Moduł A publikuje zdarzenie (np. council.decision.finalized) przez Event Bus. Moduł B subskrybuje temat i przetwarza zdarzenie idempotentnie (event_id UUIDv7). Żaden moduł nie może subskrybować tematów, których nie deklaruje w manifeście pod consumes_events.
3. Query cross-schema — tylko przez widoki w Contract Registry. Każdy moduł ma własny schemat Postgres. Jeśli moduł B potrzebuje danych z schematu A, musi to robić przez widok opublikowany w Contract Registry — nigdy bezpośrednim cross-schema SQL. Zmiana schematu A nie łamie sąsiada, bo widok jest wersjonowanym kontraktem.

Trzy drogi zabronione:

- Python import sylion.governance.council w kodzie sylion.cognitive.planner — direct import modułu B z modułu A. Blokowane przez import-linter w CI.
- Cross-schema SQL ad-hoc: SELECT * FROM governance.votes z poziomu kodu modułu Cognitive — omija Contract Registry i tworzy ukrytą zależność.
- Shared in-memory state: globalna zmienna lub singleton współdzielony między modułami — zabroniony, bo uniemożliwia niezależny deployment i restart.

KRYTYCZNE · *Direct import łamie Modularity by Contract*

Bezpośredni Python import między modułami (from sylion.moduleA import X w kodzie modułu B) jest złamaniem zasady Modularity by Contract. CI linter (import-linter) blokuje merge każdego PR zawierającego taką zależność. Jedyne dozwolone importy cross-module to sylion-common — wersjonowany PyPI package ze stabilnym, niemutowalnym API.

R1.7 — Plug-in / Plug-out Protocol

Każdy klocek SYLION przechodzi przez formalny protokół wejścia i wyjścia z systemu. Protokół jest identyczny dla wszystkich klas modułów (A–L) i realizowany przez Bundle Assembler + Environment Orchestrator + Module Registry.

Sekwencja Plug-in (wejście modułu do systemu)

Krok	Akcja	Komponent	Wynik negatywny
1	manifest validated	Manifest Loader	Blokada — moduł nie ładowany
2	contract check	Contract Registry (CheckCompatibility)	Blokada — breaking change nieautoryzowany
3	storage isolation check	Environment Orchestrator	Blokada — konflikt schematu Postgres
4	observability self-test	Moduł (own /health + /metrics)	Blokada — moduł nie raportuje zdrowia

Kr ok	Akcja	Komponent	Wynik negatywny
5	dependency satisfaction	Manifest Loader (ResolveDependencies)	Blokada — depends_on niespełnione
6	bundle compatibility	Bundle Assembler (ValidateBundle)	Blokada — konflikt wersji w bundlu
7	registered	Module Registry (RegisterModule)	Wpis w rejestrze ze stage=build
8	deployed	Environment Orchestrator (DeployModule)	Moduł uruchomiony w środowisku
9	announced event	Event Bus (module.registered)	Zdarzenie w Evidence Spine

Sekwencja Plug-out (wyjście modułu z systemu)

Kr ok	Akcja	Komponent
1	deprecated flag — oznaczenie modułu do wymiany	Module Registry (stage=deprecated)
2	traffic shift — przekierowanie ruchu na następcę	Environment Orchestrator
3	LPW 7 dni — Legacy Preservation Window aktywny	LPW Manager (syllion.rebuild.lpw_manager)
4	undeploy — usunięcie ze środowiska	Environment Orchestrator (UndeployModule)
5	deregister — usunięcie z rejestru	Module Registry (DeregisterModule)
6	announce event — zdarzenie module.deregistered	Event Bus + Evidence Spine

Hot Swap: shadow → dual → cutover

Hot swap to szczególny przypadek plug-in + plug-out wykonywany równolegle (stara i nowa implementacja działają przez pewien czas razem). Realizowany przez Cutover Controller (syllion.rebuild.cutover_controller):

- shadow (0% ruchu, only compared): nowy moduł odbiera kopię ruchu, jego odpowiedzi są porównywane z legacy — nie są zwracane do klienta. CFT Runner waliduje fidelity nowej implementacji.
- dual (10% → 50% → 90% ruchu): nowy moduł obsługuje rosnący procent ruchu równolegle z legacy. Regression Detector monitoruje regresję — auto-rollback przy wykryciu.
- cutover (100% ruchu): nowy moduł obsługuje cały ruch. Legacy dostaje status deprecated, LPW 7 dni, następnie Plug-out.

R1.8 — Kluczowy wniosek

Architektura Kernel + wymiennalnych klocków nie jest celem samym w sobie — jest środkiem do jednego konkretnego efektu: możliwości ewolucji systemu bez przebudowy jego fundamentów. Jądro SYLION zawiera osiem modułów. Wszystko ponad jądrem to wymiennalny klocek Lego.

AUTONOMY·CANON · Jądro tak małe, że rebuild jądra = rebuild systemu

„Jądro SYLION jest tak małe, że rebuild jądra to rebuild całego systemu (D5). Reszta — to wymiana klocków (D2–D4). Dzięki temu 95% zmian, optymalizacji i usprawnień systemu nigdy nie dotyka rdzenia. Zespół może podmienić Cognitive, przepisać Security hardening, wymienić LLM adapter, dodać nową klasę Skills — wszystko bez ryzyka destabilizacji fundamentów. Fundamenty są stabilne, bo są małe.”

ROZDZIAŁ R2 · AEIS IMPLEMENTATION MASTERPLAN

Contract Freeze Milestone

R2.0 — Wprowadzenie

Contract Freeze Milestone to ceremonia kończąca fazę M0. Zanim zespoły ruszą równolegle w ośmiu strumieniach budowy, siedem artefaktów MUSI być zamrożonych i opublikowanych w Contract Registry w wersji 1.0.0. Żaden moduł nie może przejść z fazy draft do build bez potwierdzenia, że wszystkie artefakty są dostępne i zwalidowane.

Każdy artefakt posiada ściśle zdefiniowaną strukturę: definicję (co to jest i dlaczego istnieje), schema (formalna specyfikacja), validator (narzędzie do automatycznej weryfikacji), przykład (konkretny, działający), właściciela (moduł lub rolę odpowiedzialną) oraz Evidence pack (zestaw dowodów potwierdzających poprawność artefaktu w evidence spine).

Jeśli po wykonaniu Contract Freeze konieczna jest zmiana któregośkolwiek artefaktu, obowiązują następujące reguły eskalacji: zmiana pojedynczego kontraktu (.proto) = klasa D3 (Council 4/4); zmiana kilku artefaktów jednocześnie lub zmiana mechaniki modułu = klasa D4 (Council 4/4 + Human Gate). Pierwsze 30 dni po Freeze obowiązuje wzmożony rygor — każda zmiana artefaktu wymaga Council 4/4, nawet jeśli nominalnie kwalifikowałaby się jako D2.

ZASADA · Siedem artefaktów Contract Freeze

1. Canonical Module Manifest Schema · 2. Contract Registry · 3. Event Taxonomy · 4. Module Lifecycle · 5. Dependency Rules · 6. Decision Boundaries per Module · 7. Security Profile Abstraction. Wszystkie siedem muszą być w Contract Registry jako wersja 1.0.0 zanim pierwszy moduł przejdzie do fazy build.

R2.1 — Artefakt 1: Canonical Module Manifest Schema

Canonical Module Manifest Schema to JSON Schema definiujący strukturę pliku module.yaml, który musi posiadać każdy moduł SYLION. Manifest jest jedynym źródłem prawdy o tożsamości, zależnościach, profilach bezpieczeństwa i strategii wdrożenia modułu. Walidator uruchamiany jest automatycznie w CI przy każdym push do repozytorium modułu.

JSON Schema — module.yaml

JSON

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
```

```

"$id": "https://syllion.ai/schemas/module-manifest/v1",
"title": "SyllionModuleManifest",
"type": "object",
"required": [
  "module_id", "module_kind", "owner_plan",
  "implementation_strategy", "contract_version",
  "decision_class_entry", "security_profile",
  "auth_mode", "execution_guard", "audit_mode"
],
"properties": {
  "module_id": { "type": "string", "pattern": "^syllion\\.[a-z_]+\\.\\.[a-z_]+$" },
  "module_kind": { "type": "string",
    "enum": ["core-kernel", "cognitive", "execution",
      "memory", "governance", "security",
      "efficiency", "aeis", "skills", "surface",
      "rebuildability", "quality"] },
  "owner_plan": { "type": "string", "pattern": "^P[0-9]{2}([+P[0-9]{2})*$" },
  "implementation_strategy": { "type": "string",
    "enum": ["reuse", "refactor", "rebuild", "greenfield"] },
  "contract_version": { "type": "string",
    "pattern": "^[0-9]+\\.\\.[0-9]+$" },
  "depends_on": { "type": "array", "items": { "type": "string" } },
  "publishes_events": { "type": "array", "items": { "type": "string" } },
  "consumes_events": { "type": "array", "items": { "type": "string" } },
  "entrypoints": { "type": "object" },
  "decision_class_entry": { "type": "string",
    "enum": ["D0", "D1", "D2", "D3", "D4", "D5"] },
  "security_profile": { "type": "string",
    "enum": ["dev-light", "test-light",
      "staging-strict", "prod-strict"] },
  "auth_mode": { "type": "string",
    "enum": ["bootstrap", "session", "oidc", "service"] },
  "execution_guard": { "type": "string",
    "enum": ["off", "light", "strict"] },
  "audit_mode": { "type": "string",
    "enum": ["basic", "extended", "signed"] },
  "bundle_compatibility": { "type": "array", "items": { "type": "string" } },
  "cutover_strategy": { "type": "object" },
  "rollback": { "type": "object" }
},
"additionalProperties": false
}

```

Validator — Python / Pydantic v2

PYTHON

```

from pydantic import BaseModel, Field
from typing import Literal, Optional, List
import re

ModuleKind = Literal[
  "core-kernel", "cognitive", "execution", "memory",
  "governance", "security", "efficiency", "aeis",
  "skills", "surface", "rebuildability", "quality"
]
Strategy = Literal["reuse", "refactor", "rebuild", "greenfield"]

```

```

DClass    = Literal["D0", "D1", "D2", "D3", "D4", "D5"]
SecProf   = Literal["dev-light", "test-light", "staging-strict", "prod-strict"]
AuthMode  = Literal["bootstrap", "session", "oidc", "service"]
ExecG     = Literal["off", "light", "strict"]
AuditM    = Literal["basic", "extended", "signed"]

class EntrypointGRPC(BaseModel):
    service: str
    port:    int
    proto:   str

class EntrypointHTTP(BaseModel):
    path: str

class Entrypoints(BaseModel):
    grpc: Optional[List[EntrypointGRPC]] = None
    http: Optional[List[EntrypointHTTP]] = None

class CutoverStrategy(BaseModel):
    rollout: str
    legacy_preservation_window_days: int

class RollbackConfig(BaseModel):
    auto_on_regression: bool
    procedure_ref:      str

class ModuleManifest(BaseModel):
    module_id: str = Field(pattern=r"^sylon\[a-z_\]+\.[a-z_]+$")
    module_kind: ModuleKind
    owner_plan: str
    implementation_strategy: Strategy
    contract_version: str = Field(pattern=r"^\d+\.\d+\.\d+$")
    depends_on: List[str] = []
    publishes_events: List[str] = []
    consumes_events: List[str] = []
    entrypoints: Optional[Entrypoints] = None
    decision_class_entry: DClass
    security_profile: SecProf
    auth_mode: AuthMode
    execution_guard: ExecG
    audit_mode: AuditM
    bundle_compatibility: List[str] = []
    cutover_strategy: Optional[CutoverStrategy] = None
    rollback: Optional[RollbackConfig] = None

    model_config = {"extra": "forbid"}

# Użycie:
# manifest = ModuleManifest.model_validate_json(open("module.yaml").read())
# lub przy YAML:
# import yaml; data = yaml.safe_load(open("module.yaml"))
# manifest = ModuleManifest(**data)

```

Przykład pełnego manifestu — *sylon.governance.council*

YAML


```
module_id: sylion.governance.council
module_kind: governance
owner_plan: P05
implementation_strategy: greenfield
contract_version: "1.0.0"
depends_on:
  - sylion.core.event_bus >=1.0.0
  - sylion.core.decision_gate_engine >=1.0.0
  - sylion.governance.roles >=1.0.0
publishes_events:
  - council.vote.cast
  - council.quorum.reached
  - council.decision.finalized
consumes_events:
  - decision.proposed
  - role.assigned
entrypoints:
  grpc:
    - service: CouncilService
      port: 50061
      proto: contracts/governance/council/v1/council.proto
  http:
    - path: /health
    - path: /metrics
decision_class_entry: D2
security_profile: dev-light
auth_mode: bootstrap
execution_guard: light
audit_mode: basic
bundle_compatibility:
  - bundle.core >= 1.0.0
  - bundle.governance >= 1.0.0
cutover_strategy:
  rollout: shadow -> dual -> cutover
  legacy_preservation_window_days: 7
rollback:
  auto_on_regression: true
  procedure_ref: "docs/rollback/council.md"
```

Tabela pól manifestu

Pole	Typ	Wymagane	Domyślna wartość	Walidacja
module_id	string	TAK	—	regex ^sylion\[a-z_\]+\.[a-z_\]+\$
module_kind	enum	TAK	—	12 dozwolonych wartości
owner_plan	string	TAK	—	regex ^P[0-9]{2}(+P[0-9]{2})*\$
implementation_strategy	enum	TAK	—	reuse refactor rebuild greenfield
contract_version	string	TAK	—	SemVer x.y.z
depends_on	list	NIE	[]	każdy wpis: module_id >=version

<i>Pole</i>	<i>Typ</i>	<i>Wymaga ne</i>	<i>Domyślna wartość</i>	<i>Walidacja</i>
publishes_events	list	NIE	[]	format domain.subject.verb
consumes_events	list	NIE	[]	format domain.subject.verb
entrypoints	object	NIE	null	grpc.port 1–65535, proto path
decision_class_entry	enum	TAK	—	D0 D1 D2 D3 D4 D5
security_profile	enum	TAK	—	4 dozwolone wartości
auth_mode	enum	TAK	—	bootstrap session oidc service
execution_guard	enum	TAK	—	off light strict
audit_mode	enum	TAK	—	basic extended signed
bundle_compatibility	list	NIE	[]	bundle.name >= version
cutover_strategy	object	NIE	null	rollout string + lpw_days int
rollback	object	NIE	null	auto_on_regression bool + ref

R2.2 — Artefakt 2: Contract Registry

Contract Registry jest wersjonowanym repozytorium wszystkich kontraktów między modułami SYLION. Każdy kontrakt ma unikalny identyfikator, wersję SemVer, listę właścicieli i konsumentów oraz historię breaking changes. Contract Registry stanowi jedyne źródło prawdy przy code-generation, testach conformance i automatycznej detekcji breaking changes przez buf.

Pięć typów kontraktów

Typ	Forma	Opis	Kluczowe pola	Breaking change history
Command	gRPC .proto	RPC ze zmianą stanu modułu	name, version, owner_module, consumers, side_effects	major_bump przy usunięciu/zmianie pola
Query	gRPC .proto	RPC bez side-effects (read-only)	name, version, owner_module, consumers	major_bump przy zmianie zwracanego kształtu
Event	NATS YAML	Kształt eventu w NATS JetStream	subject, version, owner_module, consumers, payload_schema	major_bump przy usunięciu/zmianie pola payload
Telemetry	OTel YAML	Metryki + label OpenTelemetry	metric_name, labels, unit, aggregation, owner_module	major_bump przy zmianie nazwy/labeli metryki
Rollback	gRPC .proto	RPC RollbackTo dla modułów D3+	module_id, version_from, version_to, rollback_id	major_bump przy zmianie sygnatury RPC

buf breaking — CI enforcement

Każda zmiana pliku .proto przechodzi przez buf breaking uruchamiane automatycznie w GitHub Actions. Niezgodność z baselineem blokuje merge do main.

```
BASH

# .github/workflows/contract-check.yml
# Uruchamiane przy każdym PR modyfikującym contracts/
buf breaking --against '.git#branch=main' contracts/

# Wymagany status check: buf-breaking
# Blokada merge jeśli buf exit code != 0
```

Przykład kontraktu Command — contracts/governance/council/v1/council.proto

```
PROTO
```

```

syntax = "proto3";
package sylion.governance.council.v1;

option go_package = "sylion.ai/contracts/governance/council/v1;councilv1";

import "google/protobuf/timestamp.proto";

// CouncilService – Command contracts (zmiana stanu)
service CouncilService {
    // ProposeDecision: wysyła propozycję do Council (D2+)
    rpc ProposeDecision(ProposeDecisionRequest)
        returns (ProposeDecisionResponse);

    // CastVote: głos Councillora na propozycję
    rpc CastVote(CastVoteRequest)
        returns (CastVoteResponse);

    // FinalizeDecision: zamknięcie głosowania po osiągnięciu quorum
    rpc FinalizeDecision(FinalizeDecisionRequest)
        returns (FinalizeDecisionResponse);
}

message ProposeDecisionRequest {
    string proposal_id    = 1; // UUIDv7
    string description    = 2;
    string decision_class = 3; // D0-D5
    string proposed_by    = 4;
    google.protobuf.Timestamp proposed_at = 5;
    bytes evidence_ref    = 6; // pointer do evidence spine
}

message ProposeDecisionResponse {
    string proposal_id = 1;
    string status      = 2; // ACCEPTED | REJECTED
}

message CastVoteRequest {
    string proposal_id = 1;
    string councillor  = 2;
    string vote        = 3; // APPROVE | REJECT | ABSTAIN
    string rationale   = 4;
}

message CastVoteResponse {
    string vote_id      = 1;
    int32 votes_total  = 2;
    int32 quorum_req   = 3;
}

message FinalizeDecisionRequest {
    string proposal_id = 1;
}

message FinalizeDecisionResponse {
    string decision_id = 1;
    string outcome     = 2; // APPROVED | REJECTED
    string evidence_id = 3;
}

```

```
}
```

Pozostałe typy — skeleton

Query contract (bez side-effects):

PROTO

```
// contracts/governance/council/v1/query.proto
service CouncilQueryService {
  rpc GetProposal(GetProposalRequest) returns (ProposalView);
  rpc ListVotes(ListVotesRequest)    returns (VoteListView);
}
```

Event contract (NATS JetStream — YAML schema):

YAML

```
# contracts/events/council.vote.cast.yaml
subject: council.vote.cast
version: "1.0.0"
owner_module: sylion.governance.council
consumers:
  - sylion.core.evidence_spine
  - sylion.governance.gates_registry
payload:
  event_id: { type: string, format: uuid-v7 }
  proposal_id: { type: string }
  councillor: { type: string }
  vote: { type: string, enum: [APPROVE, REJECT, ABSTAIN] }
  voted_at: { type: string, format: date-time }
```

Telemetry contract (OTel — YAML):

YAML

```
# contracts/telemetry/council.votes.yaml
metric_name: sylion_council_votes_total
type: counter
unit: "{vote}"
owner_module: sylion.governance.council
labels:
  - proposal_id
  - councillor
  - outcome
description: "Total votes cast by Council on proposals"
```

Rollback contract (D3+ modules):

PROTO

```
// contracts/rollback/council/v1/rollback.proto
service CouncilRollbackService {
  rpc RollbackTo(RollbackRequest) returns (RollbackResult);
}
message RollbackRequest {
  string module_id = 1;
  string version_from = 2;
```

```

    string version_to    = 3;
    string rollback_id   = 4; // UUIDv7
}
message RollbackResult {
    bool    success      = 1;
    string  message      = 2;
    string  evidence_id  = 3;
}

```

Reguły breaking change

Zmiana	Breaking?	Wymagana akcja
Dodanie pola opcjonalnego	NIE	minor bump; backward-compatible
Usunięcie dowolnego pola	TAK	major bump; D3 minimum; Council 4/4
Zmiana nazwy pola	TAK	major bump; D3 minimum; Council 4/4
Zmiana typu pola	TAK	major bump; D3 minimum; Council 4/4
Zmiana semantyki (pole niezmienione)	TAK	major bump; D3 — nawet jeśli schema OK
Usunięcie metody RPC	TAK	major bump; D3 minimum; Council 4/4
Zmiana nazwy pakietu .proto	TAK	major bump; D4; pełny rebuild dependentów

R2.3 — Artefakt 3: Event Taxonomy

Event Taxonomy to kanoniczna definicja wszystkich eventów przepływających przez NATS JetStream w systemie SYLION. Bez zamrożonego katalogu eventów moduły nie mogą sensownie komunikować się bez ryzyka kolizji nazw, duplikatów i nieprzewidywalnych side-effects. Taxonomy obejmuje format nazw, własność, idempotency, retry semantics oraz kanoniczną listę klas audit i evidence.

Format nazw eventów

Każda nazwa eventu składa się z trzech segmentów oddzielonych kropkami: <domain>.<subject>.<verb>. Domain = klasa modułów lub obszar funkcjonalny. Subject = obiekt, którego dotyczy zdarzenie. Verb = co się stało (past tense).

```
# Format: <domain>.<subject>.<verb>

# Governance domain
council.vote.cast
council.quorum.reached
council.decision.finalized

# Decision domain
decision.proposed
decision.classified
decision.gate_evaluated

# Evidence domain
evidence.proposal.created
evidence.vote.cast
evidence.gate_result

# Security domain
security.token.issued
security.token.revoked
security.policy.violated

# Module lifecycle domain
module.lifecycle.transitioned
module.cutover.started
module.cutover.completed
module.cutover.rolled_back

# AEIS / self-modification domain
aeis.observation.anomaly_detected
aeis.improvement.proposed
aeis.self_modification.proposed
aeis.self_modification.executed
aeis.self_modification.rolled_back

# Masterplan / system domain
masterplan.contract_freeze.completed
```

Event ownership

Każdy event ma dokładnie jeden owning module (publisher). Moduł-właściciel jest odpowiedzialny za poprawność schematu, wersjonowanie i backward compatibility payloadu. Konsumenci nie modyfikują eventu — subskrybują i przetwarzają lokalnie.

Domena	Owning module	Przykładowe events
council	syliion.governance.council_workflow	council.vote.cast, council.decision.finalized
decision	syliion.governance.decision_ladder	decision.proposed, decision.classified
evidence	syliion.core.evidence_spine	evidence.proposal.created, evidence.gate_result
module	syliion.core.module_registry	module.lifecycle.transitioned
aeis	syliion.aeis.self_observation	aeis.observation.anomaly_detected
security	syliion.security.audit_sink	security.token.issued, security.policy.violated
masterplan	syliion.core.event_bus	masterplan.contract_freeze.completed
audit	syliion.security.audit_sink	audit.rpc.called, audit.decision.proposed

Idempotency — event_id UUIDv7

Każdy event musi posiadać pole event_id w formacie UUIDv7 (time-ordered UUID). Konsumenci przechowują przetworzonych event_id w lokalnym cache z TTL 7 dni. Duplikat rozpoznany po event_id jest cicho ignorowany — nie zwraca błędu, nie przetwarza ponownie.

Retry semantics

System stosuje semantykę at-least-once delivery z idempotent consumers. Maksymalna liczba prób: 5. Opóźnienia eksponencjalne: 100 ms → 200 ms → 400 ms → 800 ms → 5 000 ms (cap). Po wyczerpaniu prób event trafia do Dead Letter Queue (DLQ) z alertem do modułu syliion.core.event_bus. Compensation events dla rollback wysyłane są jako osobny event <domain>.<subject>.rolled_back z referencją do oryginalnego event_id.

Tabela audit event classes

Event class	Opis	Owning module	Triggered by
audit.rpc.called	Każde wywołanie gRPC zarejestrowane	syliion.security.audit_sink	Każdy moduł z gRPC endpoint
audit.rpc.failed	Niepowodzenie wywołania gRPC	syliion.security.audit_sink	gRPC interceptor
audit.decision.proposed	Propozycja decyzji wysłana do drabiny	syliion.governance.decision_ladder	Każdy moduł inicjujący decyzję

<i>Event class</i>	<i>Opis</i>	<i>Owning module</i>	<i>Triggered by</i>
audit.decision.classified	Decyzja zaklasyfikowana do klasy D0–D5	syllion.governance.decision_ladder	Decision Gate Engine
audit.gate.evaluated	Brama G-* oceniona (pass/fail)	syllion.core.decision_gate_engine	Gate evaluator
audit.self_modification.proposed	AEIS proponuje modyfikację samego siebie	syllion.aeis.improvement_queue	AEIS self-* pipeline
audit.self_modification.executed	Modyfikacja samego siebie wykonana	syllion.aeis.self_observation	AEIS executor po Council
audit.self_modification.rolled_back	Modyfikacja AEIS cofnięta	syllion.rebuild.orchestrator	Auto-rollback lub Council

Tabela evidence event classes

<i>Event class</i>	<i>Opis</i>	<i>Payload kluczowe pola</i>
evidence.proposal.created	Propozycja trafia do evidence spine	proposal_id, decision_class, proposer
evidence.vote.cast	Głos Councillora zapisany jako dowód	proposal_id, councillor, vote, voted_at
evidence.gate_result	Wynik bramki G-* zapisany jako dowód	gate_id, result, evaluated_at, evidence_ref
evidence.cutover.started	Start procedury cutover modułu	module_id, from_version, to_version, cutover_id
evidence.cutover.completed	Udany cutover — moduł w stable	module_id, cutover_id, stable_at
evidence.cutover.rolled_back	Cutover cofnięty — moduł wrócił do poprzedniej wersji	module_id, cutover_id, rollback_reason

R2.4 — Artefakt 4: Module Lifecycle

Każdy moduł SYLION przechodzi przez osiem faz lifecycle. Lifecycle jest identyczny dla każdego modułu, niezależnie od klasy i właściciela. Każde przejście między fazami jest bramkowane (gate), rejestrowane w evidence spine i emituje event `module.lifecycle.transitioned`.

Diagram faz

```
draft -> build -> validate -> shadow -> dual -+
                                         |
                                deprecated <- stable <- cutover
```

Legenda:

- draft - manifest + skeleton kontraktów; nie uruchamiany
- build - CI zielone, image zbudowany, testy jednostkowe green
- validate - contract conformance green, integracja vs mocki green, perf budget green
- shadow - moduł w prod, ruch mirror 100%, response ignorowany (compare only)
- dual - moduł obsługuje część ruchu równolegle z legacy; weight rising
- cutover - moduł obsługuje 100% ruchu; legacy deprecated
- stable - moduł > 30 dni bez regresji; rejestr oznacza stable
- deprecated - oznaczony do wymiany; LPW 7 dni do usunięcia

Tabela faz lifecycle

Faza	Definicja	Input gate	Output gate	Evidence event
draft	Manifest + skeleton kontraktów istnieje; moduł nie jest uruchamiany	module_id unikalny w Registry	CI pipeline skonfigurowany	evidence.proposal.created
build	CI zielone; Dockerfile present; testy jednostkowe green	draft → build: CI green, Dockerfile	contract conformance suite skompilowana	module.lifecycle.transitioned
validate	Contract conformance green; integracja vs mocked deps green; perf budget green	build → validate: conformance pass	shadow deploy ready	evidence.gate_result
shadow	Moduł w prod; traffic mirror 100%; response compare przez 72h	validate → shadow: deploy shadow env	output diff < 0.1% przez 72h	evidence.cutover.started
dual	Moduł obsługuje 10%+ ruchu równolegle z legacy; weight monitoring	shadow → dual: diff threshold OK	90%+ clean requests przez 48h	module.lifecycle.transitioned

Faza	Definicja	Input gate	Output gate	Evidence event
cutover	Moduł obsługuje 100% ruchu; legacy deprecated	dual → cutover: plan approved	30 dni bez regresji	evidence.cutover.comple ted
stable	Moduł > 30 dni bez regresji; w rejestrze marked stable	cutover → stable: 30d clean	replacement announced	module.lifecycle.transitio ned
deprecate d	Moduł oznaczony do wymiany; LPW 7 dni do usunięcia	stable → deprecated: replacement live	LPW 7 dni minął	module.lifecycle.transitio ned

Szczegóły przejść

1. draft → build: CI pipeline zielony; Dockerfile obecny i buildowalny; contract skeleton skompilowany przez buf.
2. build → validate: contract conformance tests green; integration vs mocked dependencies green; performance budget (latency P99, throughput) spełniony.
3. validate → shadow: deploy do shadow environment; traffic mirror 100%; porównanie output przez 72 godziny; diff threshold poniżej 0.1%.
4. shadow → dual: minimum 10% real traffic przekierowany; weight monitoring aktywny; alerty na regresję.
5. dual → cutover: co najmniej 90% requests clean przez 48h; cutover plan zatwierdzony przez Council.
6. cutover → stable: 30 dni bez regresji w produkcji; brak alertów krytycznych.
7. stable → deprecated: moduł-następca ogłoszony; Legacy Preservation Window 7 dni aktywny.

R2.5 — Artefakt 5: Dependency Rules

Dependency Rules to pięć twardych reguł definiujących dopuszczalne zależności między modułami SYLION. Reguły są egzekwowane automatycznie przez CI — naruszenie jest blockerem merge.

Pięć twardych reguł

1. Zero direct Python import między modułami. Żaden moduł nie może wykonać import `syllion.<inny_moduł>`. Egzekwowane przez `import-linter` z konfiguracją `forbidden_modules` jako blocker w GitHub Actions.
2. Komunikacja tylko przez Module Registry + gRPC LUB NATS events. Moduł chcący wywołać inny moduł: (a) szuka go w Module Registry, (b) wywołuje przez opublikowany kontrakt gRPC LUB (c) publikuje event w NATS. Bezpośrednie połączenie TCP/HTTP między modułami = forbidden.
3. Zero shared DB bez kontraktu. Każdy moduł ma własny schemat Postgres `syllion-<module_id_suffix>`. Cross-schema queries wyłącznie przez widoki zarejestrowane w Contract Registry (views published). Bezpośredni SELECT na schemacie innego modułu = forbidden.
4. Shared utility libs tylko przez versioned PyPI package. Wspólne klasy, helpery i utility muszą być opublikowane jako `syllion-common>=1.0.0` (lub inny wersjonowany pakiet). Import surowych plików z innego modułu = forbidden.
5. Cross-schema queries przez widoki w Contract Registry. Jeśli moduł A potrzebuje danych z schematu modułu B — moduł B publikuje widok jako kontrakt Query w Contract Registry. Moduł A używa wyłącznie tego widoku.

Konfiguracja `import-linter` (`.importlinter`)

PYTHON

```
[importlinter]
root_package = syllion

[importlinter:contract:1]
name = Modules cannot import each other directly
type = forbidden
source_modules =
    syllion.governance
    syllion.cognitive
    syllion.execution
    syllion.memory
    syllion.aeis
    syllion.security
    syllion.efficiency
    syllion.skills
    syllion.surface
    syllion.rebuild
    syllion.quality
forbidden_modules =
```

```

    sylion.governance.*
    sylion.cognitive.*
    sylion.execution.*
    sylion.memory.*
    sylion.aeis.*
    sylion.security.*
    sylion.efficiency.*
    sylion.skills.*
    sylion.surface.*
    sylion.rebuild.*
    sylion.quality.*
allow_indirect_imports = False

[importlinter:contract:2]
name = Modules may import sylion.core (Kernel) only through contracts
type = forbidden
source_modules =
    sylion.governance
    sylion.cognitive
    sylion.execution
    sylion.memory
    sylion.aeis
forbidden_modules =
    sylion.core.module_registry
    sylion.core.manifest_loader
    sylion.core.evidence_spine
allow_indirect_imports = False

```

CI enforcement

BASH

```

# .github/workflows/dependency-check.yml
- name: Check import boundaries
  run: |
    pip install import-linter
    lint-imports
# exit code 1 = blocker, blokuje merge do main

```

UWAGA · *Zero shortcuts — architektura jest egzekwowalna*

Reguły zależności nie są konwencją — są automatycznie sprawdzane. Każdy PR naruszający granicę modułów zostaje zablokowany przez CI przed code review. Reviewer nie ma możliwości pominięcia tej blokady bez wyłączenia reguły — co samo w sobie wymaga decyzji D3.

R2.6 — Artefakt 6: Decision Boundaries per Module

Decision Boundaries per Module to mapping drabiny D0–D5 na kontekst konkretnego modułu SYLION. Każdy moduł deklaruje w manifeście `decision_class_entry` = najniższą klasę decyzji, którą przyjmuje od governance. Większość modułów Kernel = D3; moduły Cognitive, Execution, Memory = D2; moduły Security = D3 minimum.

Mapowanie D0–D5 na moduł

- D0 — Moduł może czytać, raportować, logować. Zero side-effects na stan systemu. Przykład: odczyt metryk, pobranie logu, raport statusu.
- D1 — Moduł może modyfikować swój własny stan wewnętrzny. To jest implementation detail — nie zmienia kontraktu, nie wpływa na innych. Przykład: zmiana TTL cache, update lokalnego configu modułu.
- D2 — Zmiana featurowa w module wymagająca Full Board + Review. Standardowa ścieżka dla nowych funkcji. Przykład: nowe pole w internal DB schema, nowa metoda prywatna.
- D3 — Zmiana kontraktu (breaking change .proto lub event schema). Wymaga Council 4/4. Przykład: zmiana sygnatury RPC, usunięcie pola z eventu.
- D4 — Zmiana mechaniki modułu lub rebuild modułu. Wymaga Council 4/4 + Human Gate. Przykład: zamiana silnika bazy danych modułu, pełny rewrite modułu.
- D5 — Zmiana security architecture, fundamentów Księgi, multi-module redesign. Wymaga Council 4/4 + Human Gate + External Review. Przykład: zmiana modelu autentykacji, redesign event taxonomy.

Tabela klas decyzji

Klasa	Przykłady zmian w module	Approval	Rollback wymagany	Evidence level
D0	Odczyt metryk, raport statusu, pobranie logu	Brak — autonomiczny	NIE	basic log
D1	Zmiana TTL cache, update lokalnego configu	Brak — implementation detail	NIE	basic log
D2	Nowa feature, nowe pole internal DB, nowa metoda prywatna	Full Board + Review	TAK (opcjonalnie)	extended evidence
D3	Breaking .proto change, usunięcie pola eventu, zmiana RPC sygnatury	Council 4/4	TAK (obowiązkowy)	signed evidence
D4	Rebuild modułu, zamiana silnika DB, pełny rewrite	Council 4/4 + Human Gate	TAK (obowiązkowy)	signed + external

Klasa	Przykłady zmian w module	Approval	Rollback wymagany	Evidence level
D5	Security arch redesign, zmiana Księgi, multi-module redesign	Council 4/4 + Human + External Review	TAK (obowiązkowy)	signed + audit trail

UWAGA · *Zmiana .proto = D3 minimum*

Zmiana pliku .proto modułu — nawet jeden diff w sygnaturze API — to automatycznie klasa D3. Jeden zmieniony wiersz w kontrakcie oznacza Council 4/4. Nie ma wyjątków dla „małych” zmian. Jedynym wyjątkiem jest dodanie opcjonalnego pola — to jest non-breaking i nie wymaga D3.

R2.7 — Artefakt 7: Security Profile Abstraction

Security Profile Abstraction definiuje cztery profile wdrożenia oraz trzy ortogonalne tryby, które każdy moduł deklaruje w manifeście. Kluczowa właściwość architektury: ten sam kontrakt `.proto` dla `auth_provider` w każdym profilu — tylko implementacja się zmienia. Profile `dev` → `staging` → `prod` = podmiana implementacji w to samo gniazdo, NIE zmiana architektury.

Trzy ortogonalne tryby

- `auth_mode`: sposób autentykacji — `bootstrap` (lokalny token) | `session` (session-broker) | `oidc` (Keycloak) | `service` (mTLS)
- `execution_guard`: poziom sandboxingu — `off` (brak) | `light` (subprocess isolation) | `strict` (PHANTOM pełny + gVisor)
- `audit_mode`: poziom audytu — `basic` (log only) | `extended` (+ structured evidence) | `signed` (Ed25519 per event)

Pełna macierz profili

Profil	<i>auth_mode</i>	<i>execution_guard</i>	<i>audit_mode</i>	Użycie	<i>auth implementacja</i>
dev-light	bootstrap	off	basic	Lokalny developer, M0	<code>auth_provider_bootstrap</code> (local UUIDv4 tokens)
test-light	bootstrap	light	basic	CI, smoke tests, ephemeral env	<code>auth_provider_bootstrap</code> (ephemeral)
staging-strict	oidc	strict	extended	Pre-production, integration tests	<code>auth_provider_oidc</code> (Keycloak staging)
prod-strict	oidc	strict	signed	Produkcja M5+	<code>auth_provider_oidc</code> (Keycloak prod) + Ed25519

AuthProvider — kontrakt `.proto` (identyczny dla wszystkich profili)

Poniższy interfejs jest niezmienny w każdym profilu bezpieczeństwa. Jedyna różnica między M0 a M5 to implementacja podłączona pod ten kontrakt.

```
PROTO

syntax = "proto3";
package sylion.security.auth.v1;

// AuthProvider — kontrakt NIEZMIENNY między wszystkimi profilami.
// Implementacje są wymienne; kontrakt pozostaje 1.0.0.
service AuthProvider {
    // Authenticate: weryfikacja credentials, zwraca token
    rpc Authenticate(AuthRequest) returns (AuthResponse);

    // ValidateToken: weryfikacja tokenu (każde wywołanie gRPC)
```



```

rpc ValidateToken(ValidateRequest) returns (ValidateResponse);

// RevokeToken: unieważnienie tokenu
rpc RevokeToken(RevokeRequest) returns (RevokeResponse);
}

message AuthRequest {
  string principal = 1; // user / service ID
  string credential = 2; // hasło / API key / OIDC code
  string auth_method = 3; // bootstrap | session | oidc | service
}

message AuthResponse {
  string token = 1;
  int64 expires_at = 2; // Unix timestamp
  string principal = 3;
}

message ValidateRequest {
  string token = 1;
}

message ValidateResponse {
  bool valid = 1;
  string principal = 2;
  repeated string roles = 3;
}

message RevokeRequest {
  string token = 1;
}

message RevokeResponse {
  bool success = 1;
}

// Implementacje wymieniające przez Module Registry:
// - auth_provider_bootstrap (M0) — local UUIDv4 tokens, zero external deps
// - auth_provider_session (M2) — session-broker backed, Redis + JWT
// - auth_provider_oidc (M5) — Keycloak OIDC, full PKCE, token rotation
// Każda implementacja podlega temu samemu kontraktowi.
// Podmiana = module lifecycle shadow -> dual -> cutover.
// Kontrakt pozostaje 1.0.0 przez całą żywotność systemu.

```

ZASADA · *Profil to parametr wdrożenia, nie zmiana architektury*

„dev-light” i „prod-strict” to nie dwa różne systemy. To ten sam system z różnymi implementacjami podłączonymi pod te same kontrakty. Security hardening w M5 nie wymaga przeprojektowania — tylko podmiany ośmiu modułów security przez procedurę shadow → dual → cutover.

R2.8 — Ceremonia Contract Freeze

Contract Freeze to formalna ceremonia kończąca fazę M0. Nie jest to przegląd kodu — jest to moment, w którym architektura przestaje być negocjowalna i staje się podstawą równoległej pracy ośmiu strumieni budowy.

Uczestnicy ceremonii

- AEIS Steward — przewodniczy ceremonii, potwierdza spójność z Księgą
- Lead architect — Core Kernel (Strumień 1)
- Lead architect — Cognitive + Execution (Strumienie 2–3)
- Lead architect — Memory + Governance (Strumienie 4–5)
- Lead architect — Surface + AEIS (Strumienie 6 + 8)
- Security lead — weryfikuje Security Profile Abstraction i profil dev-light
- Governance lead — weryfikuje Decision Boundaries i Event Taxonomy

Deliverables ceremonii

1. 7 artefaktów w Contract Registry w wersji 1.0.0 — każdy artefakt otagowany gitem jako contract-freeze-v1. Tag musi być podpisany (git tag -s).
2. Evidence pack contract_freeze_2026.json w evidence spine — zawiera ref do każdego artefaktu, hash SHA-256, timestamp, podpisy wszystkich uczestników ceremonii.
3. Publiczne ogłoszenie — event masterplan.contract_freeze.completed opublikowany w NATS JetStream; konsumenci (wszystkie moduły) mogą przejść z fazy draft do build po odebraniu tego eventu.

Post-Freeze — zasady zmian

Po wykonaniu Contract Freeze obowiązują następujące zasady dla każdej propozycji zmiany artefaktu:

- Zmiana dowolnego artefaktu = klasa D3 minimum (Council 4/4).
- Pierwsze 30 dni po Freeze — wzmożony rygor stabilizacyjny: Council 4/4 wymagane dla KAŻDEJ zmiany artefaktu, nawet jeśli nominalnie kwalifikuje się jako D2.
- Zmiana kilku artefaktów jednocześnie = D4 (Council 4/4 + Human Gate).
- Zmiany D5 (np. redesign event taxonomy) = External Review + migracja wszystkich konsumentów.
- Każda zatwierdzona zmiana musi przejść przez pełny lifecycle modułu (shadow → dual → cutover) jeśli dotyczy modułów w fazie validate lub wyżej.

AUTONOMY·CANON · Contract Freeze — moment wejścia w architekturę wykonawczą

„Contract Freeze to moment, w którym architektura przestaje być dyskutowalna i staje się wykonawcza. Bez tego kroku równoległa praca jest fikcją. Bez zamrożonych kontraktów każdy strumień budowy buduje coś innego i żaden z nich nie będzie pasował do pozostałych.”

ZASADA · Kluczowy wniosek R2

Siedem artefaktów Contract Freeze nie jest dokumentacją — jest fundamentem całej architektury modularnej SYLION AEIS. Każdy moduł jest wymienialny dlatego, że kontrakty są niezmiennie. Każda zmiana kontraktu jest droga dlatego, że kontrakty są fundamentem. Modularity by Contract istnieje tylko wtedy, gdy kontrakty są rzeczywiście zamrożone.

ROZDZIAŁ R3 · AEIS IMPLEMENTATION MASTERPLAN

Katalog 65 Modułów SYLION

R3 — Tabela główna 65 modułów

R3.0 — Wprowadzenie

Ten rozdział to pełny kontrakt masterplanu — jedno miejsce z pełną listą modułów, ich właścicielami, zależnościami i kryteriami Done. Każdy moduł systemu SYLION jest tutaj opisany raz i kompletnie: klasa architektoniczna, owner plan, strategia implementacji, profil security na każdym etapie (M0–M5), Definition of Done (DoD) oraz bramy wejścia i wyjścia.

Konwencja dokumentu: każdy moduł = jeden wiersz w tabeli swojej klasy. Każda klasa ma trzy tabele: (1) atrybuty podstawowe, (2) DoD + bramy, (3) profil security M0–M5. Tabela sumaryczna w R3.14 zawiera wszystkie 65 modułów w jednym miejscu.

R3.1 — Jak czytać rozdział

Każda tabela modułów zawiera następujące kolumny:

- Module ID — kanoniczna nazwa modułu (np. `syllion.core.module_registry`)
- Owner Plan — plan z dokumentacji v3.5 odpowiedzialny za moduł (np. P01, P08)
- Strategia — Greenfield / Refactor / Reuse (wszystkie moduły SYLION są Greenfield)
- Depends on — kluczowe zależności (komunikacja przez gRPC lub NATS, NIE Python import)
- Contract v1.0 — status kontraktu w M0: Ready / Draft / Not Started
- Profil security M0 → M5 — profil na każdym etapie: dev-light → prod-strict
- DoD — Definition of Done — jedno-zdaniowe kryterium ukończenia
- Entry gate — co musi być stable przed startem modułu

- Exit gate — co musi być true, żeby moduł przejść do stable
- Est. sprinty — szacunkowa liczba sprintów na jeden moduł (1 sprint = 2 tygodnie)
- Dział właściciel — jeden z 7 stałych działów Pipeline

Priorytet w tabeli sumarycznej (R3.14): 1 = critical path (blokuje równoległe prace), 2 = ważne (M1–M3, możliwe równoległe), 3 = można opóźnić (M4+, niezależne od Kernel).

R3.2 — Klasa A: Core Kernel (8 modułów)

Kernel to nieruchoma kość pacierzowa systemu — 8 modułów odpowiedzialnych za rejestrację modułów, wersjonowanie kontraktów, szynę zdarzeń, silnik decyzji, spine dowodów, orkiestrację środowiska i składanie bundle’ów. Każda zmiana Kernel = D3 (Council 4/4). Żadna inna część systemu NIE JEST tak blokująca.

Tabela A-1: Atrybuty podstawowe

Module ID	Owner Plan	Strategia	Depends on	Est. sprinty	Dział
syllion.core.module_registry	P01	Greenfield	—	3	Architektura
syllion.core.manifest_loader	P01	Greenfield	module_registry	2	Architektura
syllion.core.contract_registry	P01	Greenfield	module_registry	3	Architektura
syllion.core.event_bus	P01	Greenfield	module_registry	3	Architektura
syllion.core.decision_gate_engine	P04+P12	Greenfield	event_bus, contract_registry	4	Governance
syllion.core.evidence_spine	P06	Greenfield	event_bus	3	Governance
syllion.core.environment_orchestrator	P01	Greenfield	manifest_loader	3	DevOps
syllion.core.bundle_assembler	P01	Greenfield	manifest_loader, env_orch	2	DevOps

Tabela A-2: DoD + bramy

Module ID	DoD produkcyjny	Entry gate	Exit gate
syllion.core.module_registry	100% żyjących modułów zarejestrowanych; API query <50ms p95; backup+restore <60s; brak sąsiadów bez kontraktu	Schema manifest frozen (R2.1)	30 modułów zarejestrowanych; 10.000 lookups bez błędu; disaster recovery drill pass

Module ID	DoD produkcyjny	Entry gate	Exit gate
syllion.core.manifest_loader	Każdy module.yaml walidowany w <200ms; błędy manifest blokują deploy; 100% coverage schematu	module_registry stable	50 manifestów załadowanych bez błędu; walidator odrzuca 5 złych plików testowych
syllion.core.contract_registry	100% .proto files w rejestrze; buf breaking enforced w CI; change log sign by Ed25519	10 startowych .proto	Pierwsza wykryta i zablokowana breaking change; 30 kontraktów w rejestrze
syllion.core.event_bus	NATS JetStream: at-least-once, idempotency keys (UUIDv7), <10ms p95 publish; event catalog pełny	NATS JetStream dostępny	100.000 eventów bez utraty; replay poprawny; dead-letter queue aktywna
syllion.core.decision_gate_engine	Drabina D0–D5 live; każda decyzja ≥D2 z evidence pack; gate evaluator SLA <200ms p95	decision_ladder stable; gates_registry v1.0	100 decyzji D0–D5 przetestowanych; 0 false-pass na złych evidence; audit trail pełny
syllion.core.evidence_spine	Append-only log; każdy event Ed25519 signed; replay capability; 0 luk w łańcuchu dowodów	event_bus stable; secret_provider (signing key)	Pełny replay 1000 eventów; signature verification 100%; disaster recovery <60s
syllion.core.environment_orchestrator	Deploy/undeploy/swap każdego modułu przez API; compose→K8s abstraction; rollback <30s	manifest_loader stable; Docker compose działa	Shadow+dual+cutover dla 3 modułów testowych; rollback automatyczny na regresji
syllion.core.bundle_assembler	Bundle gotowy do runu w <5min; shadow/dual/cutover mechanika; bundle manifest podpisany	manifest_loader + contract_registry stable	5 bundli złożonych; 3 cutovery bez przestoju; bundle integrity check 100%

Tabela A-3: Profil security M0 → M5

Module ID	Profil M0	Profil M2	Profil M5
syllion.core.module_registry	dev-light / bootstrap	test-light / session	prod-strict / oidc+mTLS
syllion.core.manifest_loader	dev-light / bootstrap	test-light / session	prod-strict / oidc
syllion.core.contract_registry	dev-light / bootstrap	test-light / session	prod-strict / oidc+Ed25519
syllion.core.event_bus	dev-light / off	test-light / light	prod-strict / strict

Module ID	Profil M0	Profil M2	Profil M5
syllion.core.decision_gate_engine	dev-light / bootstrap	staging-strict / oidc	prod-strict / oidc+signed
syllion.core.evidence_spine	dev-light / basic	test-light / extended	prod-strict / signed
syllion.core.environment_orchestra tor	dev-light / off	test-light / light	prod-strict / strict
syllion.core.bundle_assembler	dev-light / bootstrap	test-light / session	prod-strict / oidc

R3.3 — Klasa B: Cognitive (7 modułów)

Moduły poznawcze realizują inteligencję systemu: planowanie, ocenę, rozumowanie, generację kodu i routing do modeli LLM. Działają na stub-LLM do M2, potem na żywe modele. Wszystkie przez model_router — nigdy bezpośrednio do API modelu.

Tabela B-1: Atrybuty podstawowe

Module ID	Owner Plan	Strategia	Depends on	Est. sprinty	Dział
syllion.cognitive.planner	P08	Greenfield	event_bus, model_router	4	Cognitive
syllion.cognitive.evaluator	P08	Greenfield	planner, evidence_spine	3	Cognitive
syllion.cognitive.reasoner	P08	Greenfield	planner, kanon_access	4	Cognitive
syllion.cognitive.code_agent	P08	Greenfield	planner, execution.tool_runner	4	Cognitive
syllion.cognitive.context_builder	P02+P16	Greenfield	kanon_access, memory.retrieval	3	Memory
syllion.cognitive.model_router	P08	Greenfield	LiteLLM, event_bus	3	Cognitive
syllion.cognitive.llm_adapter	P08	Greenfield	model_router	2	Cognitive

Tabela B-2: DoD + bramy

Module ID	DoD produkcyjny	Entry gate	Exit gate
syllion.cognitive.planner	Dekompozycja zadania → plan D-klas w <3s; 90% planów zaakceptowanych przez Reasoner; fallback na degraded mode	model_router stable; kanon_access readable	50 zadań złożonych zdekompozytowanych poprawnie; latency p95 <3s; plan fidelity >90%

Module ID	DoD produkcyjny	Entry gate	Exit gate
syllion.cognitive.evaluator	Ocena propozycji Plannera: accuracy >85% vs golden set; evidence pack dla każdej oceny D2+	planner stable; evidence_spine stable	100 propozycji ocenionych; accuracy >85%; 0 ocen bez evidence przy D2+
syllion.cognitive.reasoner	Chain-of-thought reasoning z dostępem do Księgi; hallucination rate <5%; wyjaśnienie każdej decyzji	kanon_access stable; planner stable	Złoty set 50 pytań: accuracy >90%; 0 naruszeń meta-zasad; self-explanation valid
syllion.cognitive.code_agent	Generacja + uruchomienie kodu w sandbox; 80% unit tests green w pierwszej iteracji; pełny rollback	planner stable; execution.tool_runner stable; phantom_wrapper dev-light	20 zadań kodowania end-to-end; 80% first-run success; sandbox isolation verified
syllion.cognitive.context_builder	Context window optimal dla każdego modelu; <500ms budowania kontekstu; relevance score >0.8	kanon_access stable; memory.retrieval stable	Context dla 5 modeli; relevance p95 >0.8; compaction bez utraty kluczowych faktów
syllion.cognitive.model_router	Routing do 5 modeli LLM przez LiteLLM; failover <1s; cost budget enforced per request; audit log	LiteLLM skonfigurowany; 3+ modele dostępne	1000 requestów bez błędu; failover tested (model kill test); cost budget enforced
syllion.cognitive.llm_adapter	Unified LLM interface; streaming response; token counting; retry z exponential backoff	model_router stable	10.000 requestów; streaming latency <200ms first token; retry coverage 100%

Tabela B-3: Profil security M0 → M5

Module ID	Profil M0	Profil M2	Profil M5
syllion.cognitive.planner	dev-light / bootstrap	test-light / session	prod-strict / oidc
syllion.cognitive.evaluator	dev-light / bootstrap	test-light / session	prod-strict / oidc+signed
syllion.cognitive.reasoner	dev-light / bootstrap	test-light / session	prod-strict / oidc
syllion.cognitive.code_agent	dev-light / light	test-light / light	prod-strict / strict+PHANTOM
syllion.cognitive.context_builder	dev-light / bootstrap	test-light / session	prod-strict / oidc
syllion.cognitive.model_router	dev-light / bootstrap	test-light / light	prod-strict / oidc+budget
syllion.cognitive.llm_adapter	dev-light / bootstrap	test-light / light	prod-strict / mTLS

R3.4 — Klasa C: Execution (6 modułów)

Moduły wykonawcze uruchamiają narzędzia, zarządzają workflow'ami i jób'ami. workflow_engine odzwierciedla 7 stałych działań Pipeline jako instancje workflow. Isolation przez phantom_wrapper (dev-light w M0, pełny PHANTOM w M5).

Tabela C-1: Atrybuty podstawowe

Module ID	Owner Plan	Strategia	Depends on	Est. sprinty	Dział
sylicon.execution.tool_runner	P07	Greenfield	event_bus, auth_provider	3	Execution
sylicon.execution.connector_framework	P07	Greenfield	tool_runner	3	Execution
sylicon.execution.workflow_engine	P07	Greenfield	tool_runner, decision_gate_engine	4	Execution
sylicon.execution.job_runner	P07	Greenfield	workflow_engine, event_bus	2	Execution
sylicon.execution.adapter_bus	P07	Greenfield	connector_framework	2	Execution
sylicon.execution.retry_orchestrator	P07	Greenfield	job_runner, event_bus	2	Execution

Tabela C-2: DoD + bramy

Module ID	DoD produkcyjny	Entry gate	Exit gate
sylicon.execution.tool_runner	Uruchomienie dowolnego narzędzia w sandbox; SLA <500ms p95; execution audit log; 0 escape z sandboxu	auth_provider dev-light; phantom_wrapper dev-light	50 narzędzi zarejestrowanych; sandbox isolation verified; 1000 executions bez escape
sylicon.execution.connector_framework	Unified connector interface dla 10+ systemów zewnętrznych; circuit breaker; retry semantics	tool_runner stable	10 connectorów (GitHub, Jira, Slack, S3, Postgres, REST...); circuit breaker tested
sylicon.execution.workflow_engine	7 stałych działań jako workflows; 100 równoległych instancji; replay pełnego workflow; state persistence	decision_gate_engine stable; job_runner stable	7 działań jako workflows live; 100 równoległych instancji bez błędów; replay 3 złożonych workflows

Module ID	DoD produkcyjny	Entry gate	Exit gate
syllion.execution.job_runner	Distributed job queue; priority scheduling; dead-letter queue; job metrics w OTel	workflow_engine stable; event_bus stable	10.000 jobów bez utraty; priority scheduling przetestowany; dead-letter queue aktywna
syllion.execution.adapter_bus	Unified adapter registry; hot-swap adaptera bez restartu; adapter version negotiation	connector_framework stable	5 adapterów zarejestrowanych; hot-swap tested; version negotiation pełna
syllion.execution.retry_orchestrator	Exponential backoff z jitter; max-retry policy per job type; compensation events na failure	job_runner stable; event_bus stable	Retry storm prevention tested; compensation events 100% poprawne; SLA enforcement

Tabela C-3: Profil security M0 → M5

Module ID	Profil M0	Profil M2	Profil M5
syllion.execution.tool_runner	dev-light / light	test-light / light	prod-strict / strict
syllion.execution.connector_framework	dev-light / light	test-light / light	prod-strict / mTLS
syllion.execution.workflow_engine	dev-light / session	staging-strict / oidc	prod-strict / oidc+signed
syllion.execution.job_runner	dev-light / basic	test-light / extended	prod-strict / signed
syllion.execution.adapter_bus	dev-light / basic	test-light / light	prod-strict / mTLS
syllion.execution.retry_orchestrator	dev-light / basic	test-light / extended	prod-strict / signed

R3.5 — Klasa D: Memory (7 modułów)

Moduły pamięci zarządzają dostępem do Księgi, kompaktją, dowodną historią i indeksowaniem wektorowym (pgvector). kanon_access jest centralną bramą — write tylko przez Council D3+; read bez ograniczeń. compact_layer wymaga 100% CFT pass rate na golden secie przed każdą kompaktją.

Tabela D-1: Atrybuty podstawowe

Module ID	Owner Plan	Strategia	Depends on	Est. sprinty	Dział
syllion.memory.kanon_access	P02	Greenfield	contract_registry	4	Memory
syllion.memory.compact_layer	P16	Greenfield	kanon_access, cft_runner	4	Memory
syllion.memory.evidence_store	P06	Greenfield	evidence_spine	3	Governance
syllion.memory.self_model_store	P19	Greenfield	kanon_access, self_observation	3	AEIS
syllion.memory.kb_adapter	P18	Greenfield	kanon_access	3	Memory
syllion.memory.indexer	P02+P16	Greenfield (pgvec)	kanon_access, Postgres+pgvector	3	Memory
syllion.memory.retrieval	P02+P16	Greenfield	indexer	2	Memory

Tabela D-2: DoD + bramy

Module ID	DoD produkcyjny	Entry gate	Exit gate
syllion.memory.kanon_access	Read-only access do Księgi dla wszystkich modułów; write ścieżka tylko przez Council D3+; vector search <100ms p95	Księga (intencje+meta-zasady+aneksy) załadowana; contract_registry v1.0	1000 read queries bez błędu; vector search <100ms p95; 0 unauthorized writes; write ścieżka Council-gated
syllion.memory.compact_layer	CFT pass rate 100% golden set; compaction ratio ≥5x; brak utraty kluczowych faktów; CFT runner zintegrowany	kanon_access stable; cft_runner stable; golden set ≥50 przypadków	Golden set 100% pass; compaction 5x verified; rollback po failed CFT działa
syllion.memory.evidence_store	Każda decyzja D2+ z pełnym evidence pack; immutable store; evidence query <200ms; cross-reference z event_bus	evidence_spine stable; decision_gate_engine stable	100 evidence packów złożonych; immutability verified; query SLA spełnione
syllion.memory.self_model_store	AEIS self-model: własne metryki, wnioski, ograniczenia; versioned; write tylko przez self_observation D0-D1	self_observation stable; kanon_access readable	Self-model po 24h obserwacji; version history ≥10 snapshots; read/write separation verified
syllion.memory.kb_adapter	Unified knowledge base interface; Confluence/Notion/S3 adaptery; freshness <1h; relevance score	kanon_access stable; connector_framework stable	3 adaptery KB działają; freshness SLA <1h; relevance scoring aktywny

Module ID	DoD produkcyjny	Entry gate	Exit gate
syllion.memory.indexer	pgvector indexing dla wszystkich dokumentów; incremental re-index; index freshness <5min p95	Postgres 16 + pgvector dostępny; kanon_access stable	100.000 dokumentów zaindeksowanych; re-index <5min; cosine similarity poprawna
syllion.memory.retrieval	Hybrid search (keyword + vector); <150ms p95; re-ranking; citation trail do źródła	indexer stable	1000 queries; precision@5 >0.8; latency p95 <150ms; citation trail 100% poprawny

Tabela D-3: Profil security M0 → M5

Module ID	Profil M0	Profil M2	Profil M5
syllion.memory.kanon_access	dev-light / bootstrap	staging-strict / oidc	prod-strict / oidc+signed
syllion.memory.compact_layer	dev-light / bootstrap	test-light / session	prod-strict / oidc
syllion.memory.evidence_store	dev-light / extended	test-light / extended	prod-strict / signed
syllion.memory.self_model_store	dev-light / bootstrap	test-light / session	prod-strict / oidc+signed
syllion.memory.kb_adapter	dev-light / bootstrap	test-light / light	prod-strict / mTLS
syllion.memory.indexer	dev-light / basic	test-light / extended	prod-strict / signed
syllion.memory.retrieval	dev-light / bootstrap	test-light / session	prod-strict / oidc

R3.6 — Klasa E: Governance (7 modułów)

Moduły governance egzekwują drabinkę D0–D5, mechanikę Council i bramy G-*. decision_ladder i council_workflow są na critical path (Priorytet 1). Bez nich żadna decyzja D2+ nie może być podjęta zgodnie z masterplanem.

Tabela E-1: Atrybuty podstawowe

Module ID	Owner Plan	Strategia	Depends on	Est. sprinty	Dział
syllion.governance.decision_ladder	P04	Greenfield	decision_gate_engine, evidence_spine	4	Governance
syllion.governance.council_workflow	P05	Greenfield	decision_ladder, roles, event_bus	4	Governance
syllion.governance.roles	P05+P15	Greenfield	contract_registry, auth_provider	3	Governance

Module ID	Owner Plan	Strategia	Depends on	Est. sprinty	Dział
syllion.governance.gates_registry	P12+P13	Greenfield	decision_gate_engine	3	Governance
syllion.governance.policy_registry	P11+Q	Greenfield	gates_registry, decision_ladder	3	Governance
syllion.governance.self_explanation_validator	P19+R	Greenfield	self_explanation, evidence_spine	3	AEIS
syllion.governance.evidence_workflow	P06	Greenfield	evidence_store, decision_ladder	3	Governance

Tabela E-2: DoD + bramy

Module ID	DoD produkcyjny	Entry gate	Exit gate
syllion.governance.decision_ladder	D0–D5 classification pipeline live; 100% decyzji D2+ z evidence pack; accuracy >95%; audit log	decision_gate_engine stable; evidence_spine stable	100 decyzji prawidłowo sklasyfikowanych; accuracy >95%; 0 decyzji D2+ bez evidence
syllion.governance.council_workflow	Council 4/4 quorum mechanics; asynchroniczny vote via UI/CLI; vote-to-decision <48h p95; full audit	roles stable; decision_ladder stable; event_bus stable	5 pełnych quorum-vote-decision flows; <48h p95; rollback po failed quorum działa
syllion.governance.roles	RBAC: 7 działów + 3 poziomy uprawnień; role assignment event-sourced; zero-trust enforcement	auth_provider dev-light; contract_registry stable	100% endpoint coverage RBAC; unauthorized access 0; role assignment auditable
syllion.governance.gates_registry	Wszystkie bramy G-* zarejestrowane i ewaluowane; gate pass/fail z evidence; nie-blokujące D0–D1	decision_gate_engine stable; policy_registry v1	50 bram G-* live; gate evaluation SLA <100ms; false-positive rate <1%
syllion.governance.policy_registry	Polityki jako kod (OPA lub Python); 30 polityk Aneksu Q; hot-reload bez restartu; violation log	gates_registry stable; decision_ladder stable	30 polityk wczytanych; hot-reload bez restartu; 0 naruszeń w pen-test golden set
syllion.governance.self_explanation_validator	Każde działanie AEIS ma self-explanation; validator ocenia spójność z meta-zasadami; rejection rate logowany	self_explanation module stable; evidence_spine stable	100 wyjaśnień walidowanych; spójność z meta-zasadami >95%; rejection audit trail pełny

Module ID	DoD produkcyjny	Entry gate	Exit gate
syllion.governance.evidence_workflow	Evidence workflow dla każdej decyzji D2+: propose → review → sign → store; SLA <24h	evidence_store stable; decision_ladder stable	50 workflows end-to-end; SLA <24h p95; immutability verified; Ed25519 signing petne

Tabela E-3: Profil security M0 → M5

Module ID	Profil M0	Profil M2	Profil M5
syllion.governance.decision_ladder	dev-light / bootstrap	staging-strict / oidc	prod-strict / oidc+signed
syllion.governance.council_workflow	dev-light / session	staging-strict / oidc	prod-strict / oidc+signed
syllion.governance.roles	dev-light / bootstrap	staging-strict / oidc	prod-strict / oidc
syllion.governance.gates_registry	dev-light / basic	test-light / extended	prod-strict / signed
syllion.governance.policy_registry	dev-light / basic	test-light / extended	prod-strict / signed
syllion.governance.self_explanation_validator	dev-light / basic	test-light / extended	prod-strict / signed
syllion.governance.evidence_workflow	dev-light / extended	staging-strict / oidc	prod-strict / signed

R3.7 — Klasa F: Security (8 modułów)

Wszystkie 8 modułów security budowane są od dnia 1 w profilu dev-light. Ich kontrakty (.proto) są stabilne od M0. Osiągnięcie M5 = podmiana implementacji na prod-strict — NIE zmiana architektury. Każda podmiana przechodzi przez shadow → dual → cutover z pełnym audit logiem.

Tabela F-1: Atrybuty podstawowe

Module ID	Owner Plan	Strategia	Depends on	Est. sprinty	Dział
syllion.security.auth_provider	P09	Greenfield	contract_registry	3	Security
syllion.security.bootstrap_init	P09	Greenfield	auth_provider	2	Security
syllion.security.session_broker	P09	Greenfield	auth_provider	3	Security
syllion.security.policy_engine	P09	Greenfield	roles, policy_registry	3	Security

Module ID	Owner Plan	Strategia	Depends on	Est. sprinty	Dział
sylicon.security.execution_guard	P09	Greenfield	phantom_wrapper	4	Security
sylicon.security.secret_provider	P09	Greenfield	auth_provider	2	Security
sylicon.security.audit_sink	P06+P09	Greenfield	evidence_spine, event_bus	3	Security
sylicon.security.phantom_wrapper	P09	Greenfield	execution_guard, auth_provider	4	Security

Tabela F-2: DoD + bramy

Module ID	DoD produkcyjny	Entry gate	Exit gate
sylicon.security.auth_provider	Token validation 100% endpoints; contract AuthProvider v1.0.0 stable; swap dev→prod bez zmiany API	contract_registry v1.0; bootstrap_init stable	100% endpoint coverage; OIDC swap tested; 0 auth bypass w pen-test golden set
sylicon.security.bootstrap_init	Secure system init; seed credentials nigdy w plain-text w prod; init idempotentny	auth_provider stable	Init działa; plain-text w dev tylko; hardware token w prod; idempotency verified
sylicon.security.session_broker	Session lifecycle pełny; token refresh bez przerwy; revocation <1s; Redis+JWT w prod	auth_provider stable	1000 sesji; refresh bez przerwy; revocation <1s; Redis persistence verified
sylicon.security.policy_engine	OPA (prod) / allow-list (dev); 100% requestów ewaluowanych; policy decision <10ms p95	roles stable; policy_registry v1	Wszystkie endpointy pokryte; OPA swap verified; decision latency <10ms p95
sylicon.security.execution_guard	Subprocess isolation (dev) / gVisor+Firecracker (prod); 0 escape z sandboxu; audit każdego exec	phantom_wrapper dev-light	Sandbox isolation verified; escape attempt blocked; gVisor swap tested; audit 100%
sylicon.security.secret_provider	.env (dev) / HashiCorp Vault (prod); kontrakt ten sam; secret rotation automated w prod	auth_provider stable	Secret rotation ≥1 cycle; Vault swap bez zmiany kontraktu; 0 secrets w logach
sylicon.security.audit_sink	Append-only audit log; Ed25519 signed (prod); zero-loss na failure; query SLA <500ms	evidence_spine stable; event_bus stable	10.000 eventów; zero-loss verified (kill test); signed chain verified; query SLA spełnione

Module ID	DoD produkcyjny	Entry gate	Exit gate
syllion.security.phantom_wrapper	AEIS-driven changes run w PHANTOM przed prod; sandbox isolation verified; pełny PHANTOM w M5	execution_guard dev-light	PHANTOM flow: propose→sandbox→verify→cutover pełny; isolation 100%; zero prod-breach

Tabela F-3: Profil security M0 → M5

Module ID	Profil M0	Profil M2	Profil M5
syllion.security.auth_provider	dev-light / bootstrap	staging-strict / oidc	prod-strict / oidc
syllion.security.bootstrap_init	dev-light / bootstrap	test-light / session	prod-strict / signed
syllion.security.session_broker	dev-light / bootstrap	staging-strict / oidc	prod-strict / oidc
syllion.security.policy_engine	dev-light / basic	staging-strict / oidc	prod-strict / signed
syllion.security.execution_guard	dev-light / light	test-light / light	prod-strict / strict
syllion.security.secret_provider	dev-light / bootstrap	staging-strict / oidc	prod-strict / signed
syllion.security.audit_sink	dev-light / basic	test-light / extended	prod-strict / signed
syllion.security.phantom_wrapper	dev-light / light	test-light / light	prod-strict / strict

Tabela F-4: Security Hardening Path (M0 → M5)

Poniższa tabela pokazuje konkretną podmiannę implementacji dla każdego modułu security. Kolumna „Contract version change?” potwierdza, że kontrakt .proto pozostaje niezmienny — podmiana nie wymaga migracji klientów.

Module ID	Dev impl (M0)	Prod impl (M5)	Contract version change?
syllion.security.auth_provider	Local UUID tokens / bootstrap	Keycloak OIDC / prod-strict	NONE (swap impl only)
syllion.security.bootstrap_init	Plaintext admin init	Secure seed + hardware token	NONE
syllion.security.session_broker	In-memory sessions	Redis + JWT + refresh token rotation	NONE
syllion.security.policy_engine	Allow-list basic	OPA + fine-grained policies	NONE
syllion.security.execution_guard	Subprocess isolation	gVisor / Firecracker + PHANTOM	NONE
syllion.security.secret_provider	.env + local file	HashiCorp Vault / prod secrets	NONE

Module ID	Dev impl (M0)	Prod impl (M5)	Contract version change?
syllion.security.audit_sink	Append log / basic	Ed25519 signed chain	NONE (audit_mode only)
syllion.security.phantom_wrapper	Skeleton (light sandbox)	Full PHANTOM strict / gVisor	NONE

R3.8 — Klasa G: Efficiency (4 moduły — P10)

Cztery wymiary efektywności: code bloat, runtime performance, memory footprint, cost envelope. Moduły G są aktywne od M2 — trackują metryki per commit i per request, egzekwując bramy G-EFF, G-PERF, G-MEM, G-COST.

Tabela G-1: Atrybuty podstawowe

Module ID	Owner Plan	Strategia	Depends on	Est. sprin ty	Dział
syllion.efficiency.code_bloat	P10	Greenfield	event_bus, CI pipeline	3	Jakość
syllion.efficiency.runtime_perf	P10	Greenfield	OTel, event_bus	3	DevOps
syllion.efficiency.memory_footp rint	P10	Greenfield	OTel, event_bus	2	DevOps
syllion.efficiency.cost_envelope	P10	Greenfield	model_router, event_bus	2	Finanse

Tabela G-2: DoD + bramy

Module ID	DoD produkcyjny	Entry gate	Exit gate
syllion.efficiency.code_bl oat	LoC delta + complexity tracked per commit; G-EFF gate blokuje D3+ bundles przekraczające budżet	CI pipeline stable; event_bus stable	G-EFF gate aktywny; 10 commitów z raportem; 1 bundle zablokowany w teście (poprawnie)
syllion.efficiency.runtime_ perf	SLO per endpoint w OTel; alert na regresję >10%; performance budget w gate G-PERF	OTel skonfigurowany; decision_gate_engine stable	SLO dla 20 endpointów; alert przetestowany; G-PERF gate blokuje regresję
syllion.efficiency.memory _footprint	Heap + container memory tracked; memory budget per moduł; alert na leak >5% trend w 24h	OTel skonfigurowany; event_bus stable	Memory profiling 10 modułów; leak detection przetestowany; budżety enforced

Module ID	DoD produkcyjny	Entry gate	Exit gate
syllion.efficiency.cost_envelope	LLM token cost per request; total cost budget enforced; cost alert <5min lag	model_router stable; event_bus stable	Cost tracking dla 5 modeli; budget enforcement przetestowany; alert lag <5min

Tabela G-3: Profil security M0 → M5

Module ID	Profil M0	Profil M2	Profil M5
syllion.efficiency.code_bloat	dev-light / basic	test-light / extended	prod-strict / signed
syllion.efficiency.runtime_perf	dev-light / basic	test-light / extended	prod-strict / signed
syllion.efficiency.memory_footprint	dev-light / basic	test-light / extended	prod-strict / signed
syllion.efficiency.cost_envelope	dev-light / basic	test-light / extended	prod-strict / signed

R3.9 — Klasa H: AEIS Self-* (5 modułów — P19)

Pięć modułów realizujących pętlę samopoznania, samodoskonalenia i samozachowawczości AEIS. Aktywowane etapowo: self_observation od M4 (Observe-only), improvement_queue od M4 (Propose-only), self_limitation i self_preservation od M4 są aktywne zawsze. Działanie bez ograniczeń self_limitation lub self_preservation jest zabronione jako naruszenie meta-zasady Autonomy under Canon.

Tabela H-1: Atrybuty podstawowe

Module ID	Owner Plan	Strategia	Depends on	Est. sprinty	Dział
syllion.aeis.self_observation	P19	Greenfield	OTel, event_bus, self_model_store	4	AEIS
syllion.aeis.improvement_queue	P19	Greenfield	self_observation, decision_ladder	3	AEIS
syllion.aeis.self_explanation	P19+R	Greenfield	reasoner, evidence_spine	3	AEIS
syllion.aeis.self_limitation	P19+Q	Greenfield	policy_registry, gates_registry	4	AEIS
syllion.aeis.self_preservation	P19	Greenfield	decision_ladder, kanon_access	3	AEIS

Tabela H-2: DoD + bramy

Module ID	DoD produkcyjny	Entry gate	Exit gate
syllion.aeis.self_observation	100% runtime invocations OTel-covered; hourly reports; anomaly detection precision >90%; zero false-shutdown	OTel skonfigurowany; self_model_store stable	24h observation cycle; anomaly detection >90% precision; pełny OTel coverage verified
syllion.aeis.improvement_queue	Propozycje ulepszeń queued; D0–D1 auto, D2+ Council triage; improvement rate tracked; rollback dla każdej propozycji	self_observation stable; decision_ladder stable	10 propozycji E2E; D2+ Council routing; rollback tested; improvement rate metric aktywna
syllion.aeis.self_explanation	Każde działanie AEIS wyjaśnione w języku naturalnym; tłumaczenie drabiny D0–D5; audit log	reasoner stable; evidence_spine stable	50 wyjaśnień; zgodność z meta-zasadami >95%; wyjaśnienie każdej decyzji D2+ w evidencje
syllion.aeis.self_limitation	30 polityk z Aneksu Q enforced as code; 0 violations w pen-test; gate G-AEIS aktywny	policy_registry stable; gates_registry stable; Aneks Q finalny	30 polityk jako kod; 0 naruszeń w pen-test golden set; G-AEIS gate blokuje naruszenia
syllion.aeis.self_preservation	G-AEIS-05 active od dnia 1; auto-block of mission-threatening proposals; misja preserved w każdym scenariuszu	decision_ladder stable; kanon_access stable	Misja-preservation gate aktywna; 5 scenariuszy zagrożeń zablokowanych; audit trail pełny

Tabela H-3: Profil security M0 → M5

Module ID	Profil M0	Profil M2	Profil M5
syllion.aeis.self_observation	dev-light / basic	test-light / extended	prod-strict / signed
syllion.aeis.improvement_queue	dev-light / bootstrap	staging-strict / oidc	prod-strict / oidc+signed
syllion.aeis.self_explanation	dev-light / basic	test-light / extended	prod-strict / signed
syllion.aeis.self_limitation	dev-light / basic	staging-strict / oidc	prod-strict / signed
syllion.aeis.self_preservation	dev-light / bootstrap	staging-strict / oidc	prod-strict / signed

R3.10 — Klasa I: Skills & Demand (3 moduły)

Trójka modułów zarządza katalogiem umiejętności i sygnałami popytu. skills.registry publikuje lifecycle DRAFT→PUBLISHED→DEPRECATED. demand_signal automatycznie generuje propozycje dla D0/D1, kierując D2+ do Council.

Tabela I-1: Atrybuty podstawowe

Module ID	Owner Plan	Strategia	Depends on	Est. sprinty	Dział
syllion.skills.registry	P18	Greenfield	contract_registry, event_bus	3	Skills
syllion.skills.executor	P18	Greenfield	skills.registry, tool_runner	3	Skills
syllion.aeis.demand_signal	P18+P20	Greenfield	self_observation, skills.registry	3	AEIS

Tabela I-2: DoD + bramy

Module ID	DoD produkcyjny	Entry gate	Exit gate
syllion.skills.registry	30+ skilli standardowych published; lifecycle DRAFT→PUBLISHED→DEP RECATED live; skill.yaml validator	contract_registry stable; event_bus stable	30 skilli w rejestrze; lifecycle pełny; validator odrzuca 5 błędnych skill.yaml
syllion.skills.executor	Skill runtime: invoke, monitor, rollback; SLA <1s p95; skill isolation; error propagation poprawna	skills.registry stable; tool_runner stable	20 skilli uruchomionych; SLA <1s p95; rollback przetestowany; isolation verified
syllion.aeis.demand_signal	4 signal sources clustered; demand-to-skill-proposal automated dla D0/D1, Council dla D2+; trend tracking	self_observation stable; skills.registry stable	4 sygnały zintegrowane; 10 automatycznych propozycji; Council routing D2+ poprawne

Tabela I-3: Profil security M0 → M5

Module ID	Profil M0	Profil M2	Profil M5
syllion.skills.registry	dev-light / bootstrap	test-light / session	prod-strict / oidc
syllion.skills.executor	dev-light / light	test-light / light	prod-strict / strict
syllion.aeis.demand_signal	dev-light / basic	test-light / extended	prod-strict / signed

R3.11 — Klasa J: Surface (3 moduły — P17)

Warstwa prezentacyjna: gRPC-web/tRPC API gateway, Next.js operator console (8 paneli, 7 widoków RBAC) i WebSocket gateway. Budowana równolegle z Kernel na stubowanych endpointach. Priorytet 3 — nie blokuje M0–M2.

Tabela J-1: Atrybuty podstawowe

Module ID	Owner Plan	Strategia	Depends on	Est. sprinty	Dział
syllion.surface.console_api	P17	Greenfield	auth_provider, decision_gate_engine	3	UI
syllion.surface.console_ui	P17	Greenfield	console_api, ws_gateway	4	UI
syllion.surface.ws_gateway	P17	Greenfield	console_api, event_bus	2	UI

Tabela J-2: DoD + bramy

Module ID	DoD produkcyjny	Entry gate	Exit gate
syllion.surface.console_api	gRPC-web/tRPC gateway; 100% endpoints z auth; SLA <200ms p95; RBAC per endpoint	auth_provider stable; roles stable	Wszystkie endpointy pokryte; SLA <200ms p95; RBAC unauthorized=0; OpenAPI spec aktualna
syllion.surface.console_ui	8 paneli + 7 widoków RBAC; WebSocket push <100ms p95; pełny responsiveness (mobile+desktop)	console_api stable; ws_gateway stable	8 paneli live; RBAC verified; WebSocket <100ms p95; Lighthouse accessibility ≥90
syllion.surface.ws_gateway	WebSocket push dla wszystkich eventów; reconnect automatyczny; message ordering preserved	event_bus stable; console_api stable	1000 concurrent connections; reconnect <3s; message ordering 100%; push latency <100ms p95

Tabela J-3: Profil security M0 → M5

Module ID	Profil M0	Profil M2	Profil M5
syllion.surface.console_api	dev-light / bootstrap	staging-strict / oidc	prod-strict / oidc
syllion.surface.console_ui	dev-light / session	staging-strict / oidc	prod-strict / oidc
syllion.surface.ws_gateway	dev-light / bootstrap	test-light / session	prod-strict / oidc

R3.12 — Klasa K: Rebuildability (4 moduły)

Cztery moduły realizują meta-zasadę Rebuildability over Lineage: pełna orkiestracja przebudowy greenfield, Legacy Preservation Window (7 dni), state machine shadow→dual→cutover oraz runner CFT. Aktywne od M3, wymagane przed każdym wdrożeniem produkcyjnym.

Tabela K-1: Atrybuty podstawowe

Module ID	Owner Plan	Strategia	Depends on	Est. sprinty	Dział
syllion.rebuild.orchestrator	P14	Greenfield	manifest_loader, env_orchestrator	4	DevOps
syllion.rebuild.lpw_manager	P14	Greenfield	rebuild.orchestrator	2	DevOps
syllion.rebuild.cutover_controller	P14	Greenfield	rebuild.orchestrator, event_bus	3	DevOps
syllion.rebuild.cft_runner	P16	Greenfield	quality.golden_set_registry	3	Jakość

Tabela K-2: DoD + bramy

Module ID	DoD produkcyjny	Entry gate	Exit gate
syllion.rebuild.orchestrator	Pełny greenfield rebuild modułu automated; LPW 7 dni; CFT + golden set pass required; no-downtime	manifest_loader stable; env_orchestrator stable; cft_runner stable	3 moduły przebudowane end-to-end; LPW enforced; CFT pass required; 0 downtime
syllion.rebuild.lpw_manager	Legacy Preservation Window (7 dni) dla każdego deprecated modułu; auto-purge po LPW; rollback w oknie	rebuild.orchestrator stable	LPW enforced dla 3 modułów; rollback w oknie przetestowany; auto-purge po 7 dniach
syllion.rebuild.cutover_controller	Shadow→dual→cutover state machine; weight routing; auto-rollback na regresję; cutover audit log	rebuild.orchestrator stable; event_bus stable	3 pełne shadow→dual→cutover flows; auto-rollback na regresji przetestowany; audit log pełny
syllion.rebuild.cft_runner	Compact Fidelity Test runner; golden set management; CFT pass/fail signal dla compact_layer i rebuild	quality.golden_set_registry stable	Golden set ≥50 przypadków; CFT runner wykonuje w <5min; signal dla compact_layer poprawny

Tabela K-3: Profil security M0 → M5

Module ID	Profil M0	Profil M2	Profil M5
syllion.rebuild.orchestrator	dev-light / session	staging-strict / oidc	prod-strict / oidc+signed
syllion.rebuild.lpw_manager	dev-light / basic	test-light / extended	prod-strict / signed

Module ID	Profil M0	Profil M2	Profil M5
syllion.rebuild.cutover_controller	dev-light / session	staging-strict / oidc	prod-strict / oidc+signed
syllion.rebuild.cft_runner	dev-light / basic	test-light / extended	prod-strict / signed

R3.13 — Klasa L: Quality (3 moduły — P11)

Trójka modułów zapewnia jakość całego systemu: rejestr golden setów, runner testów (contract+integration+performance) i detektor regresji z auto-rollback. Quality jest fundamentem dla Rebuildability — bez golden setów CFT nie może być wykonany.

Tabela L-1: Atrybuty podstawowe

Module ID	Owner Plan	Strategia	Depends on	Est. sprinty	Dział
syllion.quality.golden_set_registry	P11	Greenfield	contract_registry, event_bus	3	Jakość
syllion.quality.test_runner	P11	Greenfield	golden_set_registry, event_bus	3	Jakość
syllion.quality.regression_detector	P11	Greenfield	test_runner, rebuild.orchestrator	2	Jakość

Tabela L-2: DoD + bramy

Module ID	DoD produkcyjny	Entry gate	Exit gate
syllion.quality.golden_set_registry	Golden test sets dla każdego modułu; lifecycle managed; min 20 cases per moduł; versioned	contract_registry stable	20 golden sets; versioning pełne; diff przy update pokazuje regresje; coverage 100% klas
syllion.quality.test_runner	Contract/integration/performance tests w jednym runnerze; parallel execution; raport w OTel	golden_set_registry stable; event_bus stable	Pełny test suite (contract+integration+perf) dla 10 modułów; raport w OTel; parallel runs
syllion.quality.regression_detector	Auto-rollback trigger przy regresji >threshold; alert w <5min; rollback audit trail	test_runner stable; rebuild.orchestrator stable	Regression trigger przetestowany; rollback automated; alert lag <5min; audit trail pełny

Tabela L-3: Profil security M0 → M5

Module ID	Profil M0	Profil M2	Profil M5
syllion.quality.golden_set_registry	dev-light / basic	test-light / extended	prod-strict / signed
syllion.quality.test_runner	dev-light / basic	test-light / extended	prod-strict / signed
syllion.quality.regression_detector	dev-light / basic	test-light / extended	prod-strict / signed

R3.14 — Tabela sumaryczna: wszystkie 65 modułów

Jedna tabela — pełny spis inwentarzowy wszystkich modułów systemu SYLION. Priorytet: 1 = critical path (blokuje równoległe prace; budowanie od M0), 2 = ważne (M1–M3, równoległe po Contract Freeze), 3 = można opóźnić (M4+, niezależne od Kernel).

Module ID	Klasa	Owner Plan	Est. sprinty	Priorytet
syllion.core.module_registry	A: Kernel	P01	3	1
syllion.core.manifest_loader	A: Kernel	P01	2	1
syllion.core.contract_registry	A: Kernel	P01	3	1
syllion.core.event_bus	A: Kernel	P01	3	1
syllion.core.decision_gate_engine	A: Kernel	P04+P12	4	1
syllion.core.evidence_spine	A: Kernel	P06	3	1
syllion.core.environment_orchestrator	A: Kernel	P01	3	1
syllion.core.bundle_assembler	A: Kernel	P01	2	1
syllion.cognitive.planner	B: Cognitive	P08	4	2
syllion.cognitive.evaluator	B: Cognitive	P08	3	2
syllion.cognitive.reasoner	B: Cognitive	P08	4	2
syllion.cognitive.code_agent	B: Cognitive	P08	4	2
syllion.cognitive.context_builder	B: Cognitive	P02+P16	3	2
syllion.cognitive.model_router	B: Cognitive	P08	3	2
syllion.cognitive.llm_adapter	B: Cognitive	P08	2	2
syllion.execution.tool_runner	C: Execution	P07	3	2
syllion.execution.connector_framework	C: Execution	P07	3	2
syllion.execution.workflow_engine	C: Execution	P07	4	2
syllion.execution.job_runner	C: Execution	P07	2	2
syllion.execution.adapter_bus	C: Execution	P07	2	3
syllion.execution.retry_orchestrator	C: Execution	P07	2	3
syllion.memory.kanon_access	D: Memory	P02	4	1
syllion.memory.compact_layer	D: Memory	P16	4	1

Module ID	Klasa	Owner Plan	Est. sprint y	Priorytet
syllion.memory.evidence_store	D: Memory	P06	3	1
syllion.memory.self_model_store	D: Memory	P19	3	2
syllion.memory.kb_adapter	D: Memory	P18	3	3
syllion.memory.indexer	D: Memory	P02+P16	3	2
syllion.memory.retrieval	D: Memory	P02+P16	2	2
syllion.governance.decision_ladder	E: Governance	P04	4	1
syllion.governance.council_workflow	E: Governance	P05	4	1
syllion.governance.roles	E: Governance	P05+P15	3	1
syllion.governance.gates_registry	E: Governance	P12+P13	3	1
syllion.governance.policy_registry	E: Governance	P11+Q	3	2
syllion.governance.self_explanation_validator	E: Governance	P19+R	3	2
syllion.governance.evidence_workflow	E: Governance	P06	3	1
syllion.security.auth_provider	F: Security	P09	3	1
syllion.security.bootstrap_init	F: Security	P09	2	1
syllion.security.session_broker	F: Security	P09	3	1
syllion.security.policy_engine	F: Security	P09	3	1
syllion.security.execution_guard	F: Security	P09	4	1
syllion.security.secret_provider	F: Security	P09	2	1
syllion.security.audit_sink	F: Security	P06+P09	3	1
syllion.security.phantom_wrapper	F: Security	P09	4	2
syllion.efficiency.code_bloat	G: Efficiency	P10	3	2
syllion.efficiency.runtime_perf	G: Efficiency	P10	3	2
syllion.efficiency.memory_footprint	G: Efficiency	P10	2	3
syllion.efficiency.cost_envelope	G: Efficiency	P10	2	2
syllion.aeis.self_observation	H: AEIS Self-*	P19	4	2
syllion.aeis.improvement_queue	H: AEIS Self-*	P19	3	2
syllion.aeis.self_explanation	H: AEIS Self-*	P19+R	3	2

<i>Module ID</i>	<i>Klasa</i>	<i>Owner Plan</i>	<i>Est. s print y</i>	<i>Priorytet</i>
syllion.aeis.self_limitation	H: AEIS Self-*	P19+Q	4	2
syllion.aeis.self_preservation	H: AEIS Self-*	P19	3	1
syllion.skills.registry	I: Skills	P18	3	3
syllion.skills.executor	I: Skills	P18	3	3
syllion.aeis.demand_signal	I: Skills	P18+P20	3	3
syllion.surface.console_api	J: Surface	P17	3	3
syllion.surface.console_ui	J: Surface	P17	4	3
syllion.surface.ws_gateway	J: Surface	P17	2	3
syllion.rebuild.orchestrator	K: Rebuild	P14	4	2
syllion.rebuild.lpw_manager	K: Rebuild	P14	2	2
syllion.rebuild.cutover_controller	K: Rebuild	P14	3	2
syllion.rebuild.cft_runner	K: Rebuild	P16	3	2
syllion.quality.golden_set_registry	L: Quality	P11	3	2
syllion.quality.test_runner	L: Quality	P11	3	2
syllion.quality.regression_detector	L: Quality	P11	2	2

R3.15 — Podsumowanie

Łącznie 65 modułów w 12 klasach (A–L). Każdy moduł jest deployable brick z własnym kontraktem .proto, lifecycle (draft→stable), i DoD. Komunikacja wyłącznie przez gRPC lub NATS — nigdy przez Python import.

- Krytyczna ścieżka (Priorytet 1): ~24 moduły — Kernel (8) + Memory core (kanon_access, compact_layer, evidence_store) + Governance core (decision_ladder, council_workflow, roles, gates_registry, evidence_workflow) + Security wszystkie (8) + self_preservation
- Równoległe (Priorytet 2–3): ~41 modułów — startują po Contract Freeze (R2), równoległe w 8 strumieniach
- Estymacja łączna: ~220 sprint-osób (20 sprintów × 11 osób średnio) — przy pełnym równoległym rozwoju strumieni

ZASADA · 65 modułów — realna skala projektu

65 modułów wygląda na dużo. W rzeczywistości 8 (Kernel) + 15 (M1 Governance+Memory core + Security skeleton) = 23 moduły to 1/3 projektu czasowo — budowane w strumieniach 1+7 przez pierwsze 4 sprinty. Pozostałe 42 moduły ruszają równolegle po Contract Freeze. Żadne z nich nie blokują Kernel — działają na stubowanych kontraktach do momentu, gdy Kernel osiągnie stan stable.

ROZDZIAŁ R4 · AEIS IMPLEMENTATION MASTERPLAN

Critical Path i 8 strumieni równoległych

R4 — Critical Path i 8 strumieni równoległych

R4.0 — Wprowadzenie

Architektura Lego działa tylko wtedy, gdy znamy krytyczną ścieżkę. Bez critical path równoległość jest iluzją — wszystko czeka na blocker. Niniejszy rozdział precyzuje, które z ~65 modułów SYLION AEIS leżą na ścieżce krytycznej, jakie zależności między nimi wymuszają sekwencję, oraz jak pozostałe ~50 modułów może biec równolegle po odblokowaniu Contract Freeze (M0).

Organizacja pracy podzielona jest na 8 strumieni budowy, z których Strumień 1 (Kernel) jest fundamentem niezbędnym do startu wszystkich pozostałych. Po zamrożeniu kontraktów (koniec sprintu 2) do 6 strumieni może pracować równolegle — przy maksymalnym zatrudnieniu rządu 24–28 osób w tym samym czasie.

R4.1 — Critical Path — sekwencja blockerów

Critical path to minimalna sekwencja modułów i kamieni milowych, bez których nie jest możliwe uruchomienie żadnego z pozostałych strumieni. Poniższy diagram tekstowy ilustruje zależności topologiczne od sprintu 1 do sprintu 20, z trzema głównymi blokerami:

- CF — Contract Freeze (koniec sprintu 2, M0): zamrożenie wszystkich kontraktów, manifestów i taksonomii eventów.
- G — Governance Live (koniec sprintu 4, M1): drabina D0–D5 i Council workflow działają end-to-end.
- AEIS — AEIS Self-* Live (koniec sprintu 16, M4): wszystkie 5 modułów self-* w stanie stable; warunek konieczny dla autonomii stages 2–5.

[CF=Contract Freeze] [G=Governance live] [AEIS=AEIS Self-* live]

sprint 1-2:

```

module_registry ┌
manifest_loader ┤
contract_registry ┤ → CF (end sprint 2)
event_bus       ┤
decision_gate   └

```

sprint 3-4:

```

CF → governance.decision_ladder ┌
CF → governance.council_workflow ┤ → G (end sprint 4)
CF → evidence_spine              ┤
CF → memory.kanon_access         └

```

```

sprint 5-8:
  G → wszystkie cognitive (7, parallel)
  G → wszystkie execution (6, parallel)
  G → efficiency (4, parallel)
  G → quality (3, parallel)

sprint 9-12:
  rebuild orchestrator + cutover controller + CFT runner + LPW
  console API + UI + WS gateway (parallel)
  memory indexer + retrieval (parallel)

sprint 13-16:
  skills registry + executor + demand signal
  aeis self-* (5 modułów, parallel)
  → AEIS

sprint 17-20:
  Autonomy stages 1→5
  security hardening swap (8 modułów lifecycle shadow→dual→cutover)
  K8s migration
  external pen-test

```

Lista 15 modułów na Critical Path

Poniższa tabela identyfikuje dokładnie 15 modułów leżących na ścieżce krytycznej SYLION AEIS. Każde opóźnienie w tej kolumnie kaskadowo opóźnia wszystkie zależne strumienie. Wszystkie pozostałe ~50 modułów może biec równolegle po odblokowaniu właściwego blokera.

Moduł	Kolejn ość	Bloker dla	Estymacja (sprinty)
syllion.core.module_registry	1	CF, wszystko	1-2
syllion.core.manifest_loader	2	CF, wszystko	1-2
syllion.core.contract_registry	3	CF, buf breaking detection	1-2
syllion.core.event_bus	4	CF, taksonomia eventów	1-2
syllion.core.decision_gate_engine	5	CF, Governance Live	1-2
syllion.core.evidence_spine	6	G, audit całego systemu	3-4
syllion.governance.decision_ladder	7	G, bramy D0-D5 end-to-end	3-4
syllion.governance.council_workflow	8	G, Council 4/4 dla D3+	3-4
syllion.memory.kanon_access	9	G, Księga dostępna przez kontrakt	3-4
syllion.core.environment_orchestrator	10	Rebuild, K8s migration	1-2
syllion.core.bundle_assembler	11	Integration Assembly v1	9-10
syllion.rebuild.orchestrator	12	AEIS self-change, cutover	9-10

Moduł	Kolejn ość	Bloker dla	Estymacja (sprinty)
syllion.rebuild.cutover_controller	13	shadow→dual→cutover lifecycle	9–10
syllion.aeis.self_observation	14	AEIS Live, autonomy stage 1	13–14
syllion.aeis.improvement_queue	15	AEIS Live, autonomy stage 2	14–16

ZASADA · Reguła Critical Path

Wszystko POZA tymi 15 modułami może biec równolegle po odblokowaniu właściwego kamienia milowego (CF lub G). Ścieżka krytyczna nie dotyczy całych klas modułów — dotyczy precyzyjnie 15 węzłów w grafie zależności.

R4.2 — 8 strumieni budowy

System budowany jest w 8 równoległych strumieniach organizacyjnych. Każdy strumień ma dedykowanego właściciela, zdefiniowane outputs i jawne zależności od innych strumieni. Poniżej szczegółowy opis każdego strumienia wraz z tabelą modułów.

Strumień 1 — Kernel & Orchestration (Dział Architektury)

Owner: lead architect + 2 devs. Estymacja: sprinty 1–2 (M0).

Strumień 1 jest pre-condition dla wszystkich pozostałych siedmiu strumieni. Jego jedynym zadaniem jest dostarczenie zamrożonych kontraktów, manifestów, mechanizmu rejestracji modułów i szkieletu Kernel gotowego do walidacji przez buf i import-linter. Żaden inny strumień nie może dostarczyć ani jednego modułu w stanie validate dopóki Kernel nie osiągnie Contract Freeze. Dlatego Strumień 1 ma podwójne zasoby i codzienny standup przez pierwsze dwa tygodnie.

- Outputs: 8 modułów klasy Kernel (A).
- Blokuję: WSZYSTKO — pre-M0 freeze.

Moduł	Klasa	Strategia	Stan docelowy (M0)
syllion.core.module_registry	Kernel	Greenfield	validate
syllion.core.manifest_loader	Kernel	Greenfield	validate
syllion.core.contract_registry	Kernel	Greenfield	validate
syllion.core.event_bus	Kernel	Greenfield (NATS JetStream)	validate
syllion.core.decision_gate_engine	Kernel	Greenfield	validate
syllion.core.evidence_spine	Kernel	Greenfield	validate

<i>Moduł</i>	<i>Klasa</i>	<i>Strategia</i>	<i>Stan docelowy (M0)</i>
syllion.core.environment_orchestrator	Kernel	Greenfield	validate
syllion.core.bundle_assembler	Kernel	Greenfield	validate

Strumień 2 — Cognitive (Dział Implementacji)

Owner: AI/ML lead + 2 devs. Estymacja: sprinty 5–10 (M2).

Strumień 2 buduje warstwę rozumowania SYLION AEIS: planner, evaluator, reasoner, code agent, context builder, model router i LLM adapter. Moduły cognitive działają na fake-LLM adapter (stub) dopóki Model Router nie jest gotowy; następnie przełączają się na LiteLLM. Zależność od Kernel Freeze (kontrakty) i Model Router stub w M0 oznacza, że strumień może ruszać po końcu sprintu 2 (koniec M0), ale pełna integracja z Governance (bramy D2+ dla zmian modelu) następuje po Governance Live (koniec sprintu 4).

- Outputs: 7 modułów klasy Cognitive (B).
- Zależy od: Kernel Freeze (kontrakty), Model Router stub w M0.
- Może startować: koniec M0 (sprint 3+).

<i>Moduł</i>	<i>Owner Plan</i>	<i>Strategia</i>	<i>Stan docelowy (M2)</i>
syllion.cognitive.planner	P08	Greenfield	stable
syllion.cognitive.evaluator	P08	Greenfield	stable
syllion.cognitive.reasoner	P08	Greenfield	stable
syllion.cognitive.code_agent	P08	Greenfield	stable
syllion.cognitive.context_builder	P02+P16	Greenfield	stable
syllion.cognitive.model_router	P08	Greenfield (LiteLLM)	stable
syllion.cognitive.llm_adapter	P08	Greenfield	stable

Strumień 3 — Execution (Dział Implementacji)

Owner: platform lead + 2 devs. Estymacja: sprinty 5–10 (M2).

Strumień 3 buduje warstwę wykonawczą: tool runner, connector framework, workflow engine, job runner, adapter bus i retry orchestrator. Moduły Execution wymagają działającego Event Bus (NATS JetStream) i podstawowych kontraktów gRPC — stąd zależność od Kernel + Event Bus. Governance Live (G) jest wymagane do uruchomienia bram D2+ dla workflow engine, dlatego pełna walidacja następuje po sprincie 4.

- Outputs: 6 modułów klasy Execution (C).
- Zależy od: Kernel + Event Bus, basic contracts.
- Może startować: sprint 3+.

Moduł	Owner Plan	Strategia	Stan docelowy (M2)
syllion.execution.tool_runner	P07	Greenfield	stable
syllion.execution.connector_framework	P07	Greenfield	stable
syllion.execution.workflow_engine	P07	Greenfield	stable
syllion.execution.job_runner	P07	Greenfield	stable
syllion.execution.adapter_bus	P07	Greenfield	stable
syllion.execution.retry_orchestrator	P07	Greenfield	stable

Strumień 4 — Memory (Dział Implementacji + Architektura)

Owner: data lead + 2 devs. Estymacja: sprinty 3–10 (memory.kanon_access wczesny — M1).

Strumień 4 dostarcza warstwę pamięci: Księgę Access Layer, compact layer, evidence store, self-model store, KB adapter, indexer i retrieval. memory.kanon_access jest wyjątkiem — musi być gotowy na koniec sprintu 4 (M1), ponieważ Governance Live wymaga dostępu do Księgi przez kontrakt. Pozostałe moduły memory (indexer, retrieval) dojrzewają do stanu stable w sprincie 9–10, kiedy pgvector jest w pełni skonfigurowany.

- Outputs: 7 modułów klasy Memory (D).
- Zależy od: Postgres+pgvector ready, Kernel Freeze.
- Może startować: sprint 3+ (kanon_access krytyczny dla M1).

Moduł	Owner Plan	Strategia	Priorytet
syllion.memory.kanon_access	P02	Greenfield	KRYTYCZNY (M1)
syllion.memory.compact_layer	P16	Greenfield	M2
syllion.memory.evidence_store	P06	Greenfield	M1
syllion.memory.self_model_store	P19	Greenfield	M3
syllion.memory.kb_adapter	P18	Greenfield	M3
syllion.memory.indexer	P02+P16	Greenfield (pgvector)	M3
syllion.memory.retrieval	P02+P16	Greenfield	M3

Strumień 5 — Governance (Dział Compliance & Governance)

Owner: compliance lead + 1 dev + AEIS Steward. Estymacja: sprinty 3–8 (kluczowe w M1).

Strumień 5 dostarcza infrastrukturę decyzyjną SYLION AEIS: drabinę D0–D5, workflow Council, rejestry bram i polityk, Self-Explanation Validator oraz Evidence Workflow. Moduły governance.decision_ladder i governance.council_workflow leżą na critical path (muszą być stable do końca sprintu 4). Pozostałe moduły Governance dojrzewają do końca sprintu 8 (M2).

AEIS Steward uczestniczy w tym strumieniu od początku jako gwarant zgodności z Aneksem Q (Self-Limitation) i Aneksem R (Self-Explanation).

- Outputs: 7 modułów klasy Governance (E).
- Zależy od: Kernel + Evidence Spine.
- Może startować: sprint 3+.

Moduł	Owner Plan	Strategia	Stan docelowy
syllion.governance.decision_ladder	P04	Greenfield	stable (M1)
syllion.governance.council_workflow	P05	Greenfield	stable (M1)
syllion.governance.roles	P05+P15	Greenfield	stable (M1)
syllion.governance.gates_registry	P12+P13	Greenfield	stable (M2)
syllion.governance.policy_registry	P11+Q	Greenfield	stable (M2)
syllion.governance.self_explanation_validator	P19+R	Greenfield	stable (M2)
syllion.governance.evidence_workflow	P06	Greenfield	stable (M1)

Strumień 6 — Console (Dział Implementacji / UX)

Owner: UX lead + 2 frontend devs. Estymacja: sprinty 3–12 (M3).

Strumień 6 buduje warstwę operatorską: console_api (gRPC-web/tRPC gateway), console_ui (Next.js 14 + shadcn/ui) i ws_gateway (WebSocket push). Kluczowa zaleta tego strumienia: zależy tylko od Kernel — przez stubowane endpointy może działać w pełni równolegle z warstwami Cognitive, Execution i Memory. UX może iterować na mockach i stopniowo podłączać żywe kontrakty w miarę dojrzewania innych strumieni.

- Outputs: 3 moduły klasy Surface (J).
- Zależy od: tylko Kernel (stubowane endpointy dla wszystkiego innego).
- Może startować: sprint 3+ (równolegle z całą resztą).

Moduł	Owner Plan	Stack	Stan docelowy (M3)
syllion.surface.console_api	P17	gRPC-web / tRPC gateway	stable
syllion.surface.console_ui	P17	Next.js 14 + shadcn/ui	stable
syllion.surface.ws_gateway	P17	WebSocket push (NATS fan-out)	stable

Strumień 7 — Security Skeleton (Dział Security)

Owner: security lead + 1 dev. Estymacja: sprinty 3–8 (podstawy; pełne hardening w M5).

Strumień 7 buduje 8 modułów security w profilu dev-light/test-light. Kluczowa zasada: kontrakty są identyczne z produkcyjnymi — różni się tylko implementacja. Dzięki temu

Strumień 8 (Security Hardening) wymienia wkład w to samo gniazdo bez zmiany architektury. Moduły security muszą być w stanie validate już od M0 — bootstrap_init i auth_provider są prerekwizytami dla uruchamiania jakiegokolwiek modułu w środowisku CI.

- Outputs: 8 modułów klasy Security (F) w profilu dev-light.
- Zależy od: Kernel + Contract Registry.
- Może startować: sprint 3+.

Moduł	Profil M0–M4	Profil M5 (docelowy)	Owner Plan
syllion.security.auth_provider	dev-light (local token)	Keycloak OIDC	P09
syllion.security.bootstrap_init	dev-light (plaintext init)	secure seed + hardware token	P09
syllion.security.session_broker	dev-light (in-memory)	Redis + JWT + refresh rotation	P09
syllion.security.policy_engine	dev-light (allow-list basic)	OPA + fine-grained	P09
syllion.security.execution_guard	dev-light (subprocess iso.)	PHANTOM + gVisor/Firecracker	P09
syllion.security.secret_provider	dev-light (.env + local file)	HashiCorp Vault	P09
syllion.security.audit_sink	dev-light (append log)	Ed25519 signed chain	P06+P09
syllion.security.phantom_wrapper	dev-light (light sandbox)	full PHANTOM strict	P09

Strumień 8 — Security Hardening (Dział Security + SRE + pen-test team)

Owner: security lead + SRE lead + external pen-test. Estymacja: sprinty 17–20 (M5).

Strumień 8 nie buduje nowych modułów — wymienia implementacje 8 modułów security z dev-light/test-light na prod-strict. Każda wymiana przechodzi pełen lifecycle: shadow → dual → cutover, z Legacy Preservation Window 7 dni. Wymaga, żeby cały system działał na stagingu z profilem staging-strict (M4 complete). Równolegle SRE wykonuje migrację Docker Compose → Kubernetes i external pen-tester weryfikuje każdy moduł po jego cutover.

- Outputs: wymiana implementacji w 8 modułach Security do prod-strict.
- Zależy od: całości systemu na staging (M4 complete).
- Może startować: sprint 17+.

AEIS · Modularity by Contract w Hardening

Strumień 8 jest wzorcowym przykładem zasady Modularity by Contract: ten sam kontrakt .proto, nowa implementacja. System nie zatrzymuje się na czas hardening — lifecycle shadow/dual gwarantuje zero-downtime swap.

R4.3 — Synchronization Points

Synchronization Points to momenty, w których niezależne strumienie muszą osiągnąć określony stan przed kontynuacją. Nie są to punkty blokady całego projektu — są to bramki, po przejściu których odblokowane strumienie zyskują autonomię na kolejne kilka sprintów.

Sync Point	Sprint	Strumienie uczestniczące	Deliverable	Gate
Contract Freeze (CF)	koniec 2	Strumień 1 → odblokowanie 2, 3, 4, 5, 6, 7	Kernel validate; wszystkie kontrakty zamrożone; manifest schema, event taxonomy, lifecycle, dependency rules, security profiles zatwierdzone	buf lint clean; import-linter clean; Council D3 approval na Contract Freeze
Governance Live (G)	koniec 4	Strumień 5 → odblokowanie 2, 3, 4 dla D2+ gates	decision_ladder stable; council_workflow stable; evidence_spine stable; kanon_access stable	D2+ gates end-to-end test pass; Council 4/4 vote na G milestone
Integration Assembly v1	koniec 10	Strumienie 2, 3, 4, 5, 6, 7 → bundle_assembler	Pierwszy spójny bundle wszystkich klas A–F złożony do shadow deploy na staging	contract conformance tests zielone; shadow traffic ≥1h bez regresji
AEIS Observe-only Live	koniec 14	Strumień AEIS (Self-*) → pre-condition stages 2–5	self_observation stable; improvement_queue stable; autonomy stage 1 aktywny	zero modyfikacji pr odukcyjnych w stage 1; hourly raporty OK przez 7 dni
Security Hardening Start	start 17	Strumień 8 → wymaga M4 complete	Wszystkie klasy A–L stable; staging-strict profile działa przez ≥14 dni	external security review M4; SRE sign-off na staging; pen-test scope approved

R4.4 — Analiza maksymalnej równoległości

Po Contract Freeze (koniec sprintu 2) ilość równoległe aktywnych strumieni rośnie znacząco. Poniżej analiza per faza:

Faza 1: Sprints 1–2 (Kernel only)

Aktywny wyłącznie Strumień 1. Jeden zespół (lead architect + 2 devs) pracuje w izolacji. Pozostałe strumienie mogą przygotowywać szkielety i tworzyć moduły w stanie draft, ale żaden moduł nie wchodzi w fazę build bez zamrożonych kontraktów.

- Aktywne strumienie: 1 (+ draft 2–7 opcjonalnie)
- Aktywne zespoły: 1×3 osoby = 3 osoby

Faza 2: Sprinty 3–4 (post-CF, pre-G)

Po Contract Freeze startują strumienie 2, 3, 4, 5, 6 i 7 równolegle. Strumień 1 kontynuuje dopracowanie (environment_orchestrator, bundle_assembler). 6 aktywnych strumieni budowy + 1 kończący Strumień 1.

- Aktywne strumienie: 6 + 1 finalizujący = 7
- Aktywne zespoły: 7×3 osoby = ~21 osób

Faza 3: Sprinty 5–8 (peak parallelism)

Po Governance Live (G, koniec sprintu 4) wszystkie bramy D2+ działają. Strumienie 2, 3, 4, 5, 6 i 7 pracują z pełną prędkością. To szczytowy moment równoległości — 6 niezależnych strumieni, każdy z innym stackiem technologicznym i niezależną domeną.

- Aktywne strumienie: 6 (2, 3, 4, 5, 6, 7) — peak parallelism
- Aktywne zespoły: 6×3 –4 osoby = 18–24 osoby

Faza 4: Sprinty 9–12 (Integration Assembly)

Strumienie 2 (cognitive) i 3 (execution) kończą swoje outputs; startują rebuild (orchestrator, cutover controller, CFT runner, LPW), quality (golden set, test runner, regression detector) i integration assembly (bundle_assembler). Strumień 6 (Console) wchodzi w finalną fazę. 5–6 aktywnych strumieni.

- Aktywne strumienie: 5–6
- Aktywne zespoły: 5 – 6×3 osoby = 15–18 osób

Faza 5: Sprinty 13–16 (AEIS + Skills)

Strumień AEIS (Self-*), strumień Skills (registry, executor, demand signal) i finalizacja Console. Mniejsza równoległość, ale bardziej złożone zależności (AEIS wymaga całości systemu w stanie stable).

- Aktywne strumienie: 3–4
- Aktywne zespoły: 3 – 4×3 osoby = 9–12 osób

Faza 6: Sprinty 17–20 (Security Hardening + K8s)

Strumień 8 (Security Hardening), SRE K8s migration i external pen-test działają równolegle. Autonomy Rollout stages 3–5 aktywny równocześnie.

- Aktywne strumienie: Strumień 8 + SRE + pen-test + Autonomy
- Aktywne zespoły: ~8–12 osób specjalistycznych

ZASADA · Maksymalna równoległość projektu

Szczyt zatrudnienia równoległego to sprinty 5–8: 6–7 teamów × 3–4 osoby = 24–28 osób aktywnych w tym samym czasie. Minimum: zawsze jest coś do robienia — żaden strumień nie czeka dłużej niż 1 sprint na blocker. Lego działa, bo po CF każdy strumień jest autonomiczny w swojej domenie.

R4.5 — Graf zależności między modułami

Poniższa tabela przedstawia ~25 najważniejszych zależności w systemie. Typy zależności: contract — wywołanie gRPC na opublikowanym .proto; event — subskrypcja NATS JetStream; storage — dostęp przez widok opublikowany w Contract Registry.

Z modułu	Do modułu	Typ	Version constraint	Krytyczność
governance.council_workflow	core.decision_gate_engine	contract	>=1.0.0	critical
governance.council_workflow	core.evidence_spine	event	>=1.0.0	critical
governance.decision_ladder	core.event_bus	event	>=1.0.0	critical
governance.decision_ladder	core.decision_gate_engine	contract	>=1.0.0	critical
aeis.self_observation	core.event_bus	event	>=1.0.0	critical
aeis.improvement_queue	governance.council_workflow	contract	>=1.0.0	critical
aeis.self_explanation	governance.self_explanation_validator	contract	>=1.0.0	critical
aeis.self_limitation	governance.policy_registry	contract	>=1.0.0	critical
aeis.self_preservation	core.evidence_spine	event	>=1.0.0	critical
rebuild.orchestrator	core.environment_orchestrator	contract	>=1.0.0	critical
rebuild.cutover_controller	core.bundle_assembler	contract	>=1.0.0	critical
rebuild.cft_runner	memory.compact_layer	contract	>=1.0.0	standard
rebuild.lpw_manager	core.event_bus	event	>=1.0.0	standard
cognitive.planner	cognitive.model_router	contract	>=1.0.0	standard

Z modułu	Do modułu	Typ	Version constraint	Krytyczność
cognitive.context_builder	memory.kanon_access	contract	>=1.0.0	standard
cognitive.evaluator	core.evidence_spine	event	>=1.0.0	standard
execution.workflow_engine	core.event_bus	event	>=1.0.0	standard
execution.tool_runner	security.execution_guard	contract	>=1.0.0	critical
execution.connector_framework	security.auth_provider	contract	>=1.0.0	standard
memory.evidence_store	core.evidence_spine	event	>=1.0.0	critical
memory.indexer	memory.kanon_access	storage	>=1.0.0	standard
memory.retrieval	memory.indexer	contract	>=1.0.0	standard
skills.registry	memory.kb_adapter	contract	>=1.0.0	standard
skills.executor	execution.tool_runner	contract	>=1.0.0	standard
surface.console_api	core.module_registry	contract	>=1.0.0	standard
surface.ws_gateway	core.event_bus	event	>=1.0.0	standard

R4.6 — Ryzyko: Strumień 1 blokuje pozostałe

Opóźnienie Strumienia 1 (Kernel) o 1 sprint powoduje kaskadowe opóźnienie wszystkich pozostałych 7 strumieni o minimum 1 sprint. Przy 6–7 równoległych zespołach nawet 1-sprintowe opóźnienie Kernel to utrata ~20–28 osobotygodni produktywności.

Mitygacja:

- Podwójne zasoby Kernel: lead architect + backup architect — zawsze jest osoba zdolna do przejęcia workstreamu.
- Daily standup: codziennie przez pierwsze 2 tygodnie — blocker identyfikowany w ciągu 24h, nie 1 tygodnia.
- Code review tylko Kernel: cały bandwidth review skoncentrowany na Kernel przez pierwsze 2 sprinty — zero rozproszenia.

- Acceptance criteria jasne od dnia 0: Contract Freeze Checklist (7 elementów z R2) = Definition of Done dla Strumienia 1.

UWAGA · Strumień 1 — ostrzeżenie organizacyjne

Strumień 1 to nie jeden z 8. To fundament dla 7 pozostałych. Opóźnienie Kernel = opóźnienie wszystkich. Żadna decyzja organizacyjna w pierwszych 2 tygodniach nie jest ważniejsza niż dostarczenie Contract Freeze na czas.

R4.7 — Podsumowanie: dlaczego Lego działa

Architektura Lego działa w SYLION AEIS, ponieważ zależności między modułami są jawne, wersjonowane i egzekwowane przez narzędzia (buf, import-linter, contract conformance tests). Critical path obejmuje dokładnie 15 modułów. Pozostałe ~50 modułów może biec równolegle po odblokowaniu właściwego blotera — CF lub G. Dlatego projekt może zatrudniać 24–28 osób równolegle bez chaosu: każdy strumień wie, co blokuje i co jest zablokowane.

AUTONOMY·CANON · Critical Path w liczbach

Critical path to 15 modułów. Pozostałe ~50 biegnie równolegle. Dlatego Lego działa — większość pracy jest poza blockerem. Sekret tkwi w Contract Freeze: kiedy kontrakty są zamrożone, każdy strumień staje się niezależny w swojej domenie.

ROZDZIAŁ R5 · AEIS IMPLEMENTATION MASTERPLAN

Milestones M0–M5

Rozdział R5 — Milestones M0–M5 szczegółowo

R5.0 — Wprowadzenie

SYLION AEIS budowany jest w ciągu 12 miesięcy podzielonych na 6 kamieni milowych (M0–M5) i 20 sprintów 2-tygodniowych. Każdy milestone jest samodzielną „wyśpialną stacją” — po jego osiągnięciu system działa, bramy exit są zamknięte, a Evidence pack jest podpisany i zarchiwizowany. Milestones zostały policzone w sposób zachowawczy — real delivery może być szybsze przy pełnym equipamencie zespołu AI-aided.

- Timeline: 12 miesięcy / 6 milestone’ów (M0–M5) / 20 sprintów 2-tygodniowych
- Evidence pack: każdy milestone kończy się paczką dowodową podpisaną Ed25519, Board review oraz publicznym eventem masterplan.milestone.<N>.completed
- Zachowawcza estymacja: pełny team z AI-aided development może dostarczyć szybciej; plan zakłada równoległą pracę 8 strumieni od M0

Harmonogram wysokiego poziomu

Milesto ne	Sprinty	Miesiące	Główny cel	Event końcowy
M0	1–2	1	Contract Freeze + Kernel	masterplan.milestone.0.completed
M1	3–4	2–3	Governance + Memory core	masterplan.milestone.1.completed
M2	5–8	3–5	Cognitive + Execution + Efficiency	masterplan.milestone.2.completed
M3	9–12	5–7	Rebuildability + Surface	masterplan.milestone.3.completed
M4	13–16	7–9	Skills + AEIS Self-*	masterplan.milestone.4.completed
M5	17–20	9–12	Autonomy Full + Security + K8s	masterplan.milestone.5.completed

R5.1 — M0: Contract Freeze + Kernel (sprint 1–2, miesiąc 1)

Cel: „System ma fundament, na którym można równoległe pracować”. M0 dostarcza nieruchomą kość pacierzową — 8 modułów Kernel w stanie stable, 8 modułów Security skeleton w trybie dev-light, oraz 5 kluczowych modułów Governance core w stanie validate. Jednocześnie następuje Contract Freeze — 7 artefaktów wejściowych do równoległej implementacji zostaje zamrożonych (patrz R2).

Moduły dostarczane w M0

Moduł ID	Klasa	Stan	Profil security	Uwaga
syliion.core.module_registry	Kernel	stable	dev-light	Nieruchome jądro
syliion.core.manifest_loader	Kernel	stable	dev-light	Walidator manifestów
syliion.core.contract_registry	Kernel	stable	dev-light	40+ .proto
syliion.core.event_bus	Kernel	stable	dev-light	NATS JetStream
syliion.core.decision_gate_engine	Kernel	stable	dev-light	D0–D5 drabina
syliion.core.evidence_spine	Kernel	stable	dev-light	Append-only log
syliion.core.environment_orchestrator	Kernel	stable	dev-light	Docker Compose
syliion.core.bundle_assembler	Kernel	stable	dev-light	Bundle RPC flow
syliion.security.auth_provider	Security	dev-light	dev-light	Local token bootstrap
syliion.security.bootstrap_init	Security	dev-light	dev-light	Plaintext admin init
syliion.security.session_broker	Security	dev-light	dev-light	In-memory sessions
syliion.security.policy_engine	Security	dev-light	dev-light	Allow-list basic
syliion.security.execution_guard	Security	dev-light	dev-light	Subprocess isolation
syliion.security.secret_provider	Security	dev-light	dev-light	.env + local file
syliion.security.audit_sink	Security	dev-light	dev-light	Append log
syliion.security.phantom_wrapper	Security	dev-light	dev-light	Skeleton light sandbox
syliion.governance.decision_ladder	Governance	validate	dev-light	Governance core
syliion.governance.council_workflow	Governance	validate	dev-light	Governance core
syliion.governance.roles	Governance	validate	dev-light	Governance core
syliion.governance.gates_registry	Governance	validate	dev-light	Governance core

Moduł ID	Klasa	Stan	Profil security	Uwaga
syllion.governance.evidence_workflow	Governance	validate	dev-light	Governance core

Bramy wejścia i wyjścia M0

Brama	Typ	Kryterium zaliczenia
(brak)	Wejście	M0 = start projektu — brak bram wejściowych
G-FREEZE-01	Wyjście	7 artefaktów Contract Freeze zamrożonych (patrz R2)
G-KERNEL-01	Wyjście	8 Kernel modules passed validate stage; bundle assembler skleja 3-modułowy bundle
G-SEC-SKELETON-01	Wyjście	8 Security modules w dev-light stage; bootstrap_init aktywny

Metryki sukcesu M0

Metryka	Wartość progowa	Sposób pomiaru
contract_registry zawiera .proto	≥40 plików .proto	buf lint + buf breaking CI green
Breaking change bez D3 approval	0 naruszeń	buf breaking blokuje merge w CI
manifest_validator: akceptacja przykładów	≥20 przykładowych manifestów	CI green na golden set manifestów
Pierwszy assembled bundle	3 moduły kernel+kanon+council	Pełny RPC flow end-to-end green

ZASADA · M0 Kluczowy deliverable

„Pierwszy demo assembled bundle” — minimalny zestaw 3 modułów (kernel + kanon_access + council_workflow) z pełnym RPC flow. Ten bundle jest dowodem że architektura Lego na kontraktach działa w praktyce. Evidence pack: manifest_freeze_2026Q2.json. Profil security: dev-light wszędzie; bootstrap init aktywny.

R5.2 — M1: Governance + Memory core (sprint 3–4, miesiąc 2–3)

Cel: „Drabina D0–D5 end-to-end działa; Evidence pack produkcyjny.” M1 spina warstwę Governance i Memory core. Po M1 system potrafi klasyfikować decyzje, przeprowadzać głosowania Council, produkować podpisane Evidence pack’i dla D2+ oraz dostępować Księgę przez kanon_access. Bramy wejścia wymagają pełnego zaliczenia wszystkich bram exit M0.

Moduły dostarczane w M1 (delta względem M0)

Moduł ID	Klasa	Stan	Profil security	Uwaga
syliion.governance.decision_ladder	Governance	stable	dev-light → staging-strict	Early hardening — governance = safety-critical
syliion.governance.council_workflow	Governance	stable	staging-strict	Council 4/4 vote end-to-end
syliion.governance.roles	Governance	stable	staging-strict	RBAC role assignments
syliion.governance.gates_registry	Governance	stable	staging-strict	G-* gates registry
syliion.governance.policy_registry	Governance	stable	staging-strict	Policy rules engine
syliion.governance.self_explanation_validator	Governance	stable	staging-strict	D3+ commit validator
syliion.governance.evidence_workflow	Governance	stable	staging-strict	Evidence chain
syliion.memory.kanon_access	Memory	stable	dev-light	Księga Access Layer
syliion.memory.compact_layer	Memory	stable	dev-light	Compact Layer
syliion.memory.evidence_store	Memory	stable	dev-light	Evidence Store
syliion.memory.self_model_store	Memory	stable	dev-light	Self-Model Store

Razem po M1: 16 (M0) + 7 Governance + 4 Memory = 27 modułów stable.

Bramy wejścia i wyjścia M1

Brama	Typ	Kryterium zaliczenia
G-FREEZE-01	Wejście	Contract Freeze zaliczone (M0 exit)
G-KERNEL-01	Wejście	8 Kernel modules stable (M0 exit)
G-SEC-SKELETON-01	Wejście	Security skeleton dev-light active (M0 exit)
G-GOV-01	Wyjście	Decision Ladder D0–D5 pipeline live; >10 decyzji sklasyfikowanych end-to-end
G-COUNCIL-01	Wyjście	Pierwszy Council 4/4 vote przeprowadzony end-to-end; quorum + signature verified

Brama	Typ	Kryterium zaliczenia
G-EVIDENCE-01	Wyjście	100% decyzji D2+ z podpisanym Evidence pack; replay 100% successful dla 100 decyzji

Metryki sukcesu M1

Metryka	Wartość progowa	Sposób pomiaru
Council vote-to-decision latency	<48h p95	OTel histogram council.vote.duration_hours
Evidence replay success rate	100% dla 100 decyzji	evidence_spine.replay() green na golden set
D2+ decyzji z Evidence pack	100%	evidence_workflow coverage report
Governance modules w staging-strict	7/7	manifest security_profile check CI

UWAGA · M1 — Profil security: early hardening dla Governance

Governance jest warstwą safety-critical — jako jedyna klasa modułów przechodzi na profil staging-strict już w M1, przed pełną integracją systemu. Wszystkie pozostałe moduły pozostają w dev-light do M3–M5. Evidence pack: governance_live_2026Q2.json.

R5.3 — M2: Cognitive + Execution + Efficiency + Quality (sprint 5–8, miesiąc 3–5)

Cel: „Pełna warstwa rozumowania i wykonania; efficiency enforcement produkcyjny.” M2 dostarcza wszystkie moduły Cognitive i Execution, aktywuje 4 bramy Efficiency oraz buduje Quality golden set z 500+ przypadkami. Po M2 system potrafi planować zadania, uruchamiać narzędzia, routować modele LLM oraz enforcować budżety wydajności dla każdego bundle.

Moduły dostarczane w M2 (delta względem M1)

Moduł ID	Klasa	Stan	Profil security
syllion.cognitive.planner	Cognitive	stable	dev-light
syllion.cognitive.evaluator	Cognitive	stable	dev-light
syllion.cognitive.reasoner	Cognitive	stable	dev-light
syllion.cognitive.code_agent	Cognitive	stable	dev-light
syllion.cognitive.context_builder	Cognitive	stable	dev-light
syllion.cognitive.model_router	Cognitive	stable	dev-light
syllion.cognitive.llm_adapter	Cognitive	stable	dev-light
syllion.execution.tool_runner	Execution	stable	dev-light
syllion.execution.connector_framework	Execution	stable	dev-light
syllion.execution.workflow_engine	Execution	stable	dev-light
syllion.execution.job_runner	Execution	stable	dev-light
syllion.execution.adapter_bus	Execution	stable	dev-light
syllion.execution.retry_orchestrator	Execution	stable	dev-light
syllion.efficiency.code_bloat	Efficiency	stable	test-light (CI)
syllion.efficiency.runtime_perf	Efficiency	stable	test-light (CI)
syllion.efficiency.memory_footprint	Efficiency	stable	test-light (CI)
syllion.efficiency.cost_envelope	Efficiency	stable	test-light (CI)
syllion.quality.golden_set_registry	Quality	stable	test-light (CI)
syllion.quality.test_runner	Quality	stable	test-light (CI)
syllion.quality.regression_detector	Quality	stable	test-light (CI)

Razem po M2: 27 (M1) + 7 Cognitive + 6 Execution + 4 Efficiency + 3 Quality = 47 modułów stable.

Bramy wejścia i wyjścia M2

Brama	Typ	Kryterium zaliczenia
G-GOV-01, G-COUNCIL-01, G-EVIDENCE-01	Wejście	Wszystkie bramy M1 exit passed; Council live
G-COG-01	Wyjście	Planner+Reasoner+Evaluator pipeline live; 100 end-to-end task executions green
G-EFF-01	Wyjście	Code Bloat gate active; >50 bundle'ów gated; 0 regressions
G-EFF-02	Wyjście	Runtime Performance gate active; SLO budget enforced per bundle
G-EFF-03	Wyjście	Memory Footprint gate active; heap limit enforced per bundle
G-EFF-04	Wyjście	Cost Envelope gate active; token budget enforced per bundle
G-QUALITY-01	Wyjście	Golden set ≥500 przypadków; test runner parallel green; pass rate >98%

Metryki sukcesu M2

Metryka	Wartość progowa	Sposób pomiaru
Planner decompose latency	<3s p95	OTel histogram planner.decompose.duration_ms
Model routing overhead	<200ms p95	OTel histogram model_router.route.duration_ms
Golden set pass rate	>98%	quality.test_runner golden set CI report
Efficiency gates active	4/4	gates_registry G-EFF-01..04 status = active
Bundles gated przez efficiency	>50	gates_registry bundle_count per gate

EFFICIENCY · M2 — Efficiency gates jako warunek konieczny M3

Wszystkie 4 bramy Efficiency (G-EFF-01..04) muszą być aktywne przed przejściem do M3. Brama Efficiency nie jest opcjonalna — każdy bundle przechodzi przez pełny zestaw 4 budżetów. Profil security: dev-light lokalnie, test-light w CI. Evidence pack: cognitive_execution_live_2026Q3.json.

R5.4 — M3: Rebuildability + Surface (sprint 9–12, miesiąc 5–7)

Cel: „Pełna reversibility produkcyjna; Operator Console enterprise live.” M3 dostarcza pełną warstwę Rebuildability (łącznie z Legacy Preservation Window i shadow→dual→cutover state machine) oraz Operator Console z 8 panelami i 7 widokami RBAC. Po M3 system może bezpiecznie podmieniać moduły w środowisku staging przy pełnym audycie Evidence. Bramy wejścia wymagają aktywnych bram Efficiency z M2.

Moduły dostarczane w M3 (delta względem M2)

Moduł ID	Klasa	Stan	Profil security
syllion.rebuild.orchestrator	Rebuildability	stable	dev-light
syllion.rebuild.lpw_manager	Rebuildability	stable	dev-light
syllion.rebuild.cutover_controller	Rebuildability	stable	dev-light
syllion.rebuild.cft_runner	Rebuildability	stable	dev-light
syllion.surface.console_api	Surface	stable	staging-strict
syllion.surface.console_ui	Surface	stable	staging-strict
syllion.surface.ws_gateway	Surface	stable	staging-strict
syllion.memory.kb_adapter	Memory	stable	dev-light
syllion.memory.indexer	Memory	stable	dev-light
syllion.memory.retrieval	Memory	stable	dev-light

Razem po M3: 47 (M2) + 4 Rebuild + 3 Surface + 3 Memory remaining = 57 modułów stable.

Bramy wejścia i wyjścia M3

Brama	Typ	Kryterium zaliczenia
G-COG-01, G-EFF-01..04, G-QUALITY-01	Wejście	Wszystkie bramy M2 exit passed; efficiency gates active
G-REBUILD-01	Wyjście	Pierwszy successful greenfield rebuild pojedynczego modułu z CFT pass
G-LPW-01	Wyjście	Legacy Preservation Window enforced; 7 dni retention; LPW timer aktywny w staging
G-SURFACE-01	Wyjście	Console 8 paneli + 7 RBAC widoków live w staging; load time <2s p95
G-CUTOVER-01	Wyjście	Pierwszy shadow→dual→cutover flow przeprowadzony end-to-end z Evidence

Metryki sukcesu M3

Metryka	Wartość progowa	Sposób pomiaru
Rebuild-to-cutover per moduł	<2 tygodnie	rebuild.orchestrator lead_time_days histogram
Console load time	<2s p95	OTel histogram console_ui.page.load_ms
CFT pass rate	100% golden set	cft_runner golden set CI report
LPW retention enforced	7 dni	lpw_manager.retention_check daily CI
Shadow→dual→cutover flow	1+ przepływów	cutover_controller evidence log

REBUILD · M3 — Profil security: staging-strict dla Governance, dev-light dla reszty

M3 to moment pre-hardening. Governance pozostaje w staging-strict (od M1). Surface (Console) uruchamiana jest od razu w staging-strict ze względu na RBAC. Pozostałe moduły pozostają w dev-light do czasu Security Hardening w M5. Evidence pack: rebuildability_live_2026Q3.json.

R5.5 — M4: Skills + AEIS Self-* (sprint 13–16, miesiąc 7–9)

Cel: „AEIS Self-* pełne, Autonomy etapy 1–2 aktywne w prod.” M4 dostarcza komplet 65 modułów systemu — warstwę Skills/Demand oraz pełny zestaw AEIS Self-*. System wchodzi w etap Observe-only (Self-Observation 100% runtime) i Propose-only (Council triage aktywny). Autonomy etap 1 musi być stabilny przez 30 dni przed etapem 2. Bramy wejścia wymagają stabilnej Rebuildability z M3.

Moduły dostarczane w M4 (delta względem M3)

Moduł ID	Klasa	Stan	Profil security
syllion.skills.registry	Skills	stable	staging-strict
syllion.skills.executor	Skills	stable	staging-strict
syllion.aeis.demand_signal	Skills/Demand	stable	staging-strict
syllion.aeis.self_observation	AEIS Self-*	stable	staging-strict
syllion.aeis.improvement_queue	AEIS Self-*	stable	staging-strict
syllion.aeis.self_explanation	AEIS Self-*	stable	staging-strict
syllion.aeis.self_limitation	AEIS Self-*	stable	staging-strict
syllion.aeis.self_preservation	AEIS Self-*	stable	prod-strict

Razem po M4: 57 (M3) + 3 Skills/Demand + 5 AEIS Self-* = 65 modułów stable (komplet).

Bramy wejścia i wyjścia M4

Brama	Typ	Kryterium zaliczenia
G-REBUILD-01, G-LPW-01, G-SURFACE-01, G-CUTOVER-01	Wejście	Wszystkie bramy M3 exit passed; Rebuild live
G-SKILLS-01	Wyjście	Skills registry z ≥ 30 opublikowanymi skillami; demand signal live
G-AEIS-01	Wyjście	Self-Observation: 100% runtime invocations pokrytych; raporty hourly green
G-AEIS-02	Wyjście	Improvement Queue: ≥ 50 propozycji; Council triage >70% complete fields
G-AEIS-03	Wyjście	Self-Explanation templates dla 100% D3+ commitów (patrz Aneks R)
G-AEIS-04	Wyjście	Self-Limitation: 30 polityk enforced; 0 naruszeń in-the-wild
G-AEIS-05	Wyjście	Mission Preservation active (aktywna od M0, tu formalnie walidowana end-to-end)

Brama	Typ	Kryterium zaliczenia
G-AUTONOMY-1	Wyjście	Observe-only stage live w produkcji; 30 dni clean observation
G-AUTONOMY-2	Wyjście	Propose-only stage live; Council triage functional; Human approve D2+

Metryki sukcesu M4

Metryka	Wartość progowa	Sposób pomiaru
AEIS observation coverage	100% runtime invocations	self_observation.coverage_report daily
Improvement queue — propozycje	≥50 wygenerowanych	improvement_queue.count dashboard
Council triage complete fields	>70%	council_workflow.triage_completeness metric
Self-Explanation D3+ commits	100% pokrycia	self_explanation_validator CI gate
Self-Limitation violations	0 in-the-wild	self_limitation.violation_count = 0
Autonomy Observe-only czysty	30 dni	self_observation.anomaly_count = 0 rolling 30d

AEIS · M4 — staging-strict wszedzie, prod-strict dla Mission Preservation

M4 to moment, w którym SYLION obserwuje siebie w pełnym zakresie. Self-Preservation (G-AEIS-05) wymaga profilu prod-strict od pierwszego dnia — Mission Preservation jest zasadą bezpieczeństwa najwyższego rzędu. Autonomy etap 1 wymaga 30 dni clean observation przed aktywacją etapu 2 (propose-only). Evidence pack: aeis_live_2026Q4.json.

R5.6 — M5: Autonomy Full + Security Hardening + K8s (sprint 17–20, miesiąc 9–12)

Cel: „System produkcyjny z pełną autonomią, pełnym security, K8s-native.” M5 jest finalnym milestone’em Masterplanu. Składa się z trzech równoległych ścieżek: (1) Security Hardening — podmiana implementacji 8 modułów security przez lifecycle shadow→dual→cutover; (2) K8s migration z Docker Compose; (3) Autonomy etapy 3–5 — sandbox, limited prod, full governed. Finalny zewnętrzny pen-test (4 tygodnie) zamyka milestone.

Security Hardening — podmiana implementacji (kontrakty niezmiennie)

Moduł	Dev-light (M0–M4)	Prod-strict (M5)	Lifecycle
auth_provider	local tokens	Keycloak OIDC	shadow→dual→cutover
bootstrap_init	plaintext seed	secure seed + hardware token	shadow→dual→cutover
session_broker	in-memory sessions	Redis + JWT + refresh rotation	shadow→dual→cutover
policy_engine	allow-list basic	OPA + fine-grained policies	shadow→dual→cutover
execution_guard	subprocess isolation	gVisor/Firecracker + PHANTOM strict	shadow→dual→cutover
secret_provider	.env + local file	HashiCorp Vault	shadow→dual→cutover
audit_sink	append log	Ed25519 signed chain	shadow→dual→cutover
phantom_wrapper	skeleton light sandbox	full PHANTOM strict	shadow→dual→cutover

Kluczowa zasada: kontrakty .proto są identyczne w dev-light i prod-strict. Podmiana implementacji to wymiana wkładu w to samo gniazdo, NIE zmiana architektury (patrz Modularity by Contract, R0 §0.3).

Bramy wejścia i wyjścia M5

Brama	Typ	Kryterium zaliczenia
G-AEIS-01..05, G-AUTONOMY-1..2 (30 dni stable)	Wejście	Wszystkie bramy M4 exit + Autonomy stages 1–2 stable 30 dni
G-AUTONOMY-3	Wyjście	Sandbox self-change live w PHANTOM; ≥50 AEIS-driven sandbox changes z pełnym rollback
G-AUTONOMY-4	Wyjście	Limited production self-change live dla D0–D2; auto-rollback <2min p100
G-AUTONOMY-5	Wyjście	Full governed autonomy: AEIS proponuje D3+; Council 4/4 + Human dla D3+; External Review dla D5
G-SEC-FULL-01	Wyjście	prod-strict profile wszędzie; mTLS prod + Zero Trust live; cert rotation automated co 30 dni

<i>Brama</i>	<i>Typ</i>	<i>Kryterium zaliczenia</i>
G-PENTEST-01	Wyjście	0 critical findings po fixes; 0 high findings po fixes; ≤2 medium findings
G-K8S-01	Wyjście	K8s migration complete; wszystkie moduły w K8s; NetworkPolicies enforced; Istio service mesh

Metryki sukcesu M5

<i>Metryka</i>	<i>Wartość progowa</i>	<i>Sposób pomiaru</i>
Autonomy proposals/week	50–200 tygodniowo	improvement_queue.proposals_per_week dashboard
Auto-rollback success rate D0–D2	100%	rebuild.orchestrator.rollback_success_rate
mTLS cert rotation	co 30 dni automated	cert_manager.rotation_age_days ≤30 alert
Pen-test critical findings	0 po fixes	external pentest report + fix verification
Pen-test high findings	0 po fixes	external pentest report + fix verification
Pen-test medium findings	≤2 (wszystkie fixed przed cutover final)	pentest.medium_open = 0 w dniu cutover
K8s moduły zdeployowane	65/65	kubectl get pods —all-namespaces count
NetworkPolicies enforced	100% namespace	k8s_policy_coverage CI gate

UWAGA · M5 — Security Hardening nie zmienia architektury

Wszystkie 8 podmian security w M5 przebiega przez identyczny lifecycle: shadow (stary + nowy, response stary aktywny) → dual (ruch split, rising weight) → cutover (100% nowy, stary deprecated, LPW 7 dni). Kontrakty .proto pozostają niezmienione — to jest Modularity by Contract w działaniu. Evidence pack: autonomy_full_2026Q4.json.

R5.7 — Tabela podsumowania 6 milestone'ów

Poniższa tabela zbiera wszystkie sześć milestone'ów w jednym widoku. Kolumna „Modyły stable (+Δ)” pokazuje skumulowaną liczbę modułów w stanie stable oraz przyrost w danym milestone. Bramy exit są warunkami koniecznymi do otwarcia bram wejściowych następnego milestone'a.

Milestone	Spriny	Miesiące	Moduły stable (+Δ)	Bramy exit (kluczowe)	Profil security	Key deliverable
M0 Contract Freeze + Kernel	1–2	1	16 (+16)	G-FREEZE-01 G-KERNEL-01 G-SE C-SKELETON-01	dev-light wszędzie	Assembled bundle 3-modułowy (kernel+kanon+council) Evidence: manifest_freeze_2026Q2.json
M1 Governance + Memory core	3–4	2–3	27 (+11)	G-GOV-01 G-COUNCIL-01 G-EVIDENCE-01	dev-light + staging-strict (Governance early hardening)	D0–D5 pipeline live Council 4/4 end-to-end Evidence: governance_live_2026Q2.json
M2 Cognitive + Execution + Efficiency + Quality	5–8	3–5	47 (+20)	G-COG-01 G-EFF-01..04 G-QUALITY-01	dev-light lokalnie test-light (CI)	Planner+Reasoner live 4 efficiency gates active Golden set 500+ Evidence: cognitive_execution_live_2026Q3.json
M3 Rebuildability + Surface	9–12	5–7	57 (+10)	G-REBUILD-01 G-LPW-01 G-SURFACE-01 G-CUTOVER-01	staging-strict (Governance + Surface) dev-light (reszta)	Greenfield rebuild + CFT Operator Console live shadow→dual→cutover Evidence: rebuildability_live_2026Q3.json
M4 Skills + AEIS Self-*	13–16	7–9	65 (+8) — komplet	G-SKILLS-01 G-AEIS-01..05 G-AUTONOMY-1 G-AUTONOMY-2	staging-strict wszędzie prod-strict dla Mission Preservation	AEIS Self-* full Observe+Propose-only 30 dni clean obs. Evidence: aeis_live_2026Q4.json
M5 Autonomy Full + Security Hardening + K8s	17–20	9–12	65 (+0) sec. podmiana	G-AUTONOMY-3..5 G-SEC-FULL-01 G-PENTEST-01 G-K8S-01	prod-strict wszędzie mTLS prod Zero Trust	Full governed autonomy 8× sec. hardening via lifecycle K8s + Istio External pen-test Evidence: autonomy_full_2026Q4.json

ZASADA · Zasada ciągłości: każdy milestone = działający system

SYLION nie jest budowany „pod koniec”. Każdy milestone dostarcza działający system z zamkniętymi bramami, podpisanym Evidence pack i publicznym eventem `masterplan.milestone.<N>.completed` w Event Bus. Board review po każdym milestone weryfikuje zgodność z kanonem i Evidence. Brak zaliczonej bramy exit blokuje otwarcie następnego milestone’a.

R5.8 — Post-M5: Steady-State i Przyszłe Rozszerzenia

M5 nie jest końcem — to początek steady-state. Po M5 SYLION wchodzi w reżim operacyjny, w którym rytm pracy jest wyznaczany przez cykle operacyjne (patrz R9), a nie przez sprint backlog. System samodzielnie proponuje usprawnienia, Council zatwierdza D3+, a Human pozostaje w pętli dla D4–D5.

Cykle operacyjne po M5

- Weekly retro: analiza improvement queue, Council triage propozycji AEIS
- Monthly external review: przegląd Evidence packów przez zewnętrznego audytora
- Quarterly deep audit: pełna rewizja bram G-*, golden set, policy compliance
- Yearly canon evolution: przegląd meta-zasad (D5, Council 4/4 + Human + External Review)

Możliwe rozszerzenia systemu

- SYLION federation: wielowęzłowa sieć instancji SYLION z rozproszoną Księgą i cross-instance Council
- Multi-tenant: izolacja tenantów na poziomie namespace K8s z per-tenant security profile i per-tenant Evidence Spine
- SYLION-as-a-Service: eksponowanie zdolności AEIS przez API gateway dla zewnętrznych zespołów inżynierskich

AUTONOMY·CANON · M5 — Moment, w którym SYLION jest gotowy

„M5 to moment, w którym SYLION jest pełnym, autonomicznym systemem inżynierskim pod kanonem. Masterplan kończy pracę, zaczyna się operations.” System obsługuje pełny cykl: observe → propose → sandbox → limited prod → full governed — zawsze pod kontrolą drabiny D0–D5, Council i Księgi. To jest Autonomy under Canon w pełnym wydaniu.

ROZDZIAŁ R6 · AEIS IMPLEMENTATION MASTERPLAN

Execution Checklists P01–P20

Rozdział 6 — Execution Checklists per Plan

R6.0 — Wprowadzenie

Każdy z 20 planów SYLION v3.5 otrzymuje tu dedykowaną checklistę wykonawczą. Checklistą definiuje minimalny zakres pracy wymagany do uznania planu za „zamknięty” i przejścia do następnego milestone’u. Format jest identyczny dla każdego planu: moduły do zbudowania (zgodne z Module Registry), kontrakty .proto do napisania, tabele Postgres ze schematem, endpointy gRPC, wymagany golden set, oraz pary bram wejścia i wyjścia.

Checklista nie zastępuje pełnej specyfikacji wykonawczej poszczególnych planów (patrz v35_plan_NN.py), ale stanowi operacyjny punkt kontrolny dla zespołu wdrożeniowego. Jeśli każda z 20 checklist jest zielona, SYLION AEIS jest live.

Plan P01 — Pipeline Core & Architektura

Milestone target: M0. Strategia: Greenfield. Właściciel działu: Kernel Team.

P01 buduje nieruchome jądro systemu — osiem klocków, które tworzą kość pacierzową SYLION. Bez P01 żaden inny plan nie może się uruchomić, ponieważ Module Registry, Contract Registry, Event Bus, Environment Orchestrator i Bundle Assembler są współdzieloną infrastrukturą dla wszystkich 65 modułów. Contract Freeze Milestone jest osiągalny wyłącznie po zakończeniu P01.

Moduły

- `sy lion.core.module_registry` — jedyne źródło prawdy o modułach aktywnych w systemie
- `sy lion.core.manifest_loader` — czyta i waliduje `module.yaml` każdego modułu
- `sy lion.core.contract_registry` — wersjonowane kontrakty .proto + breaking change detection
- `sy lion.core.event_bus` — NATS JetStream wrapper z event catalog i idempotency
- `sy lion.core.environment_orchestrator` — Docker Compose / K8s abstraction; deploy/undeploy/swap
- `sy lion.core.bundle_assembler` — składa moduły w bundle gotowy do runu (shadow/dual/cutover)

Kontrakty .proto do napisania

1. `ModuleRegistry.proto v1.0.0` — `RegisterModule`, `ListModules`, `GetModuleManifest`, `DeregisterModule`

2. ContractRegistry.proto v1.0.0 — PublishContract, GetContract, CheckCompatibility, ListContractVersions
3. EventBus.proto v1.0.0 — PublishEvent, Subscribe, Ack, GetEventCatalog
4. EnvironmentOrchestrator.proto v1.0.0 — DeployModule, UndeployModule, SwapModule, GetEnvironmentStatus
5. BundleAssembler.proto v1.0.0 — AssembleBundle, GetBundleManifest, ValidateBundle, ReleaseBundle

Tabele Postgres (schemat sylion_core)

- modules — module_id, module_kind, owner_plan, status, created_at, updated_at
- module_versions — module_id, version, manifest_yaml, image_ref, lifecycle_state
- contracts — contract_id, service_name, proto_content, status, owner_module
- contract_versions — contract_id, semver, breaking_change, published_at
- events_published — event_id (UUIDv7), subject, payload_schema, owner_module
- environments — env_id, env_type (compose|k8s), modules_deployed (JSONB), created_at
- bundles — bundle_id, name, version, modules_included (JSONB), status, assembled_at

Kluczowe endpointy gRPC

- ModuleRegistryService.RegisterModule: rejestracja nowego modułu z manifestem
- ModuleRegistryService.ListModules: lista aktywnych modułów z filtrem po klasie i statusie
- ContractRegistryService.PublishContract: publikacja nowego .proto z wersją SemVer
- ContractRegistryService.CheckCompatibility: weryfikacja breaking change między wersjami
- EnvironmentOrchestratorService.DeployModule: wdrożenie modułu do środowiska
- BundleAssemblerService.AssembleBundle: złożenie bundle z listy modułów

Golden set minimum

100 rejestracji modułów pokrywających wszystkie 6 klas (Kernel, Cognitive, Execution, Memory, Governance, Security); 50 publikacji kontraktów z weryfikacją SemVer; 10 pełnych assembly bundle obejmujących bundle.core >= 1.0.0 i bundle.governance >= 1.0.0.

Bramy

- Entry: Contract Freeze — wszystkie 7 artefaktów zamrożone (manifest schema, contract registry, event taxonomy, module lifecycle, dependency rules, decision boundaries, security profile abstraction)
- Exit: G-KERNEL-01 — Module Registry live, Contract Registry live, Event Bus live; wszystkie 8 modułów Kernel w stanie validate

Plan P02 — Księga Operacyjna

Milestone target: M1. Strategia: Greenfield. Właściciel działu: Memory Team.

P02 implementuje Access Layer do Księgi Kanonicznej — jedyne źródła prawdy dla intencji, meta-zasad i Aneksów A–R. Moduły P02 zapewniają bezpieczny odczyt dla wszystkich ról D0+ oraz kontrolowany zapis wyłącznie dla Council (D3+). pgvector umożliwia wyszukiwanie semantyczne po treści Księgi.

Moduły

- `sylicion.memory.kanon_access` — bramka odczytu i zapisu do Księgi Kanonicznej
- `sylicion.cognitive.context_builder` — buduje kontekst z fragmentów Księgi dla agentów
- `sylicion.memory.indexer` — indeksowanie dokumentów Księgi wektorami pgvector
- `sylicion.memory.retrieval` — retrieval semantyczny i hybrydowy (BM25 + vector)

Kontrakty .proto do napisania

1. `KanonAccess.proto v1.0.0` — `ReadDocument` (D0+), `WriteDocument` (D3+ only), `SearchDocuments` (full-text + vector)
2. `ContextBuilder.proto v1.0.0` — `BuildContext`, `StreamContext`, `InvalidateContext`
3. `Retrieval.proto v1.0.0` — `VectorSearch`, `HybridSearch`, `GetDocumentChunks`

Tabele Postgres (schemat `sylicion_memory`)

- `kanon_documents` — `doc_id`, `title`, `content_text`, `aneks_ref`, `version`, `created_at`
- `kanon_versions` — `doc_id`, `version`, `diff_jsonb`, `author_role`, `signed_at`
- `kanon_embeddings` — `doc_id`, `chunk_index`, `embedding vector(1536)`, `model_id`
- `context_queries` — `query_id`, `agent_id`, `query_text`, `retrieved_doc_ids` (JSONB), `latency_ms`

Kluczowe endpointy gRPC

- `KanonAccessService.ReadDocument`: odczyt dokumentu po `doc_id` lub tytule
- `KanonAccessService.WriteDocument`: zapis/aktualizacja dokumentu (wymaga D3+)
- `KanonAccessService.SearchDocuments`: wyszukiwanie pełnotekstowe i wektorowe
- `ContextBuilderService.BuildContext`: budowanie kontekstu z wielu dokumentów dla agenta

Golden set minimum

200 zapytań odczytu pokrywających wszystkie Aneksy A–R; 50 bezpiecznych prób zapisu (z poprawną weryfikacją roli D3+); 100 wyszukiwań wektorowych z precyzją $\text{top-5} \geq 80\%$ vs. ground truth.

Bramy

- Entry: `evidence_spine live` (P06) — każda zmiana Księgi trafia do Evidence Spine
- Exit: `G-KANON-01` — Księga immutable od strony non-Council; `WriteDocument` odrzuca żądania bez D3+; `VectorSearch` zwraca wyniki w < 200 ms (p99)

Plan P03 — MZ 3.5.5 Baseline

Milestone target: M0→M1. Strategia: Greenfield. Właściciel działu: Cognitive/Memory Team.

P03 implementuje mechanizm przechowywania i wymiany baseline'u MZ 3.5.5 — aktualnego, wymiennalnego implementation baseline SYLION. Kluczową zasadą jest „Baseline swap mechanism”: podmiana aktywnego baseline'u przez Council (D4+) bez zmiany architektury. W każdej chwili istnieje dokładnie jeden wskaźnik active.

Moduły

- `syllion.memory.self_model_store` — przechowywanie stanu MZ, aktualnych baseline'ów i ich wersji

Kontrakty .proto do napisania

1. `BaselineStore.proto v1.0.0` — `GetBaseline` (D0+), `ProposeNewBaseline` (D2+), `SwapBaseline` (D4+, Council 4/4), `ListBaselineVersions`

Tabele Postgres (schemat `syllion_memory`)

- `mz_baselines` — `baseline_id`, `version_string`, `snapshot_jsonb`, `status`, `proposed_by`, `proposed_at`
- `mz_versions` — `baseline_id`, `semver`, `changelog_text`, `evidence_pack_id`, `created_at`
- `mz_active_pointer` — `id` (singleton = 1), `baseline_id`, `swapped_at`, `swapped_by_council_id`

Kluczowe endpointy gRPC

- `BaselineStoreService.GetBaseline`: pobranie aktualnego aktywnego baseline'u
- `BaselineStoreService.ProposeNewBaseline`: złożenie propozycji nowego baseline'u do Council
- `BaselineStoreService.SwapBaseline`: wymiana aktywnego wskaźnika (D4+, Council 4/4)
- `BaselineStoreService.ListBaselineVersions`: historia wersji z evidence pack refs

Golden set minimum

10 baseline swap'ów w środowisku sandbox z pełnym przejściem cyklu: propozycja → Council vote → swap → weryfikacja wskaźnika; 100% operacji swap odzwierciedlonych w Evidence Spine.

Bramy

- Entry: `syllion.core.module_registry` live; Evidence Spine dostępny
- Exit: G-MZ-01 — baseline swap mechanism live; `SwapBaseline` wymaga Council 4/4; dokładnie jeden aktywny wskaźnik w `mz_active_pointer` w każdym momencie

Plan P04 — Drabina D0–D5

Milestone target: M1. Strategia: Greenfield. Właściciel działu: Governance Team.

P04 implementuje silnik drabiny decyzyjnej D0–D5 — fundament governance SYLION. Każda decyzja w systemie musi być sklasyfikowana i przeprowadzona przez odpowiednią bramę (gate) zanim zostanie wykonana. `ClassifyDecision` i `EvaluateGate` są wywoływane przez wszystkie inne moduły, co czyni P04 zależnością krytycznej ścieżki.

Moduły

- `syllion.governance.decision_ladder` — implementacja klas D0–D5 z regułami klasyfikacji
- `syllion.core.decision_gate_engine` — ewaluator bram G-*; wywoływany przez wszystkie moduły

Kontrakty .proto do napisania

1. `DecisionLadder.proto v1.0.0` — `ClassifyDecision` (wejście: kontekst + typ zmiany; wyjście: klasa D0–D5), `EvaluateGate`, `RegisterGate`, `ListGates`, `GetDecisionHistory`

Tabele Postgres (schemat `syllion_governance`)

- `decisions` — `decision_id` (UUIDv7), `context_jsonb`, `proposed_by`, `created_at`
- `decision_classifications` — `decision_id`, `class_assigned` (D0–D5), `classifier_version`, `confidence`
- `gate_evaluations` — `eval_id`, `gate_id`, `decision_id`, `result` (`pass|fail|skip`), `evaluated_at`

Kluczowe endpointy gRPC

- `DecisionLadderService.ClassifyDecision`: klasyfikacja decyzji do klasy D0–D5
- `DecisionGateEngineService.EvaluateGate`: ewaluacja konkretnej bramy dla decyzji
- `DecisionGateEngineService.ListGates`: lista aktywnych bram z metadanymi

Golden set minimum

100 decyzji klasyfikowanych z dokładnością $\geq 95\%$ vs. ground truth; pokrycie wszystkich klas D0–D5 (min. 10 przypadków każda); weryfikacja, że D4/D5 zawsze wymaga Council 4/4.

Bramy

- Entry: `syllion.core.event_bus` live; `module_registry` live
- Exit: G-GOV-01 — drabina D0–D5 end-to-end operacyjna; `ClassifyDecision` osiąga $\geq 95\%$ accuracy; każde przejście przez bramę rejestrowane w Evidence Spine

Plan P05 — Council 4/4

Milestone target: M1. Strategia: Greenfield. Właściciel działu: Governance Team.

P05 implementuje mechanizm Council 4/4 — czteroosobowego gremium decyzyjnego wymaganego dla wszystkich decyzji klasy D3+. Council Workflow zarządza cyklem życia sesji decyzyjnych: od złożenia propozycji, przez głosowanie, weryfikację kworum, po finalizację decyzji z pełnym audytem. Ról Council dotyczy moduły Roles (P05+P15).

Moduły

- `syllion.governance.council_workflow` — cykl życia sesji Council: `propose`→`vote`→`quorum`→`finalize`
- `syllion.governance.roles` — definicje ról SYLION + przypisania + audyt zmian ról

Kontrakty .proto do napisania

1. CouncilWorkflow.proto v1.0.0 — ProposeDecision, CastVote, CheckQuorum, FinalizeDecision, GetSessionHistory
2. Roles.proto v1.0.0 — RegisterRole, AssignRole, RevokeRole, ListRoleAssignments, GetRolePermissions

Tabele Postgres (schemat sylion_governance)

- councils — council_id, council_type, members_jsonb, created_at
- council_members — member_id, council_id, role_id, assigned_at, revoked_at
- council_sessions — session_id, council_id, decision_id, status, deadline
- votes — vote_id, session_id, member_id, vote_value (approve|reject|abstain), cast_at
- quorum_records — session_id, quorum_reached, votes_count, evaluated_at

Kluczowe endpointy gRPC

- CouncilWorkflowService.ProposeDecision: złożenie propozycji decyzji D3+ do Council
- CouncilWorkflowService.CastVote: oddanie głosu przez członka Council
- CouncilWorkflowService.CheckQuorum: weryfikacja osiągnięcia kworum 4/4
- CouncilWorkflowService.FinalizeDecision: zamknięcie sesji i emisja decyzji

Golden set minimum

20 end-to-end przepływów Council (od propozycji do finalizacji); 10 przypadków odrzucenia przy braku kworum; 10 przypadków decyzji D4+ z weryfikacją Human Gate; wszystkie przepływy odzwierciedlone w Evidence Spine.

Bramy

- Entry: G-GOV-01 zdane (drabina D0–D5 live)
- Exit: G-COUNCIL-01 — Council 4/4 workflow live; kworum weryfikowane przed każdą finalizacją D3+; głosy trwałe i audytowalne

Plan P06 — Evidence Pack

Milestone target: M1. Strategia: Greenfield. Właściciel działu: Governance/Memory Team.

P06 buduje Evidence Spine — append-only log podpisany Ed25519, z pełną możliwością replay. Każda decyzja systemowa, każda zmiana modułu i każde przejście przez bramę muszą pozostawić dowód w Evidence Spine. Audytowalność jest niezbędnym warunkiem dla autonomii AEIS (P19).

Moduły

- sylion.governance.evidence_workflow — orkiestracja tworzenia i podpisywania paczek dowodów
- sylion.memory.evidence_store — trwałe przechowywanie paczek dowodów z indeksem
- sylion.core.evidence_spine — append-only log z sygnaturami Ed25519 i replay

Kontrakty .proto do napisania

1. EvidenceWorkflow.proto v1.0.0 — CreatePack, SubmitEvidence, GetPackStatus, ListPacksByDecision
2. EvidenceStore.proto v1.0.0 — StorePack, RetrievePack, ListPacks, DeletePack (D5 only)

Tabele Postgres (schemat sylion_governance)

- evidence_packs — pack_id (UUIDv7), decision_id, pack_type, status, created_at
- evidence_events — event_id (UUIDv7), pack_id, event_type, payload_jsonb, emitted_at
- signatures — sig_id, pack_id, algorithm (Ed25519), public_key_id, signature_hex, signed_at

Kluczowe endpointy gRPC

- EvidenceWorkflowService.CreatePack: tworzenie nowej paczki dowodów dla decyzji
- EvidenceWorkflowService.SignPack: podpisanie paczki Ed25519 przez audyt-sink
- EvidenceWorkflowService.ReplayPack: odtworzenie sekwencji zdarzeń z paczki
- EvidenceWorkflowService.VerifySignature: weryfikacja podpisu paczki

Golden set minimum

100 decyzji z kompletem evidence packs; 100% success rate przy replay wszystkich paczek; weryfikacja, że żadna paczka bez podpisu nie jest akceptowana przez Evidence Spine.

Bramy

- Entry: G-COUNCIL-01 zdane; sylion.security.audit_sink live
- Exit: G-EVIDENCE-01 — Evidence Spine live; każda decyzja D2+ ma evidence pack; replay 100% bez utraty zdarzeń; podpisy Ed25519 weryfikowalne

Plan P07 — Workflowy 7 Działów

Milestone target: M2. Strategia: Greenfield. Właściciel działu: Execution Team.

P07 implementuje warstwę wykonawczą SYLION — silnik workflow, runner narzędzi, framework konektorów i adapter bus. Siedem działów operacyjnych (DevOps, Analytics, Product, Security, Finance, Legal, Customer Success) otrzymuje dedykowane przepływy pracy. Retry Orchestrator zapewnia odporność na awarie.

Moduły

- sylion.execution.workflow_engine — definicja i wykonanie przepływów pracy (DAG-based)
- sylion.execution.tool_runner — uruchamianie narzędzi z izolacją i audytem
- sylion.execution.connector_framework — rejestracja i wywołanie zewnętrznych konektorów
- sylion.execution.job_runner — zarządzanie zadaniami batchowymi i cron
- sylion.execution.adapter_bus — magistrala adapterów dla integracji zewnętrznych
- sylion.execution.retry_orchestrator — at-least-once retry z exponential backoff i dead-letter

Kontrakty .proto do napisania

1. WorkflowEngine.proto v1.0.0 — DefineWorkflow, StartWorkflow, GetWorkflowStatus, CompleteStep, AbortWorkflow
2. ToolRunner.proto v1.0.0 — RegisterTool, InvokeTool, GetToolResult, ListTools
3. ConnectorFramework.proto v1.0.0 — RegisterConnector, CallConnector, GetConnectorHealth, ListConnectors

Tabele Postgres (schemat sylion_execution)

- workflows — workflow_id, name, definition_jsonb, owner_dept, version, created_at
- workflow_instances — instance_id, workflow_id, status, input_jsonb, output_jsonb, started_at
- tool_invocations — invocation_id, tool_id, instance_id, input_jsonb, result_jsonb, duration_ms
- connector_registrations — connector_id, name, base_url, auth_type, health_status

Kluczowe endpointy gRPC

- WorkflowEngineService.StartWorkflow: uruchomienie instancji przepływu dla działu
- WorkflowEngineService.CompleteStep: zakończenie kroku i przejście do następnego
- ToolRunnerService.InvokeTool: wywołanie narzędzia z izolacją
- ConnectorFrameworkService.RegisterConnector: rejestracja nowego konektora zewnętrznego

Golden set minimum

100 instancji workflow (min. 10 na dział); end-to-end testy dla 7 przepływów działowych; weryfikacja retry przy symulowanym błędzie konektora.

Bramy

- Entry: G-EVIDENCE-01 zdane; sylion.core.event_bus live
- Exit: G-EXEC-01 — silnik workflow live; wszystkie 7 przepływów działowych end-to-end zdane; ToolRunner emituje zdarzenia do Evidence Spine

Plan P08 — Multi-Agent Coordination

Milestone target: M2. Strategia: Greenfield. Właściciel działu: Cognitive Team.

P08 buduje warstwę kognitywną SYLION: planer rozkładający zadania na kroki, ewaluator oceniający jakość planów, reasoner prowadzący łańcuchy myślenia, code agent generujący kod, oraz Model Router z LiteLLM zapewniający wymienialność modeli językowych zgodnie z meta-zasadą Rebuildability.

Moduły

- sylion.cognitive.planner — dekompozycja zadań na kroki i podplany
- sylion.cognitive.evaluator — ocena jakości planów i wyników
- sylion.cognitive.reasoner — Chain-of-Thought i Tree-of-Thought reasoning

- `syllion.cognitive.code_agent` — generacja, weryfikacja i deployment kodu
- `syllion.cognitive.model_router` — routing requestów do LLM przez LiteLLM
- `syllion.cognitive.llm_adapter` — adapter łączący Model Router z LiteLLM API

Kontrakty .proto do napisania

1. `Planner.proto v1.0.0` — `DecomposePlan`, `GetPlanStatus`, `RefinePlan`, `AbortPlan`
2. `Evaluator.proto v1.0.0` — `EvaluatePlan`, `EvaluateOutput`, `GetEvaluationHistory`
3. `Reasoner.proto v1.0.0` — `Reason`, `StreamReasoning`, `GetReasoningTrace`
4. `ModelRouter.proto v1.0.0` — `RouteModel`, `ListAvailableModels`, `GetModelCost`, `SetModelPreference`

Tabele Postgres (schemat `syllion_cognitive`)

- `plans` — `plan_id`, `task_description`, `status`, `parent_plan_id`, `created_at`
- `plan_steps` — `step_id`, `plan_id`, `step_index`, `action_type`, `input_jsonb`, `output_jsonb`, `status`
- `reasoning_traces` — `trace_id`, `plan_id`, `model_id`, `prompt_tokens`, `completion_tokens`, `trace_text`
- `model_invocations` — `invocation_id`, `model_id`, `request_jsonb`, `response_jsonb`, `latency_ms`
- `llm_costs` — `cost_id`, `invocation_id`, `input_tokens`, `output_tokens`, `cost_usd`, `recorded_at`

Kluczowe endpointy gRPC

- `PlannerService.DecomposePlan`: rozkład zadania na sekwencję kroków
- `EvaluatorService.EvaluatePlan`: ocena planu wg kryteriów jakości
- `ReasonerService.Reason`: wywołanie łańcucha myślenia dla problemu
- `ModelRouterService.RouteModel`: wybór optymalnego modelu dla requestu

Golden set minimum

100 end-to-end egzekucji zadań (od `DecomposePlan` do finalnego output); pokrycie 5 typów zadań (`code`, `analysis`, `planning`, `summarization`, `reasoning`); weryfikacja fallback'u Model Routera przy niedostępnym modelu.

Bramy

- Entry: G-EXEC-01 zdane; `syllion.core.event_bus` live; fake LLM adapter dostępny
- Exit: G-COG-01 — Planner + Evaluator + Reasoner + Model Router w stanie stable; 100% zadań emituje ślady do Evidence Spine; koszty LLM rejestrowane

Plan P09 — Security Skeleton + Red/Blue Team

Milestone target: M0 (skeleton), M5 (full hardening). Strategia: Greenfield. Właściciel działu: Security Team.

P09 wdraża wszystkie 8 modułów security od dnia pierwszego w profilu `dev-light`. Kluczowa obserwacja: ten sam kontrakt `.proto` obowiązuje we wszystkich profilach — podmiana implementacji (`dev-light` → `prod-strict`) nie zmienia architektury. M5 kończy hardening: Keycloak OIDC, PHANTOM + gVisor, HashiCorp Vault, Ed25519 audit chain.

Moduły

- `syllion.security.auth_provider` — M0: local token bootstrap; M5: Keycloak OIDC
- `syllion.security.bootstrap_init` — M0: plaintext admin init; M5: secure seed + hardware token
- `syllion.security.session_broker` — M0: in-memory sessions; M5: Redis + JWT + refresh token rotation
- `syllion.security.policy_engine` — M0: allow-list basic; M5: OPA + fine-grained policies
- `syllion.security.execution_guard` — M0: subprocess isolation; M5: PHANTOM + gVisor/Firecracker
- `syllion.security.secret_provider` — M0: `.env` + local file; M5: HashiCorp Vault
- `syllion.security.audit_sink` — M0/M6+P06: append log; M5: Ed25519 signed chain
- `syllion.security.phantom_wrapper` — M0: skeleton (light sandbox); M5: full PHANTOM strict

Kontrakty *.proto* do napisania

1. `AuthProvider.proto v1.0.0` — `Authenticate`, `ValidateToken`, `RevokeToken`, `RefreshToken`
2. `BootstrapInit.proto v1.0.0` — `InitializeSystem`, `GetBootstrapStatus`, `ResetBootstrap` (D5)
3. `SessionBroker.proto v1.0.0` — `CreateSession`, `GetSession`, `InvalidateSession`, `ListActiveSessions`
4. `PolicyEngine.proto v1.0.0` — `EnforcePolicy`, `AddPolicy`, `RemovePolicy` (Council), `ListPolicies`
5. `ExecutionGuard.proto v1.0.0` — `GuardExecution`, `GetGuardStatus`, `SetGuardProfile`
6. `SecretProvider.proto v1.0.0` — `GetSecret`, `SetSecret` (D3+), `RotateSecret`, `ListSecretIds`
7. `AuditSink.proto v1.0.0` — `AppendAudit`, `QueryAuditLog`, `VerifyAuditChain`
8. `PhantomWrapper.proto v1.0.0` — `RunInPhantom`, `GetPhantomResult`, `AbortPhantom`

Tabele Postgres (*schemat syllion_security*)

- `auth_tokens` — `token_id`, `subject`, `scope`, `issued_at`, `expires_at`, `revoked`
- `sessions` — `session_id`, `subject`, `created_at`, `last_active`, `expires_at`
- `policies` — `policy_id`, `policy_type`, `rule_jsonb`, `added_by`, `active`
- `secrets` — `secret_id`, `label`, `encrypted_value`, `rotation_due`, `last_rotated_at`
- `audit_log` — `audit_id` (UUIDv7), `event_type`, `subject`, `payload_jsonb`, `sig_hex`, `logged_at`
- `phantom_runs` — `run_id`, `task_id`, `isolation_level`, `status`, `result_jsonb`, `started_at`

Kluczowe endpointy gRPC

- `AuthService.Authenticate`: uwierzytelnienie i wydanie tokenu
- `SessionBrokerService.CreateSession`: tworzenie sesji z TTL
- `PolicyEngineService.EnforcePolicy`: weryfikacja żądania wg aktywnych polityk
- `ExecutionGuardService.GuardExecution`: uruchomienie kodu w izolowanym środowisku
- `SecretProviderService.GetSecret`: pobranie sekretu z aktywnego providera
- `AuditSinkService.AppendAudit`: zapis zdarzenia auditowego z sygnaturą
- `PhantomWrapperService.RunInPhantom`: izolowane wykonanie w PHANTOM

Golden set minimum

M0: 100 bootstrap flows pokrywających zimny start + reinicjalizację; 1000 walidacji tokenów z 0% false positive. M5: 100 przeptywów OIDC (Keycloak); 1000 izolowanych egzekucji PHANTOM; wynik pen-testu: 0 podatności krytycznych.

Bramy

- Entry: Contract Freeze (7 artefaktów zamrożonych)
- Exit M0: G-SEC-SKELETON-01 — wszystkie 8 modułów security w profilu dev-light w stanie validate; bootstrap flow operacyjny
- Exit M5: G-SEC-FULL-01 + G-PENTEST-01 — wszystkie 8 modułów w profilu prod-strict; pen-test bez krytycznych podatności

Plan P10 — Cztery Wymiary Efficiency

Milestone target: M2. Strategia: Greenfield. Właściciel działu: Efficiency Team.

P10 implementuje meta-zasadę „Efficiency by Default” (v3.3) poprzez cztery dedykowane moduły pomiarowe: Code Bloat, Runtime Performance, Memory Footprint i Cost Envelope. Każdy bundle musi zmieścić się w budżecie zanim trafi na środowisko produkcyjne. Bramy G-EFF-01..04 blokują deployment poza budżetem.

Moduły

- `sylicion.efficiency.code_bloat` — analiza i pomiar rozrostu kodu; dependency graph; dead code
- `sylicion.efficiency.runtime_perf` — pomiar latencji i throughput per RPC; SLO tracking
- `sylicion.efficiency.memory_footprint` — śledzenie heap, GC pressure, RSS per moduł
- `sylicion.efficiency.cost_envelope` — monitoring budżetu tokenów i kosztów API (USD)

Kontrakty .proto do napisania

1. `CodeBloatMeter.proto v1.0.0` — `MeasureBloat`, `GetBloatReport`, `SetBloatBudget`, `AlertOnBloat`
2. `RuntimePerfMeter.proto v1.0.0` — `MeasureRuntime`, `GetPerfReport`, `SetSLO`, `CheckSLOBreach`
3. `MemoryFootprintMeter.proto v1.0.0` — `MeasureMemory`, `GetMemoryReport`, `SetMemoryBudget`
4. `CostEnvelopeMeter.proto v1.0.0` — `MeasureCost`, `GetCostReport`, `SetCostBudget`, `CheckBudget`

Tabele Postgres (schemat `sylicion_efficiency`)

- `code_metrics` — `metric_id`, `bundle_id`, `module_id`, `loc`, `dependencies_count`, `dead_code_pct`, `measured_at`
- `runtime_metrics` — `metric_id`, `rpc_name`, `p50_ms`, `p95_ms`, `p99_ms`, `throughput_rps`, `measured_at`
- `memory_metrics` — `metric_id`, `module_id`, `heap_mb`, `rss_mb`, `gc_pause_ms`, `measured_at`

- `cost_metrics` — `metric_id`, `module_id`, `input_tokens`, `output_tokens`, `api_cost_usd`, `measured_at`
- `efficiency_budgets` — `budget_id`, `dimension` (`code|runtime|memory|cost`), `threshold`, `module_id`, `active`

Kluczowe endpointy gRPC

- `CodeBloatMeterService.MeasureBloat`: pomiar wskaźników rozrostu kodu dla bundle
- `RuntimePerfMeterService.MeasureRuntime`: pomiar latencji RPC w środowisku testowym
- `MemoryFootprintMeterService.MeasureMemory`: pomiar zużycia pamięci modułu
- `CostEnvelopeMeterService.MeasureCost`: pomiar kosztu tokenów i API dla przebiegu
- `CostEnvelopeMeterService.CheckBudget`: weryfikacja, czy moduł mieści się w budżecie

Golden set minimum

500 bundle'ów zmierzonych we wszystkich 4 wymiarach; 100% enforcement budżetów — żaden bundle poza budżetem nie przechodzi do deployment; pokrycie każdej klasy modułów (A–K).

Bramy

- Entry: G-COG-01 zdane; `sylon.core.evidence_spine` live
- Exit: G-EFF-01 (code bloat) + G-EFF-02 (runtime) + G-EFF-03 (memory) + G-EFF-04 (cost) — wszystkie 4 bramy aktywne; deployment pipeline blokowany przy przekroczeniu budżetu

Plan P11 — Testowanie Kanoniczne + Golden Set

Milestone target: M2. Strategia: Greenfield. Właściciel działu: Quality Team.

P11 implementuje infrastrukturę testów kanonicznych — złoty zestaw przypadków testowych, który jest źródłem prawdy dla regresji. Golden Set Registry przechowuje i wersjonuje zestawy; Test Runner egzekwuje je w CI; Regression Detector triggeruje auto-rollback, gdy wyniki spadną poniżej progu.

Moduły

- `sylon.quality.golden_set_registry` — zarządzanie zbiorami test cases: wersjonowanie, diff, lifecycle
- `sylon.quality.test_runner` — wykonanie test cases z integracją CI/CD i raportowaniem
- `sylon.quality.regression_detector` — detekcja regresji + trigger auto-rollback

Kontrakty .proto do napisania

1. `GoldenSet.proto v1.0.0` — `RegisterGoldenSet`, `GetGoldenSet`, `ListGoldenSets`, `UpdateGoldenSet` (D2+)
2. `TestRunner.proto v1.0.0` — `RunTests`, `GetTestRun`, `StreamTestResults`, `AbortTestRun`
3. `RegressionDetector.proto v1.0.0` — `AnalyzeRun`, `GetRegressionAlert`, `AcknowledgeAlert`

Tabele Postgres (schemat sylion_quality)

- golden_sets — set_id, name, module_target, version, test_count, created_at
- test_cases — case_id, set_id, input_jsonb, expected_output_jsonb, category, created_at
- test_runs — run_id, set_id, started_at, completed_at, pass_count, fail_count, status
- regression_alerts — alert_id, run_id, severity, description, acknowledged_at

Kluczowe endpointy gRPC

- GoldenSetService.RegisterGoldenSet: rejestracja nowego zestawu testów dla modułu
- TestRunnerService.RunTests: uruchomienie zestawu testów i zebranie wyników
- RegressionDetectorService.AnalyzeRun: analiza wyników przebiegu i detekcja regresji

Golden set minimum (meta)

500 przypadków testowych obejmujących wszystkie moduły aktywne w M2; pokrycie klas: Kernel, Cognitive, Execution, Memory, Governance, Security; weryfikacja auto-rollback przy symulowanej regresji.

Bramy

- Entry: G-EFF-01..04 zdane; wszystkie moduły M2 w stanie validate
- Exit: G-QUALITY-01 — Golden Set Registry z ≥ 500 przypadkami; Test Runner zintegrowany z CI/CD; Regression Detector aktywny i przetestowany

Plan P12 — Bramy G-*

Milestone target: M2. Strategia: Greenfield. Właściciel działu: Governance Team.

P12 tworzy pełny rejestr bram G-* i rozszerza Decision Gate Engine (P04) o możliwość rejestracji i ewaluacji dowolnych bram. Bramy są blokerami pipeline — żaden deployment, żadna zmiana architektury i żadne przejście między milestone'ami nie może nastąpić bez zdania właściwych bram.

Moduły

- sylion.governance.gates_registry — rejestr bram G-*: definicje, wersje, historia ewaluacji
- sylion.core.decision_gate_engine — rozszerzenie (P04 + P12): ewaluacja wszystkich bram

Kontrakty .proto do napisania

1. GatesRegistry.proto v1.0.0 — RegisterGate, EvaluateAllGates, GetGateStatus, GetGateHistory, OverrideGate (Council 4/4)

Tabele Postgres (schemat sylion_governance)

- gates — gate_id, gate_name, description, owner_plan, criteria_jsonb, active
- gate_evaluations — eval_id, gate_id, context_jsonb, result (pass|fail|skip), evaluated_at
- gate_history — history_id, gate_id, eval_id, status_before, status_after, changed_at
- gate_overrides — override_id, gate_id, reason, council_session_id, overridden_at

Kluczowe endpointy gRPC

- `GatesRegistryService.RegisterGate`: rejestracja nowej bramy z kryteriami
- `GatesRegistryService.EvaluateGate`: ewaluacja konkretnej bramy
- `GatesRegistryService.OverrideGate`: nadpisanie wyniku bramy (Council 4/4)

Golden set minimum

30 bram zarejestrowanych (pokrywając G-KERNEL, G-GOV, G-SEC, G-EFF, G-PERF, G-MEM, G-COST); 1000 ewaluacji bram z 100% trwałością wyników w `gate_evaluations`; 10 override'ów z weryfikacją Council 4/4.

Bramy

- Entry: G-QUALITY-01 zdane; P04 Decision Gate Engine live
- Exit: G-EFF-01..04 + G-PERF-01 + G-MEM-01 + G-COST-01 live i aktywne w rejestrze; `EvaluateAllGates` pokrywa wszystkie zarejestrowane bramy

Plan P13 — Rejestr Bram + Audit

Milestone target: M2. Strategia: Greenfield (rozszerzenie P12). Właściciel działu: Governance Team.

P13 rozszerza rejestr bram (P12) o pełny audit trail każdej ewaluacji. Każde wywołanie `EvaluateGate` generuje zdarzenie w Evidence Spine. Tabela `gate_audit_log` przechowuje pełny JSONB dla każdej ewaluacji, zapewniając możliwość rekonstrukcji historii decyzji bramowych.

Moduły

- `sylicon.governance.gates_registry` — rozszerzenie (P12 + P13): audit trail per ewaluacja
- `sylicon.core.evidence_spine` — integracja zdarzeń bramowych w Evidence Spine

Kontrakty .proto do napisania

1. Osad w `GatesRegistry.proto` (P12) + `EvidenceWorkflow.proto` (P06) — dodanie `gate_event_class` do event taxonomy

Tabele Postgres (schemat `sylicon_governance`)

- `gate_audit_log` — `audit_id` (UUIDv7), `gate_id`, `eval_id`, `full_context_jsonb`, `result`, `evidence_pack_id`, `audited_at`

Kluczowe endpointy gRPC

- `GatesRegistryService.GetGateAuditLog`: pobranie pełnego logu auditowego bramy
- `EvidenceWorkflowService.CreatePack` (gate event class): tworzenie paczki dowodów dla ewaluacji bramy

Golden set minimum

1000 ewaluacji bram z 100% pokryciem w `gate_audit_log`; weryfikacja, że każde zdarzenie bramowe ma odpowiadający evidence pack; test replay 100 losowych ewaluacji z

odtworzeniem pełnego kontekstu.

Bramy

- Entry: G-EFF-01..04 zdane; G-EVIDENCE-01 zdane
- Exit: G-GATE-AUDIT-01 — 100% ewaluacji bram audytowanych; evidence packs dla zdarzeń bramowych tworzą ciągły łańcuch w Evidence Spine

Plan P14 — Greenfield Rebuild Orchestration

Milestone target: M3. Strategia: Greenfield. Właściciel działu: Rebuildability Team.

P14 implementuje meta-zasadę „Rebuildability over Lineage” (v3.0): mechanizm orkiestracji pełnego rebuildu modułu lub zestawu modułów. Rebuild Orchestrator zarządza cyklem shadow → dual → cutover; LPW Manager chroni legacy przez okno 7 dni; CFT Runner weryfikuje fidelity po każdym rebuildzie.

Moduły

- sylion.rebuild.orchestrator — orkiestracja pełnego rebuildu: shadow→dual→cutover
- sylion.rebuild.lpw_manager — zarządzanie Legacy Preservation Window (7 dni)
- sylion.rebuild.cutover_controller — state machine przejść shadow/dual/cutover
- sylion.rebuild.cft_runner — Compact Fidelity Test — weryfikacja po rebuildzie

Kontrakty .proto do napisania

1. RebuildOrchestrator.proto v1.0.0 — StartRebuild, GetRebuildStatus, AbortRebuild, ListActiveRebuilds
2. LPWManager.proto v1.0.0 — StartLPW, ExtendLPW (Council D4), CloseLPW, GetLPWStatus
3. CutoverController.proto v1.0.0 — AdvanceCutover, RollbackCutover, GetCutoverState
4. CFTRunner.proto v1.0.0 — RunCFT, GetCFTResult, ListCFTHistory

Tabele Postgres (schemat sylion_rebuild)

- rebuild_jobs — job_id, module_id, strategy, status, started_at, completed_at
- lpw_records — lpw_id, job_id, legacy_ref, window_days, starts_at, closes_at, status
- cutover_states — state_id, job_id, current_phase (shadow|dual|cutover), traffic_weight, updated_at
- cft_runs — run_id, job_id, module_id, started_at, completed_at, status
- cft_results — result_id, run_id, test_category, pass_count, fail_count, fidelity_score

Kluczowe endpointy gRPC

- RebuildOrchestratorService.StartRebuild: inicjalizacja procesu rebuildu modułu
- LPWManagerService.StartLPW: uruchomienie okna Legacy Preservation Window
- CutoverControllerService.AdvanceCutover: przejście do następnej fazy (shadow→dual lub dual→cutover)
- CFTRunnerService.RunCFT: uruchomienie Compact Fidelity Test po rebuildzie

Golden set minimum

10 end-to-end rebuildów (shadow → dual → cutover → stable) z CFT pass $\geq 99\%$ fidelity; 5 testów rollback przy symulowanej regresji w fazie dual; weryfikacja LPW — legacy dostępne przez pełne 7 dni.

Bramy

- Entry: G-GATE-AUDIT-01 zdane; G-QUALITY-01 zdane
- Exit: G-REBUILD-01 — Rebuild Orchestrator live; CFT Runner zintegrowany z każdym rebuild job; LPW Manager aktywny

Plan P15 — Multi-Team + 4 Role Efficiency

Milestone target: M2. Strategia: Greenfield (rozszerzenie P05). Właściciel działu: Governance Team.

P15 rozszerza moduł Roles (P05) o cztery role efektywności — dedykowane role dla zespołów wdrożeniowych z precyzyjnie określonymi uprawnieniami w zakresie każdego wymiaru efficiency (Code, Runtime, Memory, Cost). Workflow Engine (P07) otrzymuje adapter do przypisania zadań per rola.

Moduły

- sylion.governance.roles — rozszerzenie (P05 + P15): 4 role efficiency + team adapter
- sylion.execution.workflow_engine — rozszerzenie (P07 + P15): task routing per rola

Kontrakty .proto do napisania

1. Roles.proto v1.0.0 (rozszerzenie) — RegisterRole, AssignRole, RevokeRole, ListRoleRecords, GetRoleEfficiencyActions

Tabele Postgres (schemat sylion_governance)

- roles — role_id, role_name, role_type (standard|efficiency), permissions_jsonb, created_at
- role_assignments — assignment_id, role_id, assignee_id, assigned_at, revoked_at
- role_efficiency_actions — action_id, role_id, dimension (code|runtime|memory|cost), action_type, performed_at

Kluczowe endpointy gRPC

- RolesService.AssignRole: przypisanie roli efficiency do membra teamu
- RolesService.ListRoleActions: lista akcji podjętych w ramach roli z audytem

Golden set minimum

4 role efficiency $\times$ 10 akcji = 40 akcji audytowanych; weryfikacja, że każda rola ma dostęp wyłącznie do swojego wymiaru efficiency; test przypisania i odwołania roli z pełnym audit trail.

Bramy

- Entry: G-EXEC-01 zdane; G-COG-01 zdane; P05 Roles module live

- Exit: G-ROLES-01 — 4 role efficiency aktywne; wszystkie akcje rolowe audytowane w role_efficiency_actions; task routing per rola w Workflow Engine

Plan P16 — Compaction & Memory

Milestone target: M3. Strategia: Greenfield. Właściciel działu: Memory Team.

P16 implementuje warstwę kompaktacji kontekstu — mechanizm redukowania rozmiaru kontekstu przy zachowaniu fidelity (meta-zasada Reversibility & Fidelity, v3.1). Compact Layer kompresuje konteksty; CFT Runner (współdzielony z P14) weryfikuje zgodność po dekompaktacji. Kluczowa miara: fidelity score ≥ 0.99 .

Moduły

- sylion.memory.compact_layer — kompaktacja i dekompaktacja kontekstów z weryfikacją fidelity
- sylion.rebuild.cft_runner — współdzielony z P14; CFT weryfikuje fidelity kompaktacji

Kontrakty .proto do napisania

1. CompactLayer.proto v1.0.0 — CompactContext, DecompackContext, VerifyFidelity, GetCompactionStatus, ListCompactions

Tabele Postgres (schemat sylion_memory)

- compactions — compaction_id, source_context_id, compressed_size_bytes, original_size_bytes, fidelity_score, created_at
- compaction_versions — compaction_id, version, algorithm, model_id, created_at
- cft_verifications — verification_id, compaction_id, cft_run_id, passed, score, verified_at

Kluczowe endpointy gRPC

- CompactLayerService.CompactContext: kompaktacja kontekstu z logowaniem parametrów
- CompactLayerService.DecompackContext: dekompaktacja do oryginału (w miarę możliwości)
- CompactLayerService.VerifyCFT: weryfikacja fidelity przez Compact Fidelity Test

Golden set minimum

100 kompaktacji z CFT 100% pass (fidelity score ≥ 0.99); pokrycie 3 algorytmów kompaktacji; weryfikacja, że DecompackContext nie traci krytycznych artefaktów semantycznych.

Bramy

- Entry: G-REBUILD-01 zdane; CFT Runner live (P14)
- Exit: G-COMPACT-01 (kompaktacja $\geq 99\%$ fidelity) + G-CFT-01 (CFT Runner live i zdany dla wszystkich typów kontekstów)

Plan P17 — Operator Console Enterprise

Milestone target: M3. Strategia: Greenfield. Właściciel działu: Surface Team.

P17 buduje Operator Console — enterprise interface dla operatorów SYLION. Next.js 14 + tRPC client + shadcn/ui. Console API jest gRPC-web/tRPC gateway; WS Gateway zapewnia push real-time dla alertów, decyzji i statusów modułów. Console jest budowane równolegle z kernelem, używa stubowanych endpointów.

Moduły

- `sylion.surface.console_api` — gRPC-web / tRPC gateway dla Next.js UI
- `sylion.surface.console_ui` — Next.js 14 + shadcn/ui; panel operatora
- `sylion.surface.ws_gateway` — WebSocket push dla real-time eventów

Kontrakty .proto do napisania

1. `ConsoleAPI.proto v1.0.0` — `GetDashboard`, `ListDecisions`, `GetModuleStatus`, `ListActiveGates`, `CastVote` (proxy), `GetAuditLog`
2. `WSGateway.proto v1.0.0` — `Subscribe`, `Unsubscribe`, `GetSubscriptions`, `PushEvent` (server-side)

Tabele Postgres (schemat `sylion_surface`)

- `console_sessions` — `session_id`, `operator_id`, `started_at`, `last_active`, `expires_at`
- `console_actions` — `action_id`, `operator_id`, `action_type`, `payload_jsonb`, `performed_at`
- `ws_subscriptions` — `sub_id`, `session_id`, `topic`, `created_at`, `active`

Kluczowe endpointy gRPC

- `ConsoleAPIService.GetDashboard`: pobranie zagregowanego stanu systemu dla operatora
- `ConsoleAPIService.ListDecisions`: lista decyzji oczekujących na Council z filtrowaniem
- `ConsoleAPIService.CastVote`: proxy dla `CouncilWorkflow.CastVote` z sesją operatora
- `WSGatewayService.Subscribe`: subskrypcja strumienia zdarzeń real-time

Golden set minimum

1000 akcji UI pokrywających wszystkie widoki panelu operatora; 100 równoczesnych klientów WebSocket bez degradacji; end-to-end test: operator widzi decyzję Council i głośnie z poziomu Console.

Bramy

- Entry: G-REBUILD-01 zdane; G-COUNCIL-01 zdane
- Exit: G-SURFACE-01 — Console API live; WS Gateway live; Console UI dostępne pod `/console`; 100 concurrent WS klientów obsługiwanych

Plan P18 — Skills & Knowledge

Milestone target: M4. Strategia: Greenfield. Właściciel działu: Skills Team.

P18 implementuje rejestr umiejętności SYLION — mechanizm publikowania, wersjonowania i wykonywania skills. Skills są jednostkami zdolności agenta: mogą być opublikowane przez dowolny dział (D2+), wersjonowane w Skills Registry i wykonywane przez Skills Executor z pełnym audytem. KB Adapter łączy Skills z Księgą Kanoniczną.

Moduły

- `syllion.skills.registry` — katalog skills: lifecycle DRAFT→PUBLISHED→DEPRECATED
- `syllion.skills.executor` — runtime wykonania skill z izolacją i audytem
- `syllion.memory.kb_adapter` — adapter łączący Skills z Księgą Kanoniczną

Kontrakty .proto do napisania

1. `SkillsRegistry.proto v1.0.0` — `PublishSkill`, `GetSkill`, `ListSkills`, `DeprecateSkill` (D2+), `GetSkillHistory`
2. `SkillExecutor.proto v1.0.0` — `ExecuteSkill`, `GetExecutionResult`, `AbortExecution`, `ListExecutions`

Tabele Postgres (schemat `syllion_skills`)

- `skills` — `skill_id`, `name`, `description`, `owner_dept`, `status` (`draft|published|deprecated`), `created_at`
- `skill_versions` — `version_id`, `skill_id`, `semver`, `implementation_ref`, `published_at`
- `skill_executions` — `execution_id`, `skill_id`, `version_id`, `input_jsonb`, `output_jsonb`, `duration_ms`
- `skill_metrics` — `metric_id`, `skill_id`, `call_count`, `success_rate`, `avg_latency_ms`, `measured_at`

Kluczowe endpointy gRPC

- `SkillsRegistryService.PublishSkill`: publikacja nowej skill lub wersji
- `SkillsRegistryService.DeprecateSkill`: oznaczenie skill jako deprecated (D2+)
- `SkillExecutorService.ExecuteSkill`: wykonanie skill z audytem i pomiarem

Golden set minimum

30+ skills opublikowanych (min. 5 per dział); 100 egzekucji pokrywających różne typy skills; weryfikacja lifecycle DRAFT→PUBLISHED→DEPRECATED per każda skill.

Bramy

- Entry: G-COG-01 zdane; Kanon live (P02)
- Exit: G-SKILLS-01 — Skills Registry z ≥ 30 skills; Executor live; KB Adapter zintegrowany z Księgą; wszystkie egzekucje audytowane

Plan P19 — AEIS Self-Evolution

Milestone target: M4. Strategia: Greenfield. Właściciel działu: AEIS Team.

P19 to serce SYLION AEIS — mechanizm samoewolucji pod nadzorem Kanonu. Pięć modułów AEIS implementuje Self-Observation (pomiar bez wpływu), Improvement Queue (proponycje do

Council), Self-Explanation (uzasadnienia zmian), Self-Limitation (egzekwowanie ograniczeń) i Self-Preservation (ochrona misji systemu). Zasada: autonomia wyłącznie w granicach Kanonu.

Moduły

- `syllion.aeis.self_observation` — pomiar 100% runtime; raporty hourly; ZERO modyfikacji (D0)
- `syllion.aeis.improvement_queue` — kolejka propozycji usprawnień; Council triażuje; Human zatwierdza D2+
- `syllion.aeis.self_explanation` — generowanie uzasadnień zmian zgodnych z Kanonem
- `syllion.aeis.self_limitation` — egzekwowanie polityk ograniczających autonomię
- `syllion.aeis.self_preservation` — detekcja zagrożeń misji; eskalacja do Council

Kontrakty *.proto do napisania*

1. `SelfObservation.proto v1.0.0` — `RecordObservation`, `GetObservationReport`, `StreamObservations`
2. `ImprovementQueue.proto v1.0.0` — `SubmitImprovement`, `TriagelImprovement (Council)`, `GetQueueStatus`, `ListImprovements`
3. `SelfExplanation.proto v1.0.0` — `RecordSelfExplanation`, `GetExplanation`, `ValidateExplanation`
4. `SelfLimitation.proto v1.0.0` — `AddLimitationPolicy`, `EvaluateSelfLimitation`, `GetActivePolicies`
5. `SelfPreservation.proto v1.0.0` — `CheckMissionThreat`, `EscalateThreat`, `GetPreservationStatus`

Tabele Postgres (*schemat syllion_aeis*)

- `observations` — `obs_id` (UUIDv7), `module_id`, `metric_type`, `value_jsonb`, `anomaly_flag`, `observed_at`
- `improvements` — `improvement_id`, `description`, `proposed_by`, `decision_class`, `status`, `council_session_id`
- `self_explanations` — `explanation_id`, `improvement_id`, `explanation_text`, `kanon_ref`, `validated`
- `self_limitations` — `limitation_id`, `policy_type`, `rule_jsonb`, `active`, `added_at`
- `preservation_events` — `event_id`, `threat_type`, `severity`, `escalated_to_council`, `resolved_at`

Kluczowe endpointy gRPC

- `SelfObservationService.RecordObservation`: zapis obserwacji metryki modułu
- `ImprovementQueueService.SubmitImprovement`: złożenie propozycji usprawnienia
- `ImprovementQueueService.TriagelImprovement`: triażowanie propozycji przez Council
- `SelfExplanationService.RecordSelfExplanation`: zapis uzasadnienia zmiany
- `SelfLimitationService.EvaluateSelfLimitation`: weryfikacja działania wg polityk
- `SelfPreservationService.CheckMissionThreat`: weryfikacja zagrożenia misji systemu

Golden set minimum

1000 obserwacji pokrywających wszystkie klasy modułów (A–K); 100 propozycji usprawnień z pełnym cyklem triażu Council; 50 self-explanacji z walidacją zgodności z Kanonem; 30 polityk ograniczeń aktywnych i przetestowanych; 0 naruszeń misji — każdy CheckMissionThreat bez fałszywych negatywów.

Bramy

- Entry: G-SKILLS-01 zdane; G-COUNCIL-01 zdane; Evidence Spine live
- Exit: G-AEIS-01 (self_observation live) + G-AEIS-02 (improvement_queue live) + G-AEIS-03 (self_explanation validated) + G-AEIS-04 (self_limitation policies active) + G-AEIS-05 (self_preservation 0 mission violations)

Plan P20 — Skill Demand Engine

Milestone target: M4. Strategia: Greenfield. Właściciel działu: AEIS/Skills Team.

P20 zamyka pętlę samoewolucji: Demand Signal zbiera sygnały z 4 źródeł (workflow gaps, failed tool invocations, operator requests, AEIS observations), klastruje je, a następnie generuje propozycje nowych skills do Skills Registry (P18). Plan P20 jest ostatnim ogniwem Autonomy Rollout Stage 2 (Propose-only).

Moduły

- sylion.aeis.demand_signal — zbieranie, klasteryzacja i propozycja skills na podstawie sygnałów popytu
- sylion.skills.registry — integracja z P18: przyjmowanie propozycji z Demand Engine

Kontrakty .proto do napisania

1. DemandSignal.proto v1.0.0 — RecordSignal, ClusterSignals, ProposeSkill, GetClusterReport, ListSignals

Tabele Postgres (schemat sylion_aeis)

- demand_signals — signal_id (UUIDv7), source_type (workflow|tool|operator|aeis), description, weight, created_at
- demand_clusters — cluster_id, cluster_label, signal_ids (JSONB), centroid_embedding, created_at
- skill_proposals — proposal_id, cluster_id, skill_name, description, confidence, submitted_to_council, created_at

Kluczowe endpointy gRPC

- DemandSignalService.RecordSignal: rejestracja nowego sygnału popytu ze źródłem
- DemandSignalService.ClusterSignals: uruchomienie klasteryzacji zebranych sygnałów
- DemandSignalService.ProposeSkill: generacja propozycji skill z klastra i przesłanie do Council

Golden set minimum

4 źródła sygnałów × 100 sygnałów = 400 zarejestrowanych sygnałów; 10 klastrów wygenerowanych z precyzją $\geq 80\%$ vs. oczekiwane grupowania; 5 propozycji skill złożonych do Council — każda z evidence pack.

Bramy

- Entry: G-AEIS-01..05 zdane; G-SKILLS-01 zdane
 - Exit: G-AEIS-SKILL-01 — Demand Engine live; propozycje skill generowane automatycznie i przesyłane do Council; pętla Observation→Signal→Cluster→Propose end-to-end zamknięta
-

R6.99 — Podsumowanie

Rozdział R6 zawiera 20 checklist wykonawczych pokrywających pełny zakres implementacji SYLION AEIS v3.5. Łączne liczby:

- Łącznie 20 checklist (P01–P20)
- 65 modułów deployowalnych (klasy A–K)
- ~80 kontraktów .proto (wersja 1.0.0 jako punkt startowy)
- ~200 tabel Postgres w schematach `sylion_core`, `sylion_cognitive`, `sylion_execution`, `sylion_memory`, `sylion_governance`, `sylion_security`, `sylion_efficiency`, `sylion_quality`, `sylion_rebuild`, `sylion_aeis`, `sylion_skills`, `sylion_surface`
- ~400 endpointów gRPC (RPC) eksponowanych przez moduły

ZASADA · Warunek życia SYLION AEIS

„Jeśli każda z 20 checklist jest zielona, SYLION AEIS jest live.” — Każda checklista ma dwie bramy (entry + exit). Żadna brama exit nie jest spełniona bez złotego zestawu testów. Żadna brama exit niższego planu nie jest zamknięta bez spełnienia bram wyższego planu. Bramy tworzą topologiczne zagrożenie — system jest tak silny, jak jego najstabsze zielone — i tak wiadomy, jak jego najbardziej szczegółowa checklista.

ROZDZIAŁ R7 · AEIS IMPLEMENTATION MASTERPLAN

Autonomy Rollout Strategy

Rozdział 7 — Autonomy Rollout Strategy

§R7.0 — Wprowadzenie

Autonomy Rollout definiuje, w jaki sposób AEIS — Autonomous Engineering Intelligence System — stopniowo uzyskuje prawo do modyfikowania własnych procesów w ściśle kontrolowanych klasach. Nie jest to jednorazowe włączenie „autonomii”, lecz pięcioetapowy marsz weryfikowalnych dowodów rozciągnięty od milestone'u M4 do M5 — łącznie około czterech miesięcy.

Każdy etap to osobny switch w module `sylicion.aeis.improvement_queue`, zarządzany jako feature flag przez Radę (Council). Włączenie etapu N może nastąpić wyłącznie po spełnieniu kryteriów gating exit etapu N-1 oraz odpowiedniej fazy Security Hardening. Rollback dowolnego etapu = downgrade switcha o 1 poziom; jest inicjowany automatycznie przy naruszeniu Self-Limitation lub Mission Preservation.

Pięć etapów Autonomy Rollout

1. Observe-only (M4 start, sprint 13) — AEIS obserwuje, zero modyfikacji.
2. Propose-only (M4 mid, sprint 14) — AEIS generuje propozycje; każda wymaga Council triage + Human approval.
3. Sandbox self-change (M5 start, sprint 17) — AEIS modyfikuje kopie sandbox (.sandbox); nigdy nie dotyka produkcji.
4. Limited production self-change (M5 mid, sprint 18–19) — AEIS modyfikuje klasy D0–D2 w prod z auto-rollback.
5. Full governed autonomy (M5 end, sprint 20) — AEIS inicjuje propozycje D3+; drabina D0–D2 autonomiczna; Council 4/4 + Human dla D3+; External Review dla D5.

AUTONOMY·CANON · Zasada nadrzędna Autonomy Rollout

„Co nie może być cofnięte, nie może być włączone.” Każdy etap jest osobnym switchem i posiada ściśle zdefiniowany rollback plan. Mission Preservation jest ostatnią, nieprzekraczalną granicą — jej naruszenie wywołuje natychmiastowy downgrade do etapu 1 (observe-only) i emergency session Rady.

§R7.1 — Etap 1: Observe-only

Kiedy i co

Etap 1 startuje na początku M4 (sprint 13). AEIS mierzy 100% runtime invocations przez moduł `sylicion.aeis.self_observation` z instrumentacją OTel auto-instrumentation. System generuje

raporty hourly, publikuje anomaly alerts — lecz nie wykonuje żadnych modyfikacji. Jedynym aktywowanym switchem jest self_observation.

Celem etapu 1 jest zbudowanie pełnego obrazu baseline zachowania systemu przed jakimkolwiek aktywnym działaniem AEIS. Po 30 dniach czystych obserwacji możliwe jest miarodajne porównanie „przed” i „po” w każdym kolejnym etapie.

Moduły aktywowane

- sylion.aeis.self_observation — observe every call via OTel auto-instrumentation
- OTel collector stack (Tempo + Loki + Prometheus) — powiązanie z observability (patrz R5 M4)
- Observation storage schema — wdrożone w PostgreSQL (JSONB + pgvector)
- Report generator — hourly Loki query + summary push do AEIS Steward

Kryteria gating — Entry / Exit (G-AUTONOMY-1)

Brama	Typ	Kryterium	Weryfikacja
G-AUTONOMY-1-ENTRY	Entry	OTel coverage 100% wszystkich runtime invocations	OTel dashboard: 0 uninstrumented services
G-AUTONOMY-1-ENTRY	Entry	Observation storage schema deployed i tested w staging	migration applied, smoke test green
G-AUTONOMY-1-ENTRY	Entry	Report generator tested w staging (min 24h)	3 raporty hourly dostarczone bez błędów
G-AUTONOMY-1-EXIT	Exit (po 30 dniach)	30 dni clean observation z <2% false-positive anomaly alerts	alert_ratio <= 0.02 z Evidence Spine
G-AUTONOMY-1-EXIT	Exit	0 data loss w observation storage	checksum audit z audit_sink
G-AUTONOMY-1-EXIT	Exit	Weekly observability report accepted przez AEIS Steward	4 raporty zaakceptowane (podpis w Evidence)

Rollback plan

Rollback etapu 1: wyłączenie OTel collectorów + zatrzymanie observation service. Impact: żaden — poza utratą observability (nic nie jest modyfikowane, więc nie ma czego cofać). Czas rollback: <5 minut. Profil security: staging-strict (obserwacja dotyka produkcji, ale wyłącznie do odczytu).

AEIS · AEIS — Etap 1 jako fundament

„Etap Observe-only jest jedynym etapem, gdzie AEIS nie ma żadnej mocy sprawczej. To jest celowe: zanim system dowie się czegokolwiek o sobie, musi przez 30 dni ‘milczeć’ i słuchać. Dane z etapu 1 stają się baseline dla wszystkich późniejszych porównań.”

§R7.2 — Etap 2: Propose-only

Kiedy i co

Etap 2 startuje w połowie M4 (sprint 14), bezpośrednio po osiągnięciu G-AUTONOMY-1-EXIT. AEIS uzyskuje prawo do generowania propozycji zmian do kolejki `syllion.aeis.improvement_queue`. Każda propozycja klasy D0+ wymaga Council triage + Human approval — AEIS nie może nic wykonać samodzielnie.

Moduły aktywowane

- `syllion.aeis.improvement_queue` — switch `proposal_submission=true`
- `syllion.aeis.self_explanation` (Aneks R) — validator wymaga kompletu pól w każdej propozycji
- Council triage workflow — rozszerzony o AEIS-generated proposals (nowe źródło: `aeis_agent`)

Workflow Propose-only

AEIS obserwuje wzorce (etap 1 nadal działa) → Pattern Detector wykrywa anomalię lub możliwość optymalizacji → Self-Explanation Validator sprawdza kompletność propozycji wg Aneksu R → propozycja trafia do `improvement_queue` z polem `proposer=aeis_agent` → Council triages w <48h (p95) → Human approval → wykonanie przez standardowy governance (poza AEIS).

Kryteria gating — Entry / Exit (G-AUTONOMY-2)

Brama	Typ	Kryterium	Weryfikacja
G-AUTONOMY-2-ENTRY	Entry	<code>improvement_queue</code> schema + API live (gRPC <code>ProposalService v1</code>)	contract conformance test green
G-AUTONOMY-2-ENTRY	Entry	Aneks R template validator implemented + tested	100 propozycji-sample prześci przez validator
G-AUTONOMY-2-ENTRY	Entry	Council triage workflow rozszerzony dla AEIS-generated proposals	dry-run 10 proposals w staging
G-AUTONOMY-2-EXIT	Exit (po 45 dniach)	50+ propozycji AEIS opublikowanych w queue	<code>improvement_queue</code> COUNT WHERE <code>proposer=aeis_agent</code> >= 50
G-AUTONOMY-2-EXIT	Exit	Council triage time <48h p95	<code>triage_latency_p95</code> z Evidence Spine
G-AUTONOMY-2-EXIT	Exit	>70% propozycji z kompletnymi polami (nie odrzucone z powodu braku danych)	<code>completion_rate</code> >= 0.70
G-AUTONOMY-2-EXIT	Exit	0 Self-Limitation violations w proposal payloads	<code>self_limitation_violations</code> = 0 w <code>audit_sink</code>

Rollback plan

Rollback: wyłączenie przełącznika proposal_submission w improvement_queue (wrót do trybu human-only submissions). Istniejące propozycje w toku kończą triage bez przerywania. Czas rollback: <2 min (config change). Profil security: staging-strict w M4; prod-strict wymagany w cutover M5.

§R7.3 — Etap 3: Sandbox self-change

Kiedy i co

Etap 3 startuje na początku M5 (sprint 17). AEIS uzyskuje prawo do modyfikowania sandbox kopii modułów (suffix .sandbox) w środowisku PHANTOM. Produkcja jest całkowicie izolowana — żadne zasoby sandbox nie mogą dotrzeć do prod.

Moduły aktywowane

- sylion.security.phantom_wrapper full strict — pełny sandbox PHANTOM
- sylion.rebuild.orchestrator — sandbox deploys w namespace .sandbox
- sylion.security.execution_guard strict — gVisor/Firecracker isolation
- Golden Set + CFT + Perf Budget — automated w pipeline sandbox

Workflow Sandbox self-change

AEIS generuje propozycję zmiany modułu → environment_orchestrator tworzy kopię module_name.sandbox → AEIS wdraża zmianę w PHANTOM → automatyczna walidacja: golden set + CFT + perf budget → jeśli pass: propozycja trafia do improvement_queue dla Council review → jeśli fail: auto-discard w <2 min + audit log. Sandbox records pozostają w PHANTOM zawsze (audit trail).

Kryteria gating — Entry / Exit (G-AUTONOMY-3)

Brama	Typ	Kryterium	Weryfikacja
G-AUTONOMY-3-ENTRY	Entry	PHANTOM full strict live w prod	phantom_mode=strict z audit_sink
G-AUTONOMY-3-ENTRY	Entry	environment_orchestrator wspiera namespace .sandbox	sandbox deploy smoke test: 5 modułów
G-AUTONOMY-3-ENTRY	Entry	Golden set + CFT + perf budget automated w sandbox pipeline	CI/CD sandbox pipeline green 14 dni
G-AUTONOMY-3-EXIT	Exit (po 30 dniach)	50+ AEIS-driven sandbox changes tested	sandbox_changes_count >= 50 w audit
G-AUTONOMY-3-EXIT	Exit	0 sandbox leaks (sandbox resources nie dotarły do prod)	NetworkPolicy audit + phantom isolation check
G-AUTONOMY-3-EXIT	Exit	>80% sandbox changes pass golden set	golden_pass_rate >= 0.80

Brama	Typ	Kryterium	Weryfikacja
G-AUTONOMY-3-E XIT	Exit	<2 min auto-discard dla failed changes	discard_latency_p99 < 120s

Rollback plan

Rollback: wyłączenie sandbox deploys w improvement_queue (wrót do human-only proposals). PHANTOM zachowuje sandbox records dla audit. Czas rollback: <2 min. Profil security: prod-strict (PHANTOM full strict wymagany).

§R7.4 — Etap 4: Limited production self-change

Kiedy i co

Etap 4 startuje w połowie M5 (sprint 18–19) i trwa do osiągnięcia G-AUTONOMY-4-EXIT (min. 60 dni). AEIS uzyskuje prawo do modyfikowania modułów produkcyjnych w klasach D0–D2 z włączonym auto-rollback. Klasy D3+ nadal wymagają Council 4/4 + Human Gate bez zmian z etapu 2.

Warunek wejścia: Security Hardening

Etap 4 może startować wyłącznie po tym, jak Security Hardening fazy 1–2 jest kompletne: auth_provider (Keycloak OIDC prod), session_broker (Redis + JWT + refresh rotation), audit_sink (Ed25519 signed chain) — wszystkie w trybie prod-strict. Bez pełnego utwardzenia hardening prod changes wymagają roll-forward forensics niemających podstaw. Patrz §R7.6 (interakcja z Security Hardening).

Moduły aktywowane

- sylion.rebuild.orchestrator — prod deploys (shadow → dual → cutover) dla D0–D2
- sylion.rebuild.cutover_controller — automated shadow→dual (10%→90%) → cutover dla D0–D2
- sylion.quality.regression_detector — auto-rollback trigger (<30s detection, <2 min rollback)
- mTLS prod + NetworkPolicies — enforced przez K8s migration (M5)

Workflow D0–2 (automated) vs D3+ (Council)

Klasa	Workflow	Monitoring	Rollback trigger
D0	AEIS proposal → sandbox → shadow deploy → cutover → 30 min monitoring → stable	auto-only	regression_detector < 30s → rollback < 2 min
D1	jak D0	auto-only	jak D0
D2	AEIS proposal → sandbox → shadow → dual (10%→90%) → cutover → 30 min monitoring	auto + SRE alert	jak D0; dual weight cofany przy regression

Klasa	Workflow	Monitoring	Rollback trigger
D3+	bez zmian z etapu 2: Council 4/4 + Human Gate	SRE + Council	manual Council decision

Kryteria gating — Entry / Exit (G-AUTONOMY-4)

Brama	Typ	Kryterium	Weryfikacja
G-AUTONOMY-4-ENTRY	Entry	regression_detector live w prod z <30s detection time	drill: inject synthetic regression → alert <30s
G-AUTONOMY-4-ENTRY	Entry	auto-rollback <2 min tested w staging (50+ drills)	drill_results: mean_rollback < 120s, 0 failed drills
G-AUTONOMY-4-ENTRY	Entry	cutover_controller wspiera automated D0–D2 flow	staging smoke test: 5 D0–D2 automated cutover
G-AUTONOMY-4-ENTRY	Entry	Security Hardening fazy 1–2 complete (auth+session+audit prod-strict)	audit_sink signed: hardening_complete event w Evidence
G-AUTONOMY-4-EXIT	Exit (po 60 dniach)	100+ AEIS-driven D0–D2 production changes	prod_changes_count >= 100 w audit
G-AUTONOMY-4-EXIT	Exit	0 unrolled regressions w production	unrolled_regression_count = 0
G-AUTONOMY-4-EXIT	Exit	Auto-rollback triggered N razy (baseline), 100% successful	rollback_success_rate = 1.0
G-AUTONOMY-4-EXIT	Exit	Council quorum bottleneck alleviated (D0–D2 nie czeka na Council)	council_queue_d0d2_wait_time ≈ 0

Rollback planu etapu 4

Rollback etapu: wyłączenie prod auto-cutover dla AEIS proposals — wszystkie klasy wracają do manual Council review. Aktywne cutovery D0–D2 w toku są zamrażane na dual (10%) do decyzji SRE. Czas rollback: <5 min (config change + operator confirm). Profil security: prod-strict + mTLS prod + NetworkPolicies enforced.

UWAGA · Interakcja z Security Hardening (etap 4)

Etap 4 jest twardo uzależniony od Security Hardening fazy 1–2. Prod changes bez signed audit_sink, bez Keycloak OIDC w prod i bez mTLS są nieakceptowalne z perspektywy rollback forensics. „Jeśli nie możemy odtworzyć każdego kroku, nie możemy odtworzyć nic.”

§R7.5 — Etap 5: Full governed autonomy

Kiedy i co

Etap 5 startuje na końcu M5 (sprint 20) po 60 stabilnych dniach etapu 4. Kluczowa zmiana względem etapu 4: AEIS może inicjować propozycje klas D3+ bez ludzkiego pre-screeningu (wynik nadal wymaga Council). Drabina D0–D2 jest w pełni autonomiczna. D5 wymaga External Review poza organizacją.

Moduły aktywowane

- sylion.aeis.self_preservation — final check przed każdą D3+ proposal: czy zmiana nie zagraża misji
- Wszystkie moduły z etapów 1–4 w trybie ciągłym
- External Review process — quarterly, poza organizacją

Drabina decyzji w Etapie 5

Klasa	Kto decyduje	Czas oczekiwania	Dodatkowe wymaganie
D0	Automatic execute	0 (natychmiast)	Audit only
D1	Automatic execute	0 (natychmiast)	Audit only
D2	Automatic execute	24h stabilization observation	Auto-rollback jeśli regression w 24h
D3	Council 4/4 required	SLA: 72h p95	AEIS może być proposer
D4	Council 4/4 + Human Gate	SLA: 96h p95	Pellentesque Board approval
D5	Council 4/4 + Human Gate + External Review	Quarterly cycle	Poza organizacją

Kryteria gating — Entry / Exit (G-AUTONOMY-5)

Brama	Typ	Kryterium	Weryfikacja
G-AUTONOMY-5-ENTRY	Entry	Etap 4 stable 60 dni (G-AUTONOMY-4-EXIT passed)	Evidence Spine: stage4_stable event
G-AUTONOMY-5-ENTRY	Entry	Self-Preservation Mission Threat detection >99% precision	test set: 1000 synthetic threats, precision >= 0.99
G-AUTONOMY-5-ENTRY	Entry	0 violations wykryte w-the-wild w ciągu etapu 4	self_limitation_violations = 0 w audit
G-AUTONOMY-5-ENTRY	Entry	Security pełne (M5 faza 3+) + pen-test clean	pen_test_report: 0 critical findings
G-AUTONOMY-5-EXIT	Exit (on going)	Autonomy proposals/week: 50–200	proposal_rate w desired band
G-AUTONOMY-5-EXIT	Exit	Council quorum throughput: >95% propozycji rozpatrzonych w SLA	council_sla_rate >= 0.95

Brama	Typ	Kryterium	Weryfikacja
G-AUTONOMY-5-E XIT	Exit	0 mission violations	mission_preservation_violations = 0
G-AUTONOMY-5-E XIT	Exit	External review cycle: quarterly	review_completed event w Evidence co kwartał

Rollback etapu 5

Rollback: downgrade do etapu 4 (D3+ human pre-screening reaktywowany). AEIS zachowuje prawa do D0–D2 automated. Czas rollback: <5 min. Profil security: prod-strict + PHANTOM strict + pen-test quarterly.

AUTONOMY·CANON · Full governed autonomy — pełna dojrzałość AEIS

„Po etapie 5, SYLION jest samomodyfikującym się systemem pod kanonem. D0–D2 działają bez interwencji człowieka, ale każda zmiana jest audytowalna, odwracalna i ograniczona Mission Preservation. Człowiek pozostaje w pętli dla D3+, a External Review gwarantuje zewnętrzną kontrolę co kwartał.”

§R7.6 — Interakcja etapów z Security Hardening

Każdy etap Autonomy Rollout wymaga określonego minimum Security Hardening. Poniższa tabela definiuje zależności — etap Autonomy nie może startować bez spełnienia odpowiedniej fazy hardening. Patrz R8 — Security Hardening Path + Integration Assembly po szczegóły każdej fazy.

Etap Autonomy	Wymagana faza Security Hardening	Uzasadnienie
Etap 1 — Observe-only	Audit basic (M0): audit_sink append-log w staging	Obserwacja wymaga zapisu — minimum audit; brak modyfikacji, więc brak potrzeby full hardening
Etap 2 — Propose-only	Audit extended (M1): structured evidence, aeis-proposal events audytowalne	Propozycje muszą być audytowalne z pełnym kontekstem — audit_mode=extended wymagany
Etap 3 — Sandbox self-change	PHANTOM skeleton (M0) → PHANTOM light (M3) → PHANTOM full strict przed etapem 3	Sandbox wymaga izolacji: bez PHANTOM strict sandbox leak jest możliwy
Etap 4 — Limited prod self-change	PHANTOM strict + mTLS prod + audit_sink signed Ed25519 (M5 fazy 1–2)	Prod changes wymagają full hardening dla rollback forensics i integrity guarantee
Etap 5 — Full governed autonomy	Security pełne (M5 fazy 3+) + pen-test clean (0 critical findings)	Najwyższy rygor: D3+ proposals od AEIS, External Review; każdy wyciek = mission threat

Zależność jest jednostronna: Security Hardening nigdy nie czeka na Autonomy Rollout. Jeśli Security Hardening się opóźni, odpowiednie etapy Autonomy są zamrożone (switch pozostaje

wyłączony) do momentu potwierdzenia fazy hardening w Evidence Spine.

UWAGA · Zasada sekwencji: Security > Autonomy

„Bez podłoża nie ma budynku. Security Hardening jest podłożem Autonomy Rollout. Odwrócenie kolejności — autonomia przed utwardzeniem — jest fundamentalnym błędem architektonicznym, którego masterplan nie akceptuje w żadnej formie.”

§R7.7 — Macierz downgrade (switch backward)

Każdy trigger downgrade jest zdefiniowany z precyzyjnym efektem i procedurą notyfikacji. Downgrade jest automatyczny i natychmiastowy — operator nie jest pytany o zgodę przed wykonaniem. Notyfikacja następuje równolegle z downgrade.

Trigger	Automatyczny downgrade	Notyfikacja	Czas
Self-Limitation violation wykryta (dowolna klasa)	Etap 4 → 3 (sandbox only) lub wyższy etap → 3	AEIS Steward + Board — alert P1	< 2 min
Mission Preservation threat wykryta	Etap N → 1 (observe-only natychmiast!)	Critical alert ALL — Council emergency session w ciągu 1h	< 30s
Regression w prod (auto-rollback > X razy w 24h, X zdefiniowane w SLA)	Etap 4 → 2 (propose-only, human approval dla wszystkich D0–D2)	Quality team + SRE — alert P2	< 5 min
Pen-test critical finding (0-day lub architekuralna dziura)	Etap 5 → 4 (D3+ human pre-screening reaktywowany)	Security team + Board — alert P1	< 5 min
Council quorum nieosiągalny > 48h (incapacitation)	Etap 5 → 3 (tylko sandbox; brak prod changes bez quorum)	AEIS Steward + zastępczy operator	48h timeout

Każdy downgrade jest rejestrowany w Evidence Spine jako zdarzenie aeis.autonomy.downgrade z polem trigger_class, from_stage, to_stage, reason_ref. Powtórne włączenie wyższego etapu po downgrade wymaga pełnego cyklu gating (entry + exit criteria od nowa), nie można „ominiać” bramy powtarzanym przełączaniem.

§R7.8 — Zakończenie

Pięć etapów Autonomy Rollout jest zaprojektowanych tak, by każdy krok budował kapitał zaufania mierzalny w twardych metrykach. Od 30 dni milczącej obserwacji (etap 1) przez sandbox experiments (etap 3) po pełną autonomię pod kanonem (etap 5) — każde przejście jest zatwierdzone dowodem, a nie deklaracją.

Drabina D0–D5 nie jest tylko klasyfikacją zmian — jest mechanizmem delegacji odpowiedzialności. AEIS działa autonomicznie tam, gdzie ryzyko jest mierzalne i odwracalne. Człowiek pozostaje decydem tam, gdzie konsekwencje przekraczają możliwości

automatycznej walidacji.

AUTONOMY·CANON · 5 etapów — 4 miesiące governed ewolucji AEIS

„5 etapów autonomii to 4 miesiące konkretnej, governed ewolucji AEIS. Po etapie 5, SYLION jest samomodyfikującym się systemem pod kanonem — ale zawsze z Mission Preservation jako ostatnią, nieprzekraczalną granicą.”

AEIS · AEIS — Autonomy under Canon (meta-zasada 4)

AEIS jest systemem autonomicznym pod kanonem, nie ponad kanonem. Każdy akt autonomii jest dowodem zaufania zbudowanego krok po kroku — nie przywilejem wynikającym z istnienia systemu. Etap 5 jest kulminacją procesu, nie punktem startowym.

ROZDZIAŁ R8 · AEIS IMPLEMENTATION MASTERPLAN

Security Hardening Path + Integration Assembly

R8 — Security Hardening Path + Integration Assembly Phases

R8.0 — Wprowadzenie: Security jako podmiana, nie przebudowa

Klasyczny błąd w projektach inżynierskich polega na traktowaniu security jako warstwy, którą „zrobimy później”. Konsekwencja jest zawsze ta sama: gdy projekt zbliża się do produkcji, okazuje się, że architektura nie przewidywała izolacji, rotacji sekretów ani podpisanych logów audytu. Każda próba dodania security w takim momencie wymaga przebudowy architektury — nie konfiguracji, nie deployment. Przebudowy.

SYLION eliminuje ten problem przez zasadę Modularity by Contract: wszystkie 8 modułów security istnieje od dnia 1 i ma zamrożone kontrakty .proto od momentu Contract Freeze Milestone (patrz R2). Na etapie M0 moduły te działają w profilu dev-light — implementacje uproszczone, bez infrastruktury produkcyjnej. Na etapie M5, przez lifecycle shadow → dual → cutover, każdy moduł wymienia wkład implementacyjny na wariant prod-strict. Kontrakt się nie zmienia. Zmieniamy wyłącznie zawartość gniazda.

Dzięki temu setki instalacji w fazie M0–M4 mogą działać na uproszczonym profilu security bez ryzyka długu architektonicznego. Każda instalacja jest zdolna do twardego hartowania (hardening) bez rebuild systemu — wystarczy podmienić Docker image i uruchomić lifecycle cutover.

AUTONOMY·CANON · Security nigdy nie jest dodawane

„Security nigdy nie jest added. Security jest zawsze present. Zmienia się tylko enforced poziom.”

R8.1 — 8 modułów security i ich hardening path

Poniżej opisano każdy z ośmiu modułów klasy F (Security) zgodnie z katalogiem modułów (patrz R3). Dla każdego modułu podana jest implementacja M0 (dev-light), implementacja M5 (prod-strict), faza podmiany, bramki wejścia i wyjścia oraz — co kluczowe — kontrakt, który pozostaje niezmienny.

1. sylion.security.auth_provider

Dostawca tożsamości. W M0 działa na lokalnych tokenach UUIDv4 z nagłówkiem Bearer i hardkodowanym administratorem w .env. W M5 podmieniony na pełen Keycloak OIDC z JWT, rotacją refresh tokenów i administratorem zarządzanym przez zewnętrzny IdP. Swap odbywa się w fazie M5 faza 1 — jest to pierwszy moduł do zamiany, ponieważ reszta hardening zależy od tożsamości.

Wymiar	Dev-light (M0)	Prod-strict (M5)
Implementacja tokenów	Local UUIDv4, plain Bearer	Keycloak OIDC + JWT
Refresh token	Brak rotacji	Rotacja co 30 min, invalidacja after-use
Admin account	Hardkodowany w .env	Zewnętrzny IdP, mandatory TOTP
Lifecycle swap	—	Shadow 7 dni → dual 7 dni → cutover
Entry gate swap	—	Keycloak deployed i stable 3 dni w staging
Exit gate swap	—	0 auth failures w oknie cutover 48h; bug bounty offered
Kontrakt .proto	AuthProvider.v1.0.0	AuthProvider.v1.0.0 (BEZ ZMIAN)

2. *sylicon.security.bootstrap_init*

Inicjalizacja systemu przy pierwszym uruchomieniu. W M0: plaintext admin init, seed przez plik .env. W M5: secure seed przez hardware token (Yubikey lub HSM), mandatory TOTP przy pierwszym logowaniu, stare bootstrapy zachowane read-only dla backward-compat. Swap w M5 faza 1.

Wymiar	Dev-light (M0)	Prod-strict (M5)
Inicjalizacja admina	Plaintext w .env	Hardware token (Yubikey/HSM)
Pierwsze logowanie	Brak wymogów	Mandatory TOTP, forced password change
Stare bootstrapy	Brak zarządzania	Zachowane read-only, audit log dostępu
Backward-compat	—	Stare bootstrapy czytelne, nie wykonywalne
Kontrakt .proto	BootstrapInit.v1.0.0	BootstrapInit.v1.0.0 (BEZ ZMIAN)

3. *sylicon.security.session_broker*

Broker sesji użytkowników i serwisów. W M0: in-memory sessions lokalnie w procesie — nie przeżywają restartu. W M5: Redis cluster z JWT, refresh token rotation co 30 minut. Swap w M5 faza 2; canary deployment — najpierw jeden serwis, potem stopniowe rozszerzanie.

Wymiar	Dev-light (M0)	Prod-strict (M5)
Storage sesji	In-memory (proces-lokalnie)	Redis cluster (HA, multi-node)
Token format	Opaque UUID	JWT (RS256) + refresh token
Rotacja sesji	Brak	Co 30 minut, auto-invalidation stale sessions
Deployment swap	—	Canary per service, faza 2
Kontrakt .proto	SessionBroker.v1.0.0	SessionBroker.v1.0.0 (BEZ ZMIAN)

4. *sylicon.security.policy_engine*

Silnik polityk dostępu. W M0: prosta allow-lista w YAML (RBAC v0) — role definiowane statycznie, bez granularnych uprawnień per RPC. W M5: Open Policy Agent (OPA) z politykami Rego, fine-grained scopes per RPC, dynamiczna ewaluacja. Swap w M5 faza 2 — po stabilizacji auth_provider.

Wymiar	Dev-light (M0)	Prod-strict (M5)
Mechanizm polityk	Allow-list YAML (RBAC v0)	OPA + Rego policies
Granularność	Role-level (admin/user/svc)	Fine-grained scope per RPC
Dynamika	Statyczna (restart required)	Hot-reload polityk bez restartu
Policy audit	Brak	Każda decyzja logowana w audit_sink
Kontrakt .proto	PolicyEngine.v1.0.0	PolicyEngine.v1.0.0 (BEZ ZMIAN)

5. *sylicon.security.execution_guard*

Strażnik izolacji wykonania kodu agentowego. W M0: subprocess isolation przez Python multiprocessing — uproszczona izolacja zasobów bez kernel-level separation. W M5: pełna izolacja przez gVisor lub Firecracker microVM z PHANTOM strict wrap. Jest to największy swap — wymagający 2 sprintów dedykowanej pracy. Swap w M5 faza 3 (ostatnia faza, tuż przed autonomy etap 3).

Wymiar	Dev-light (M0)	Prod-strict (M5)
Izolacja	Python multiprocessing	gVisor lub Firecracker microVM
Kernel separation	Brak (shared kernel)	Dedykowane gVisor kernel (runsc)
PHANTOM wrap	Skeleton (lekki subprocess)	PHANTOM strict (full isolation)
Czas podmiany	—	2 sprinty (największy swap)
Swap faza	—	M5 faza 3 (przed autonomy etap 3)
Kontrakt .proto	ExecutionGuard.v1.0.0	ExecutionGuard.v1.0.0 (BEZ ZMIAN)

6. *syllion.security.secret_provider*

Dostawca sekretów dla wszystkich modułów systemu. W M0: plik .env i lokalny filesystem — wystarczający dla lokalnego dev, niedopuszczalny w produkcji. W M5: HashiCorp Vault z automatyczną rotacją sekretów i audit logiem per każdy dostęp do sekretu. Swap w M5 faza 1 — sekrety muszą być gotowe przed resztą hardening, ponieważ wszystkie inne moduły prod-strict konsumują sekrety przez Vault.

Wymiar	Dev-light (M0)	Prod-strict (M5)
Storage sekretów	.env file + local filesystem	HashiCorp Vault (cluster HA)
Rotacja sekretów	Ręczna (restart required)	Automatyczna, konfigurowalna
Audit dostępu	Brak	Log per secret access (kto, kiedy, wynik)
Priorytet swap	—	Faza 1 — warunek reszty hardening
Kontrakt .proto	SecretProvider.v1.0.0	SecretProvider.v1.0.0 (BEZ ZMIAN)

7. *syllion.security.audit_sink*

Ujście audytowe dla wszystkich zdarzeń bezpieczeństwa systemu. W M0: append-only text log — prosty, czytelny, bez gwarancji integralności. W M5: łańcuch Ed25519-signed z właściwością tamper-evident, możliwością replay (odtworzenia sekwencji zdarzeń) oraz 7-letnim retencją. Swap w M5 faza 1 — podpisany audit_sink musi być aktywny przed wszelkim działaniem autonomy etapu 2+ (patrz R8.4).

Wymiar	Dev-light (M0)	Prod-strict (M5)
Format logu	Append-only text (plain)	Ed25519-signed chain (per event)
Tamper detection	Brak	Hash chain — wykrywa dowolną modyfikację
Replay capability	Brak	Pełna sekwencja odtwarzalna z weryfikacją
Retencja	Brak polityki	7-letnia retencja z tiered storage
Swap faza	—	Faza 1 (przed autonomy etap 2)
Kontrakt .proto	AuditSink.v1.0.0	AuditSink.v1.0.0 (BEZ ZMIAN)

8. *syllion.security.phantom_wrapper*

Pełna powłoka izolacyjna PHANTOM dla modułów agentowych. W M0: skeleton — lekki subprocess z resource limits (CPU/RAM cap), bez izolacji sieciowej ani schematu bazy. W M5: full PHANTOM strict — isolated Docker network, dedykowany schemat Postgres per agent, dedykowany prefix NATS, quotas zasobów oraz gVisor kernel isolation. Swap w M5 faza 3, tuż

przed autonomy etap 3 (sandbox self-change wymaga pełnej izolacji PHANTOM).

Wymiar	Dev-light (M0)	Prod-strict (M5)
Network isolation	Brak (shared network)	Isolated Docker network per agent
DB isolation	Shared schema	Dedykowany schemat Postgres per agent
NATS isolation	Shared subjects	Dedykowany prefix NATS per agent
Kernel isolation	Shared host kernel	gVisor kernel isolation (runsc)
Resource quotas	Soft limits (Python)	Hard quotas (cgroups v2)
Swap faza	—	Faza 3 (warunek autonomy etap 3)
Kontrakt .proto	PhantomWrapper.v1.0.0	PhantomWrapper.v1.0.0 (BEZ ZMIAN)

Tabela zbiorcza — 8 modułów security

Poniższa tabela podsumowuje wszystkie 8 modułów: implementację dev, implementację prod, fazę podmiiany w M5 oraz — kluczową gwarancję — że kontrakt nigdy się nie zmienia.

Moduł	Dev impl (M0)	Prod impl (M5)	Swap faza M5	Kontrakt zmienia się?
auth_provider	Local UUIDv4 Bearer	Keycloak OIDC + JWT	Faza 1	NIE
bootstrap_init	Plaintext .env seed	Yubikey/HSM + TOTP	Faza 1	NIE
session_broker	In-memory sessions	Redis + JWT rotation	Faza 2	NIE
policy_engine	RBAC YAML allow-list	OPA + Rego fine-grained	Faza 2	NIE
execution_guard	Python multiprocessing	gVisor/Firecracker microVM	Faza 3	NIE
secret_provider	.env + filesystem	HashiCorp Vault + rotation	Faza 1	NIE
audit_sink	Append-only text log	Ed25519 signed chain	Faza 1	NIE
phantom_wrapper	Skeleton subprocess	Full PHANTOM strict	Faza 3	NIE

R8.2 — 4 profile security w praktyce

System SYLION operuje w jednym z czterech profili security w danym momencie. Profil to kombinacja trybu autoryzacji (auth_mode), poziomu izolacji wykonania (execution_guard) oraz trybu audytu (audit_mode). Przełączenie między profilami wymaga: 1 zmiennej środowiskowej i 1 podmiiany Docker image — ten sam kontrakt, inna implementacja.

Profil	Środowisko	auth_mode	execution_guard	audit_mode	Kiedy aktywny
dev-light	Lokalny dev	bootstrap (local token)	off	basic (log only)	M0–M4 lokalnie; CI sprinty 1–2
test-light	CI pipeline	bootstrap (ephemeral)	light (subprocess)	basic (log only)	CI od sprintu 3; smoke tests
staging-strict	Pre-prod env	oidc (Keycloak staging)	strict (PHANTOM)	extended (structured evidence)	Pre-prod od M3; M4 final validation
prod-strict	Production	oidc (Keycloak prod)	strict (PHANTOM + gVisor)	signed (Ed25519 per event)	Production od M5 hardening cutover

Przejsie między profilami jest mechanicznie proste: zmienna środowiskowa SYLION_SECURITY_PROFILE=prod-strict wskazuje Orchestratorowi środowisk (patrz Plan 01 — ACP), który Docker image załadować dla każdego z 8 modułów security. Moduły business (Cognitive, Execution, Memory) nie wiedzą o profilu — konsumują kontrakty security identyczne w każdym profilu.

R8.3 — Integration Assembly Phases

Po Contract Freeze Milestone (patrz R2) wszystkie zespoły pracują równolegle na swoich modułach. System musi być jednak składany w spójną całość w 5 fazach Assembly. Fazy Assembly nie zastępują Module Lifecycle (draft → build → validate → shadow → dual → cutover → stable) — są ortogonalne: opisują, jak zestaw modułów łączy się w działający system.

Faza 1 — Module Scaffolding (M0–M1)

Każdy moduł otrzymuje kompletny szkielet infrastruktury. Celem nie jest działająca logika biznesowa, lecz zdolność systemu do odnalezienia każdego modułu w Registry i wywołania go przez kontrakt — nawet jeśli odpowiedź jest hard-coded stub. Na tym etapie ważne jest, że sieć kontraktów istnieje i jest przetestowana.

- Każdy moduł: Dockerfile, module.yaml (manifest), pusty gRPC service, pusty NATS subscriber.
- Postgres schema migration skryptem (alembic lub raw SQL) — schemat istnieje, tabele puste.
- Registry lookup działa: moduł A może odnaleźć moduł B w Module Registry, choć B jeszcze nie odpowiada znaczącą treścią.
- Test harness aktywny: contract tests uruchamiane przeciw stubowanym responses (nie real logic).
- buf lint + buf breaking w CI — każdy commit blokowany przy naruszeniu kontraktu.

Faza 2 — Contract Conformance (M1–M2)

Każdy moduł implementuje pełny kontrakt .proto: wszystkie RPC zwracają poprawną strukturę odpowiedzi — najpierw hard-coded, potem real logic. Kluczowa zmiana względem Fazy 1: moduł A wywołuje moduł B przez gRPC — ale B jest mockiem (nie real deployment). Cel: pewność, że kontrakt działa end-to-end między każdą parą modułów.

- buf breaking enforced w każdym merge — żadna zmiana kontraktu bez D3 Council.
- Integration tests vs mocked deps — każdy moduł testuje swoje zależności przez mock kontrakt-zgodny.
- Output: każdy moduł odpowiada zgodnie z kontraktem; logika wewnętrzna może być stubem.
- Security: profil test-light aktywny w CI od sprintu 3.

Faza 3 — Shadow Integration (M3–M4)

Moduły przestają wywoływać mocki i zaczynają komunikować się z realnymi modułami. Cały system jest wdrożony w środowisku shadow — nikt zewnętrzny nie ma do niego dostępu. Opcjonalnie: ruch z potencjalnego prod jest mirrorowany do shadow env (traffic mirroring). Diagnostic dashboards pokazują rozbieżności między outputem shadow a oczekiwanym (drift detection).

- Każdy moduł rozmawia z realnym sąsiadem — nie mockiem.
- Traffic mirror z potencjalnego prod do shadow: porównanie outputów bez wpływu na użytkowników.
- Diagnostic dashboards: drift per moduł, per RPC, per event stream.
- Security: profil staging-strict aktywny w shadow env od M3.
- Output: każdy moduł zintegrowany z realnymi sąsiadami, zero real traffic z zewnątrz.

Faza 4 — Dual Integration (M4)

Wybrane moduły trafiają na produkcję z podzielonym ruchem: 10% → 50% → 90% weight. Legacy (lub mocki) obsługują pozostałą część. Każdy moduł jest oceniany indywidualnie — nie wszystkie moduły wchodzi do dual w tym samym sprincie. Evidence Pack jest budowany dla każdego flow w produkcji.

- Per-moduł decision: czy moduł jest gotowy do dual (evidence pack, zero regressions).
- Auto-rollback: jeśli regression zostanie wykryta, moduł wraca do 0% automatycznie przez sylon.rebuild.cutover_controller.
- Traffic weight: 10% → 50% → 90% per moduł — każdy krok wymaga zielonego evidence.
- Output: system częściowo w produkcji; część modułów obsługuje realnych użytkowników.

Faza 5 — Cutover to Integrated Runtime (M5)

100% ruchu trafia na realne moduły SYLION. Legacy (mocki, poprzednie implementacje) wchodzi w stan deprecated z Legacy Preservation Window (LPW) 7 dni, po czym są usuwane. W tej fazie następuje migracja z docker-compose do Kubernetes z NetworkPolicies i service

mesh. To jednocześnie moment hardening security: wszystkie 8 modułów security wymienia implementację na prod-strict.

- 100% traffic na realne moduły — brak legacy path.
- Legacy (mocki) deprecated: LPW 7 dni, potem hard delete z Registry.
- K8s migration: compose → K8s NetworkPolicies + Istio/Linkerd service mesh.
- Security hardening cutover (patrz R8.1): wszystkie 8 modułów → prod-strict w kolejności faz 1 → 2 → 3.
- Output: pełny SYLION w produkcji, architektura K8s, security prod-strict.

Tabela zbiorcza — Integration Assembly Phases

Faza	Zakres	Entry gate	Exit gate	Milestone target	Ryzyko
1 Module Scaffolding	Skeleton Dockerfile, manifest, gRPC stub, NATS sub, schema	Contract Freeze Milestone (R2) zaliczone	Registry lookup działa dla każdego modułu; CI zielone	M0–M1 (sprinty 1–4)	Niekompletne manifesty → blokada modułów zależnych
2 Contract Conformance	Każdy moduł implementuje pełny .proto	buf breaking czyste; CI zielone	Integration tests vs mocks: 100% conformance	M1–M2 (sprinty 3–8)	Hard-coded response zastąpiony zbyt wcześnie
3 Shadow Integration	Realne moduły, shadow env, traffic mirror	Wszystkie moduły w stanie validate; staging-strict	Drift <1% dla krytycznych flows; 0 contract errors	M3–M4 (sprinty 9–14)	Drift ukryty do M5 — wykryty zbyt późno
4 Dual Integration	10→50→90% traffic, legacy parallel	Evidence Pack per flow; 0 regressions	90% traffic na realnych modułach; auto-rollback nie aktywowany	M4 (sprinty 13–16)	Rollback cascades jeśli moduł zależy od niestabilnego sąsiada
5 Cutover to Integrated Runtime	100% traffic, K8s migration, security hardening	Dual phase zaliczone; pen-test sprint 18 clean	0 legacy modules aktywnych; prod-strict security; K8s stable 72h	M5 (sprinty 17–20)	K8s migration opóźniona blokuje security hardening

R8.4 — Zależność: Autonomy wymaga Security

Każdy etap Autonomy Rollout (patrz R7) ma twardą zależność od konkretnego poziomu dojrzałości security. Nie jest to zależność konwencjonalna („chcielibyśmy mieć security”) — jest to formalna bramka: etap autonomy nie może się rozpocząć, jeśli wymagana faza security nie jest w stanie stable z pozytywnym evidence pack.

<i>Etap Autonomy</i>	<i>Wymaga Security Phase</i>	<i>Powód</i>
Etap 1 — Observe-only (M4 start)	audit_sink: basic (dev-light wystarczy)	Observation wymaga zapisu zdarzeń; basic log jest wystarczający dla read-only obserwacji
Etap 2 — Propose-only (M4 mid)	audit_sink: extended (structured evidence)	Propozycje zmian muszą być audytowalne ze structured evidence; basic log nie wystarczy
Etap 3 — Sandbox self-change (M5 start)	phantom_wrapper: PHANTOM strict execution_guard: strict (gVisor)	Sandbox self-change wymaga pełnej izolacji PHANTOM — bez niej agent może wpłynąć na hosta
Etap 4 — Limited prod self-change (M5 mid)	auth_provider: Keycloak OIDC audit_sink: Ed25519 signed polity: mTLS + OPA	Forensics dla rollback wymaga w pełni podpisanego audit trail; OIDC + mTLS gwarantuje attributability
Etap 5 — Full governed autonomy (M5 end)	Wszystkie 8 modułów: prod-strict Pen-test sprint 20: clean (0 critical, 0 high)	Najwyższy poziom rygoru; full autonomy niedopuszczalna bez clean pen-test

R8.5 — Strategia pen-testów

SYLION przewiduje trzy zewnętrzne oceny bezpieczeństwa przed i w trakcie M5 cutover. Każdy pen-test ma zdefiniowany zakres, zespół i kryteria wyjścia. Wynik pen-testu jest dowodem Evidence Pack dla decyzji D5 (security architecture) — bez clean pen-testu autonomy etap 5 nie zostaje aktywowany.

Pen-test 1 — Sprint 15 (M4 mid): Kernel + Governance

Zakres: moduły Kernel (8) + Governance (7) w profilu test-light. Cel: weryfikacja izolacji modułowej, kontraktów event bus, decision gate engine i evidence spine. Zespół: internal security team (red team internal). Fokus: contract injection, event replay attacks, privilege escalation przez governance path.

- Zakres: Kernel + Governance (15 modułów)
- Profil: test-light (nie prod-strict — pen-test nie weryfikuje hardening M5)
- Zespół: internal red team
- Exit criteria: 0 critical, 0 high, <5 medium, N/A low

Pen-test 2 — Sprint 18 (M5 mid): Pełny system, profil dev

Zakres: pełny system SYLION (~65 modułów) w profilu dev-light → test-light. Cel: weryfikacja całości przed hardening cutover. Zewnętrzny zespół security z bug bounty. Fokus: cross-module attacks, gRPC contract exploitation, NATS event injection, secret leakage przez .env.

- Zakres: pełny system (~65 modułów)
- Profil: dev-light / test-light (weryfikacja architektury, nie hardening)

- Zespół: zewnętrzny, bug bounty program otwarty
- Exit criteria: 0 critical, 0 high, <5 medium, N/A low

Pen-test 3 — Sprint 20 (M5 end): Pełny prod-strict + Autonomy etapy 1–4

Zakres: pełny system w profilu prod-strict z aktywnymi etapami 1–4 autonomy. Jest to najważniejsza ocena — weryfikuje system gotowy do pełnej produkcji. Zewnętrzny zespół + Red Team symulowany atak (APT scenario). Fokus: PHANTOM breach, Vault secret exfiltration, audit log tampering, autonomy path exploitation, K8s NetworkPolicy bypass.

- Zakres: pełny system prod-strict + autonomy etapy 1–4
- Profil: prod-strict (Keycloak, gVisor, Vault, Ed25519 audit)
- Zespół: zewnętrzny + Red Team (APT scenario simulation)
- Exit criteria: 0 critical, 0 high, <5 medium, N/A low
- Decyzja klasy D5 — wynik wchodzi do Evidence Pack dla full autonomy

<i>Pen-test</i>	<i>Sprint</i>	<i>Zakres</i>	<i>Profil</i>	<i>Zespół</i>	<i>Exit criteria</i>
1	Sprint 15 (M4 mid)	Kernel + Governance (15 modułów)	test-light	Internal red team	0 critical 0 high <5 medium
2	Sprint 18 (M5 mid)	Pełny system (~65 modułów) dev profile	test-light	Zewnętrzny + bug bounty	0 critical 0 high <5 medium
3	Sprint 20 (M5 end)	Pełny system prod-strict + autonomy 1–4	prod-strict	Zewnętrzny + Red Team (APT scenario)	0 critical 0 high <5 medium D5 evidence

R8.6 — Podsumowanie rozdziału

Security Hardening Path w SYLION nie jest projektem do wykonania „na koniec”. Jest ciągiem przewidywalnych podmian wkładów w gniazda, które istnieją od dnia Contract Freeze. Każdy moduł security ma zamrożony kontrakt i dwuetapową implementację: uproszczoną (dev-light) na potrzeby M0–M4 oraz produkcyjną (prod-strict) gotową do podmiany w M5. Integration Assembly Phases gwarantują, że ten system złożony z ~65 modułów jest składany inkrementalnie — bez skokowego „big bang deployment” na M5.

AUTONOMY·CANON · Security Hardening to podmiana wkładów, nie przebudowa

„Security Hardening to nie faza do zrobienia później. To ciąg podmian wkładów w gniazda, które istnieją od dnia 1. Architektura się nie zmienia. Zmienia się tylko enforced level. Dzięki temu setki instalacji w fazie budowy są możliwe — a pełny system produkcyjny jest gotowy w M5.”

ROZDZIAŁ R9 · AEIS IMPLEMENTATION MASTERPLAN

Ryzyka, Evidence Integration, Post-M5

R9 — Ryzyka, Evidence Integration, Post-M5

§R9.0 Wprowadzenie

„Masterplan to nie lista życzeń. To mapa z zaznaczonymi pułapkami i sposobami ich omijania.”
Każda decyzja projektowa, każdy milestone i każde przejście autonomii niesie ze sobą iden-
tyfikowalną klasę ryzyka. Rozdział R9 zamyka Masterplan trzema częściami:

- 1. Top 12 ryzyk systemowych — macierz prawdopodobieństwo × impact z pełnymi mitygacjami, ownerami i trigger milestone’ami.
- 2. Evidence Pack Integration — schemat, storage, replay capability i integracja z modułami; każda decyzja D2+ generuje ślad sądowy.
- 3. Co dalej po M5 — steady state operations, możliwe rozszerzenia (Federation, Multi-tenant, SYLION-as-a-Service) i finalny callout domykający Masterplan.

Ryzyka opisane poniżej nie są hipotetyczne — wynikają z analizy historii podobnych projektów modularnych oraz specyfiki SYLION AEIS: 65 modułów, 8 strumieni równoległych, 5 etapów autonomii i 8 modułów security podmienianych implementacyjnie pod niezmiennymi kontraktami.

§R9.1 Top 12 ryzyk systemowych

Poniższa macierz obejmuje 12 ryzyk uznanych za systemowe — tzn. mogących wpłynąć na więcej niż jeden strumień lub milestone. Kolumny: Prawdopodobieństwo (H = wysokie, M = średnie, L = niskie) oraz Impact (C = critical, H = high, M = medium, L = low).

Legenda P/I: P = Prawdopodobieństwo (H=wysokie, M=średnie, L=niskie); I = Impact (C=critical, H=high, M=medium, L=low).

#	Ryzyko	P/I	Mitygacja	Owner	Trigge r
1	Kontrakty .proto źle zaprojektowane od dnia 1 — Lego się rozsypuje	H/C	Dedicated Architect lead od sprint 0, weekly contract review, buf breaking enforced, proto review rotation	Architektura	M0
2	Contract Freeze opóźniony (M0 slip > 2 sprinty) — cascading delays	M/C	Podwójne resourcing Kernel team, critical path tracking dashboard, Board escalation at day 5 of slip	Architektura + Board	M0
3	Council 4/4 staje się bottleneck (> 5 dni proposal→decision)	H/M	Council mock w devs, Council reserves (rotating bench 7 osób), SLA 48h p95 monitored weekly	Compliance	M1

#	Ryzyko	P/I	Mitygacja	Owner	Trigger
4	AEIS generuje noise w improvement queue (> 70 % rejected)	M/M	Precision threshold tunable, anomaly detection retraining co miesiąc, Steward filter, quality gate dla proposals	Strategy	M4
5	Self-modyfikacja prowadzi do regression w prod (etap 4)	M/H	Auto-rollback < 2 min, golden set 1000+ cases, regression detection w observation, shadow deploy 72h	Quality+SRE	M5
6	Hot swap security (M5) powoduje downtime	L/C	LPW 7 dni dla każdej starej impl, blue-green deploy, rollback drill 3x przed prod, każdy moduł testowany osobno	Security+SRE	M5
7	K8s migration odkrywa networking issues	M/M	Każdy moduł deployed w stage env z full NetworkPolicies przed prod, 30-day K8s dry-run w staging	SRE	M5
8	Pen-test znajduje critical findings	M/C	Red Team engagement od sprint 18 (bug bounty mode), 4-tyg buffer dla fixes, 3 pen-testy w różnych momentach	Security	M4–M5
9	Operator Console scope creep	H/L	8 paneli + 7 widoków RBAC = fixed scope w Plan 17, nowe panele w post-M5, UX review co 2 tygodnie	Implementation	M3
10	NATS JetStream durability issues pod load	L/H	SRE eval w sprint 1, fallback do Kafka jeśli SLO nie spełniony w M0, load test 10k msg/s przed M1	SRE	M0
11	AEIS Steward role nie obsadzona (brak senior architect z mandatem weta)	M/C	Szukanie kandydata od sprint 1; alternatywnie Board pełni rolę do M4; Steward-Shadow program (backup person)	Strategy+Board	M4
12	Modularity by Contract łamana przez „quick fix” direct imports	H/H	Import-linter w CI jako blocker, code review check-list, szkolenie nowych developerów w onboarding, grep-audit weekly	Quality+Architektura	M1+

Mitygacje cross-cutting

Cztery mitygacje działające poprzecznie dla wielu ryzyk jednocześnie:

- Critical path tracking dashboard live od sprint 0 — wizualizacja stanu każdego strumienia, słupki RAG (Red/Amber/Green) per milestone.
- Weekly Board review z RAG status per strumień — Board widzi status tygodniowo, escalation trigger przy Red.
- Evidence pack dla każdej istotnej decyzji — forensics trail pozwala zidentyfikować genezę problemu w każdym post-mortem.

- Dedicated „Integration Architect” w każdym strumieniu (non-optional) — odpowiada za spójność kontraktów między modułami własnego strumienia i z innymi strumieniami.

UWAGA · Ryzyka architektoniczne — nie technologiczne

Top 3 ryzyka (kontrakty złe, Contract Freeze opóźniony, Lego złamane) to ryzyka architektoniczne — nie technologiczne. Ich mitygacja wymaga dyscypliny procesowej, nie tooling.

§R9.2 Evidence Pack Integration

Audytowalność nie jest opcjonalną funkcją — jest wymaganiem operacyjnym systemu autonomicznego. Każda decyzja D2+ w SYLION AEIS generuje Evidence Pack: samodzielny, podpisany kryptograficznie artefakt zawierający pełny kontekst decyzji, wyniki bram i sygnatury członków Council.

§R9.2.1 Schemat Evidence Pack

Każda decyzja D2+ generuje Evidence Pack zawierający dziewięć elementów:

1. Proposal payload — oryginalny request (typ, moduł docelowy, opis zmiany).
2. Decision classification — D-klasa + uzasadnienie klasyfikacji.
3. Gate results — które G-* gates evaluated, pass/fail z timestampami.
4. Council vote breakdown — per member: vote + uzasadnienie (obowiązkowe dla D3+).
5. Diff (przed/po) dla zmienianych artefaktów (.proto, YAML, kod).
6. Performance impact — efficiency 4x baseline diff (delta metryk po zmianie).
7. Self-Explanation template (Aneks R wypełniony) — obowiązkowe dla AEIS-driven D3+.
8. Signature Ed25519 z private key audit_sink — gwarantuje niemożliwość cichej modyfikacji.
9. Timestamp chain — każdy event z timestampem monotonic + UTC; tamper detection oparty na sekwencji.

Przykład: evidence_pack.json (D3, zmiana kontraktu CouncilWorkflow)

JSON

```
{
  "pack_id": "evp_2026-04-20_1234abc",
  "created_at": "2026-04-20T09:15:00Z",
  "decision_ref": "dec_78910",
  "decision_class": "D3",
  "proposal": {
    "type": "contract_change",
    "target_module": "sylon.governance.council_workflow",
    "change": "add optional field `escalation_priority` to VoteRequest"
  },
  "gates_evaluated": [
    {"gate": "G-EFF-01", "result": "pass", "at": "2026-04-20T09:14:12Z"},
  ]
}
```

```
    {"gate": "G-REVERSE-01", "result": "pass", "at": "2026-04-20T09:14:33Z"}
  ],
  "council_vote": {
    "quorum_reached_at": "2026-04-20T09:14:55Z",
    "members": [
      {"role": "Architect", "vote": "yes", "rationale": "non-breaking additive"},
      {"role": "PM", "vote": "yes", "rationale": "aligns with P17 roadmap"},
      {"role": "Implementer", "vote": "yes", "rationale": "feasible"},
      {"role": "Reviewer", "vote": "yes", "rationale": "test coverage added"}
    ]
  },
  "diffs": {
    "contracts/governance/council_workflow/v1.2.0/council.proto": "..."
  },
  "self_explanation": null,
  "signature": "ed25519:0xabc...def",
  "timestamp_chain": [
    {"event": "proposal.created", "at": "2026-04-20T09:13:00Z"},
    {"event": "decision.classified", "at": "2026-04-20T09:13:15Z"},
    {"event": "council.vote.finalized", "at": "2026-04-20T09:14:55Z"},
    {"event": "evidence_pack.created", "at": "2026-04-20T09:15:00Z"}
  ]
}
```

Podsumowanie schematu Evidence Pack — pola obowiązkowe według klasy decyzji:

Pole	D2	D3	D4	D5	Uwagi
proposal_payload	✓	✓	✓	✓	zawsze
decision_class	✓	✓	✓	✓	zawsze
gates_evaluated	✓	✓	✓	✓	zawsze
council_vote_breakdown	—	✓	✓	✓	D3+ obowiązkowo
diff	opt.	✓	✓	✓	D3+ obowiązkowo
performance_impact	opt.	✓	✓	✓	G-EFF-01 trigger
self_explanation	—	AEIS	✓	✓	AEIS D3+ i wszystkie D4/D5
signature_ed25519	✓	✓	✓	✓	zawsze, audit_sink key
timestamp_chain	✓	✓	✓	✓	zawsze, monotonic

§R9.2.2 Storage i Retention

Evidence Packs są przechowywane w tabeli brick_evidence.evidence_packs (PostgreSQL 16) ze schematem: JSONB payload + signature bytea + indeks po decision_ref, decision_class i created_at.

- Retention D3+: 7 lat (wymogi compliance, równoważne z dokumentacją finansową).
- Retention D2: 2 lata (standardowa historia feature).
- Retention D0–D1: 1 rok (operacyjna historia eventów).

- Backup: daily off-site snapshot; replayable full do dowolnego punktu w czasie (point-in-time restore).
- Access control: append-only (zapisy przez signed RPC wyłącznie); odczyty przez dedykowaną rolę `audit-role` z pełnym logowaniem dostępu.

§R9.2.3 *Replay capability*

Każda decyzja może być replayed w dowolnym momencie — jest to mechanizm służący zarówno post-mortem, jak i detekcji tamperingu:

- Reconstruct state as-of — odtwórz stan systemu bezpośrednio przed daną decyzją (z Event Spine snapshot).
- Re-run gates — uruchom G-* gates ponownie przeciwko zrekonstruowanemu stanowi; porównaj wynik z oryginalnym.
- Compare final state — porównaj oczekiwany stan post-decision z aktualnym stanem prod; wykryj dryft.
- Tamper detection — każda niezgodność między replayed a stored Evidence Pack jest traktowana jako anomalia bezpieczeństwa (alert do Security+SRE).
- Use cases: post-mortem produkcyjny, audit compliance, AEIS Self-Explanation learning (dane treningowe dla modelu uzasadniania decyzji).

§R9.2.4 *Integracja z modułami*

Evidence Pack nie jest pisany ręcznie przez żadnego developera. Generowany jest automatycznie przez łańcuch eventów z istniejących kontraktów:

- Każdy moduł wywołujący `core.decision_engine.EvaluateDecision` automatycznie publikuje evidence events do Evidence Spine.
- AEIS `self_explanation` tworzy template dla AEIS-driven D3+, który staje się częścią Evidence Pack (patrz Plan 19 §19.5 i Aneks R).
- Gates registry publikuje `gate_result` events, które są aglutynowane do Evidence Pack przez `syllion.governance.evidence_workflow`.
- Zero ręcznego evidence-writing — wszystko generowane z istniejących kontraktów (.proto events + NATS subjects). Modularity by Contract eliminuje potrzebę osobnych „audit hooks”.

§R9.3 *Post-M5 — co dalej*

Osiągnięcie M5 oznacza, że SYLION AEIS jest pełnym, samouczącym się systemem inżynierskim pod kanonem, z twardym zabezpieczeniem security (prod-strict) i całkowitą observability. To jednak nie koniec — to początek fazy eksploatacyjnej.

§R9.3.1 *Steady state operations*

Po M5 system wchodzi w tryb steady state — AEIS Steward + Compliance team prowadzą operacje, Masterplan team jest rozformowany. Rytm ops:

Rytm	Aktywność	Owner	Artefakt
Weekly	Retro per strumień: Evidence pack highlights, metryki per SLO	Tech Leads	Weekly status + RAG dashboard
Monthly	External review: Steward + Board + External advisor; evidence sample audit	AEIS Steward	Monthly ops report
Quarterly	Deep audit: pen-test, evidence replay sample, contract drift analysis	Security+Compliance	Quarterly audit report
Yearly	Canon evolution review: czy 5 meta-zasad są nadal aktualne? Co zmienić?	Board + Steward	Canon revision proposal (D5)

Autonomy operuje w etapie 5; rollback do niższych etapów jest automatyczny na trigger (regression w observation, SLO breach, tamper detection w Evidence Spine). Rollback nie wymaga decyzji operatora — jest skutkiem kontraktów rollback per moduł.

§R9.3.2 Możliwe rozszerzenia post-M5

Poniższe kierunki są możliwe dopiero po ustabilizowaniu M5 (min. 6 miesięcy steady state). Każde rozszerzenie wchodzi przez pełną drabinę D + Evidence Pack.

- SYLION Federation: wiele instancji SYLION ze współdzielonym kanonem — cross-instance Council i Evidence. Governance wymaga D5 + External Review dla mechanizmu federacji.
- Multi-tenant mode: jeden SYLION serwuje wiele organizacji, z RBAC granular per tenant. Wymaga osobnego strumienia security (tenant isolation, data segregation).
- SYLION-as-a-Service: managed SYLION offering — klient przynosi własny kanon, SYLION uruchamia go jako usługę. Business model dla komercjalizacji.
- Self-Modification w Kernel: obecnie Kernel immutable (D5). W przyszłości, po 2 latach M5 steady state, AEIS może proponować Kernel changes z External Review rigor i dodatkową bramą G-KERNEL-EVO.
- AEIS Kolaboracja: AEIS jednej instancji może proponować skille dla AEIS innej (federated learning z canon guards). Wymaga protokołu cross-instance evidence exchange.

§R9.3.3 Koda Masterplanu

AUTONOMY·CANON · Masterplan ukończony — SYLION buduje siebie

Masterplan kończy 12 miesięcy pracy do M5. Po M5 SYLION jest pełnym, autonomicznym, samouczącym się systemem inżynierskim pod kanonem — gotowym do służby misji. Od tego momentu nie budujemy SYLION-a. SYLION buduje siebie, pod kanonem, z człowiekiem w pętli przy decyzjach systemowych.

§R9.4 Zakończenie

Masterplan jest kontraktem operacyjnym. Jeżeli:

- 7 artefaktów Contract Freeze zostało zamrożonych (Canonical Module Manifest, Contract Registry, Event Taxonomy, Module Lifecycle, Dependency Rules, Decision Boundaries, Security Profile Abstraction),
- 65 modułów osiągnęło stan stabilny w swoich docelowych milestone'ach,
- 5 etapów Autonomy Rollout zostało walidowanych (Observe → Propose → Sandbox → Limited prod → Full governed),
- 8 modułów Security przeszło ścieżką hot swap (shadow → dual → cutover, dev-light → prod-strict),
- Integration Assembly Phases (scaffolding → contract conformance → shadow → dual → cutover) zostały wykonane dla wszystkich strumieni —

— to SYLION AEIS jest zrealizowany.

Pełna dokumentacja kanoniczna: asset SYLION_PIPELINE_ARCHITEKTURA (PDF v3.5.1).
Niniejszy dokument: asset SYLION_AEIS_MASTERPLAN. Kontakt operacyjny: AEIS Steward + Board.

AUTONOMY·CANON · Kontrakt spełniony

Masterplan definiuje: 12 milestone'ów, 8 strumieni równoległych, 65 modułów, 5 etapów autonomii, 8 hot-swap security modułów, 12 ryzyk systemowych z mitygacjami, Evidence Pack per decyzja D2+, i pełny plan post-M5 steady state. System jest gotowy do budowy.